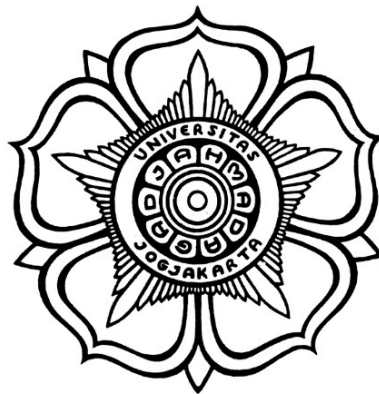


SKRIPSI

**OPTIMALISASI SISTEM *ELECTRIC BIKE-SHARING* DI YOGYAKARTA
DENGAN PENDEKATAN OPTIMISASI STOKASTIK DUA TAHAP
MENGUNAKAN *FIRE HAWK OPTIMIZER***

**(*ELECTRIC BIKE-SHARING SYSTEM OPTIMIZATION IN
YOGYAKARTA WITH TWO-STAGE STOCHASTIC OPTIMIZATION
APPROACH USING FIRE HAWK OPTIMIZER*)**



Saddam Aditya Hartanto
20/459353/PA/20014

**PROGRAM SARJANA PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

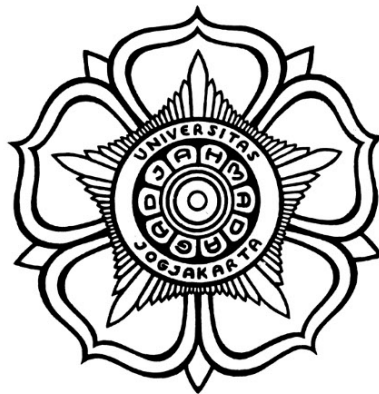
2024

SKRIPSI

**OPTIMALISASI SISTEM *ELECTRIC BIKE-SHARING* DI YOGYAKARTA
DENGAN PENDEKATAN OPTIMISASI STOKASTIK DUA TAHAP
MENGUNAKAN *FIRE HAWK OPTIMIZER***

**(*ELECTRIC BIKE-SHARING SYSTEM OPTIMIZATION IN
YOGYAKARTA WITH TWO-STAGE STOCHASTIC OPTIMIZATION
APPROACH USING FIRE HAWK OPTIMIZER*)**

Diajukan untuk memenuhi salah satu syarat memperoleh derajat
Sarjana Matematika Program Sarjana Program Studi Matematika



Saddam Aditya Hartanto
20/459353/PA/20014

**PROGRAM SARJANA PROGRAM STUDI MATEMATIKA
DEPARTEMEN MATEMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2024

HALAMAN PENGESAHAN

SKRIPSI

**OPTIMALISASI SISTEM *ELECTRIC BIKE-SHARING* DI YOGYAKARTA
DENGAN PENDEKATAN OPTIMISASI STOKASTIK DUA TAHAP
MENGUNAKAN *FIRE HAWK OPTIMIZER***

Telah dipersiapkan dan disusun oleh

Saddam Aditya Hartanto
20/459353/PA/20014

Telah dipertahankan di depan Tim Penguji
pada tanggal 17 Oktober 2024

Susunan Tim Penguji

Dr. Noorma Yulia Megawati, M.Sc.
Pembimbing Utama

Dr. Fajar Adi Kusumo, S.Si., M.Si.
Ketua Tim Penguji

Dr. Indarsih, S.Si., M.Si.
Penguji

Dr.rer.nat. Yeni Susanti, S.Si., M.Si.
Penguji

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini:

Nama : Saddam Aditya Hartanto

NIM : 20/459353/PA/20014

Tahun terdaftar : 2020

Program Studi : Program Sarjana Program Studi Matematika

Fakultas : Fakultas Matematika dan Ilmu Pengetahuan Alam

menyatakan bahwa dalam dokumen ilmiah skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 17 Oktober 2024

Materai Rp10.000,00

Saddam Aditya Hartanto

20/459353/PA/20014

Untuk penulis di masa lalu
yang telah berjuang dengan segala keterbatasan,
tidak menyerah meski dalam kesulitan.

”Try to find the argmax over the things that you’re interested in”

(Andrej Karpathy)

PRAKATA

Puji dan syukur Penulis panjatkan ke hadirat Allah Subhanahu Wa Ta'ala, karena atas rahmat dan karunia-Nya Penulis bisa menyelesaikan Skripsi yang berjudul "Optimalisasi Sistem *Electric Bike-Sharing* dengan Pendekatan Optimisasi Stokastik Dua Tahap Menggunakan *Fire Hawk Optimizer*". Shalawat serta salam Penulis curahkan kepada Nabi Muhammad Shalallahu'Alaihi Wassalam dan semoga kita mendapatkan syafaatnya kelak.

Penulisan Skripsi ini bertujuan untuk memenuhi salah satu syarat memperoleh gelar sarjana Program Studi S1 Matematika Fakultas Matematika dan Ilmu Pengetahuan Alam Universitas Gadjah Mada. Dalam penulisan dan penyusunan Skripsi ini tentunya tidak terlepas dari bantuan, bimbingan dan dukungan dari berbagai pihak. Oleh karena itu, Penulis ingin menyampaikan ucapan terima kasih kepada :

1. Prof. Dr. Eng. Kuwat Triyana, M.Si. selaku Dekan Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Gadjah Mada
2. Dr. Nanang Susyanto, S.Si., M.Sc. selaku Ketua Departemen Matematika Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Gadjah Mada.
3. Dr. Sutopo, S.Si., M.Si. selaku Ketua Program Sarjana Program Studi Matematika, Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Gadjah Mada.
4. Dr. Noorma Yulia M., S.Si., M.Sc. selaku dosen pembimbing tugas akhir yang telah membimbing penulis, memberikan ilmu, semangat, dan motivasi sehingga penulis dapat menyelesaikan skripsi ini.
5. Prof. Dr.rer.nat. Indah Emilia Wijayanti, M.Si. selaku dosen pembimbing

akademik penulis selama menimba ilmu di Program Studi Matematika, Universitas Gadjah Mada.

6. Seluruh civitas akademika Fakultas Matematika dan Ilmu Pengetahuan Alam, Universitas Gadjah Mada khususnya seluruh dosen yang telah sabar membagikan ilmunya kepada Penulis.
7. Ayah dan Ibu yang elalu memberikan dukungan moral dan motivasi kepada penulis untuk selalu melangkah ke depan, serta Adik tercinta Iqbal Prakusa Hartanto.
8. Amelia Gina Salsabila beserta keluarga yang turut memberikan dukungan maupun saran dalam penyusunan Tugas Akhir.
9. Dosen pendamping, PI-Koor, serta seluruh panitia LMNas 34 dan Semnastika 15 terkhusus Fauzan Nauvally Muhammad yang telah banyak membantu kesuksesan LMNas 34 dan Semnastika 15.
10. Madellin de Cartelle dan teman-teman Kondik yang sering menampung keluh kesah kuliah.
11. Tim *Wastewater Surveillance* yang turut mendukung segera selesainya penulisan Tugas Akhir.
12. Teman-teman mahasiswa Program Studi S1 Matematika Angkatan 2020, kakak dan adik tingkat yang membersamai dan membantu Penulis baik dalam menjalani perkuliahan maupun dalam penyusunan Skripsi ini.
13. Seluruh pihak yang terlibat dalam proses penyelesaian Skripsi ini yang tidak dapat disebutkan satu per satu.

Yogyakarta, 17 Oktober 2024

Penulis

DAFTAR ISI

HALAMAN JUDUL	i
HALAMAN PENGESAHAN	ii
HALAMAN PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
HALAMAN MOTTO	v
PRAKATA	vi
DAFTAR ISI	viii
DAFTAR TABEL	x
DAFTAR GAMBAR	xi
DAFTAR LAMBANG	xii
INTISARI	xiv
ABSTRACT	xv
I PENDAHULUAN	1
1.1. Latar Belakang Masalah	1
1.2. Tujuan dan Manfaat Penelitian	3
1.3. Tinjauan Pustaka	4
1.4. Metodologi Penelitian	7
1.5. Sistematika Penulisan	8
II DASAR TEORI	10
2.1. <i>Electric Bike-Sharing</i>	10
2.2. Graf Berarah	12
2.3. Program Linear Bilangan Bulat	14
2.4. Proses Stokastik dan Probabilitas	19
2.5. Teori Optimisasi	23
III Optimisasi Stokastik	27
3.1. Program Stokastik	27
3.1.1. Program Bilangan Bulat Stokastik	29
3.1.2. Program Stokastik Dua Tahap	30
3.2. <i>Fire Hawk Optimizer</i>	32
IV SISTEM <i>ELECTRIC BIKE-SHARING</i> DI KOTA YOGYAKARTA	52
4.1. Pemodelan Sistem <i>Electric Bike-Sharing</i>	52
4.1.1. Kompleksitas Sistem <i>Electric Bike-Sharing</i>	53
4.1.2. Pendekatan Pemodelan Stokastik Dua Tahap	53

4.1.3.	Komponen Utama Model	54
4.1.4.	Struktur Model	55
4.2.	Asumsi dan Notasi	56
4.3.	Formulasi Permasalahan	59
4.4.	Simulasi	63
4.4.1.	Generating Data	64
4.5.	Prosedur Algoritma <i>Fire Hawk Optimizer</i>	66
4.5.1.	Inisialisasi	66
4.5.2.	Evaluasi dan Seleksi	68
4.5.3.	Penentuan Teritori	69
4.5.4.	Pembaruan Posisi Elang Api	70
4.5.5.	Pembaruan Posisi Mangsa	71
4.5.6.	Tempat Aman	72
4.5.7.	Proses Iterasi dan Kriteria Penghentian	72
4.5.8.	Output Simulasi	74
V	PENUTUP	77
5.1.	Kesimpulan	77
5.2.	Saran	77
	DAFTAR PUSTAKA	79
A	DATA	83
B	LAMPIRAN PENYELESAIAN SISTEM <i>ELECTRIC BIKE-SHARING</i> SEDERHANA DENGAN LINGO	91
C	LAMPIRAN PENYELESAIAN SISTEM <i>ELECTRIC BIKE-SHARING</i> SEDERHANA DENGAN <i>FIRE HAWK OPTIMIZER</i>	93
D	LAMPIRAN SKRIP PROGRAM GENERATING DATA	100
E	LAMPIRAN SKRIP PENYELESAIAN ALGORITMA FIRE HAWK OPTIMIZER	107

DAFTAR TABEL

1.1	Data Stasiun	83
1.2	Weekday Data Skenario dan Profit	84
1.3	Saturday Data Skenario dan Profit	85
1.4	Sunday Data Skenario dan Profit	86
1.5	Weekday Data Skenario Lainnya	87
1.6	Saturday Data Skenario Lainnya	88
1.7	Sunday Data Skenario Lainnya	89
1.8	Trip yang Diterima tiap Skenario	90

DAFTAR GAMBAR

2.1	Stasiun Sepeda Listrik (MacMichael (2021))	10
2.2	Digraf	13
2.3	Graf Berbobot	14
2.4	Daerah Fisibel untuk Contoh 2.3.6	17
3.1	Foto Perilaku Elang Api Terhadap Api (Azizi dkk. (2023))	33
3.2	Skema Penentuan Elang Api dan Mangsa di Ruang Pencarian (Azizi dkk. (2023))	35
3.3	Skema Perhitungan Jarak Antara Elang Api dan Mangsa (Azizi dkk. (2023))	37
3.4	Skema Penentuan Teritori Elang Api di Ruang Pencarian (Azizi dkk. (2023))	38
3.5	Skema Proses Pembaruan Posisi Elang Api di Ruang Pencarian (Azizi dkk. (2023))	39
3.6	Skema Proses Pembaruan Posisi Mangsa di Luar Teritori Elang Api (Azizi dkk. (2023))	40
3.7	Flowchart FHO	44
4.1	Peta Lokasi Potensial Pembangunan Stasiun	66
4.2	Iterasi Algoritma Elang Api Optimizer	73
4.3	Peta Stasiun yang Dibangun dan Trip yang Diterima	74
2.1	Hasil Model Sistem <i>Electric Bike-Sharing</i> Sederhana	92

DAFTAR LAMBANG

Ω	: Himpunan skenario
K_ω	: Himpunan perjalanan dalam skenario ω
S	: Himpunan lokasi potensial stasiun
T	: Himpunan periode waktu
H	: Himpunan sepeda
π_ω	: Probabilitas skenario ω
p_k	: Kontribusi keuntungan perjalanan k
F_i	: Biaya pembangunan stasiun di lokasi i
F_{bike}	: Biaya pembelian satu sepeda
φ_i	: Biaya operasional stasiun i
ϕ	: Biaya operasional per sepeda
C_i	: Kapasitas stasiun i
W	: Anggaran yang tersedia
o_k, d_k	: Asal dan tujuan perjalanan k
s_k, e_k	: Waktu mulai dan selesai perjalanan k
b_k	: Konsumsi baterai perjalanan k
ρ	: Laju pengisian daya per unit waktu
r	: Jarak maksimum yang diizinkan antar stasiun
y_i	: Variabel keputusan apakah stasiun i dibangun
z_h	: Variabel keputusan apakah sepeda h dibeli
x_k^ω	: Variabel keputusan apakah perjalanan k diterima dalam skenario ω
$x_k^{h,\omega}$	: Variabel keputusan apakah perjalanan k ditugaskan ke sepeda h dalam skenario ω
$f_a^{h,\omega}$	: Variabel keputusan apakah sepeda h melintasi arc a dalam skenario ω

- G^ω : Graf lokasi dengan ekspansi waktu dalam skenario ω
 A_I^ω : Himpunan *edge* alokasi awal dalam skenario ω
 A_W^ω : Himpunan *edge* tunggu dalam skenario ω
 A_C^ω : Himpunan *edge* pengumpulan akhir dalam skenario ω
 A_T^ω : Himpunan *edge* perjalanan dalam skenario ω
 r^ω : Node akar dalam skenario ω
 s^ω : Node tujuan dalam skenario ω

INTISARI

OPTIMALISASI SISTEM *ELECTRIC BIKE-SHARING* DI YOGYAKARTA DENGAN PENDEKATAN OPTIMISASI STOKASTIK DUA TAHAP MENGUNAKAN *FIRE HAWK OPTIMIZER*

Oleh

Saddam Aditya Hartanto

20/459353/PA/20014

Pertumbuhan pariwisata di Yogyakarta menuntut adanya sistem transportasi yang efisien, ramah lingkungan, dan mampu memenuhi kebutuhan mobilitas pengunjung serta masyarakat lokal. Sebagai salah satu alternatif transportasi berkelanjutan, sistem *electric bike-sharing* memiliki potensi besar untuk mengurangi kemacetan dan emisi gas rumah kaca, sekaligus mendukung perkembangan sektor pariwisata yang lebih ramah lingkungan. Dalam skripsi ini dibahas mengenai optimalisasi penempatan stasiun pada sistem *electric bike-sharing* di Yogyakarta, dengan pendekatan model optimisasi stokastik dua tahap yang dibangun berdasarkan ketidakpastian permintaan yang diselesaikan menggunakan algoritma *Fire Hawk Optimizer* (FHO). Algoritma FHO dipilih karena keunggulannya dalam menghindari *local optima*, konvergensi yang cepat, dan kemampuannya menangani masalah optimisasi kompleks dengan banyak variabel dan kendala. Dalam studi ini diselesaikan permasalahan penentuan lokasi stasiun secara optimal berdasarkan keuntungan terbaik dengan mempertimbangkan pola pergerakan pengguna, aksesibilitas ke lokasi-lokasi strategis, dan fluktuasi permintaan musiman. Hasil simulasi menunjukkan bahwa penerapan algoritma FHO pada sistem *electric bike-sharing* mampu menghasilkan solusi yang optimal dalam hal pemilihan lokasi stasiun, pengalokasian trip, serta jumlah sepeda yang dibutuhkan, sehingga memberikan keuntungan maksimal bagi sistem secara keseluruhan.

ABSTRACT

ELECTRIC BIKE-SHARING SYSTEM OPTIMIZATION IN YOGYAKARTA WITH TWO-STAGE STOCHASTIC OPTIMIZATION APPROACH USING FIRE HAWK OPTIMIZER

By

Saddam Aditya Hartanto

20/459353/PA/20014

The growth of tourism in Yogyakarta demands an efficient, environmentally friendly transportation system that can meet the mobility needs of both visitors and local residents. As a sustainable transportation alternative, the electric bike-sharing system holds great potential for reducing traffic congestion and greenhouse gas emissions while supporting the development of a more eco-friendly tourism sector. This thesis discusses the optimization of station placement in the electric bike-sharing system in Yogyakarta, using a two-stage stochastic optimization model solved by the Fire Hawk Optimizer (FHO) algorithm. The study addresses the problem of determining optimal station locations by considering user movement patterns, accessibility to strategic locations, and seasonal demand fluctuations. The simulation results show that the application of the FHO algorithm in the electric bike-sharing system can produce optimal solutions in terms of station location selection, trip allocation, and the number of bikes required, thereby maximizing the overall system benefit.

BAB I

PENDAHULUAN

1.1. Latar Belakang Masalah

Perubahan iklim global telah menjadi isu mendesak karena peningkatan suhu rata-rata bumi yang disebabkan oleh akumulasi gas rumah kaca di atmosfer, sehingga mendorong negara-negara di seluruh dunia untuk mencari solusi yang lebih ramah lingkungan terutama di sektor transportasi, yang menjadi salah satu kontributor utama emisi gas rumah kaca. Intensifikasi penggunaan kendaraan bermotor pribadi telah mengakibatkan peningkatan signifikan emisi CO₂, yang berkontribusi langsung pada efek rumah kaca dan pemanasan global. Sebagai bagian dari upaya ini, negara-negara berkomitmen pada Perjanjian Paris untuk membatasi pemanasan global di bawah 2°C (UNFCCC, 2015). Sistem *electric bike-sharing* muncul sebagai solusi alternatif transportasi yang rendah emisi, dan fleksibel (Fishman, 2020).

Meskipun sepeda konvensional sudah menawarkan solusi transportasi tanpa emisi, *electric bike-sharing* memberikan keunggulan tambahan dalam hal jangkauan dan aksesibilitas. Campbell dkk. (2016) menemukan bahwa sepeda listrik lebih disukai untuk perjalanan yang lebih panjang dan oleh pengguna yang mencari pengurangan upaya fisik, terutama di daerah berbukit atau saat cuaca panas. Meskipun produksi listrik masih bertumpu pada pembangkit berbahan bakar fosil, sistem *electric bike-sharing* tetap menawarkan keunggulan dalam efisiensi energi dibandingkan kendaraan konvensional. Konsumsi energi per kilometer *electric bike* hanya sebesar 0,0075 kWh/km dibandingkan mobil listrik yang membutuhkan 0,2 kWh/km. Sistem ini menawarkan peluang untuk mengurangi emisi dari sektor transportasi, yang pada tahun 2020 menyumbang sekitar 24% dari total emisi CO₂ terkait energi di seluruh dunia (IEA, 2021). Dibandingkan dengan sistem transportasi massal seperti MRT atau LRT yang

membutuhkan infrastruktur besar dan investasi tinggi, *electric bike-sharing* menawarkan solusi yang lebih adaptif dan cepat diimplementasikan, terutama untuk perjalanan jarak pendek hingga menengah dalam kawasan bersejarah dan padat penduduk. Sistem ini juga dapat berperan sebagai penghubung "*first-mile*" dan "*last-mile*" yang melengkapi, bukan menggantikan, sistem transportasi massal yang ada, sekaligus mendukung tujuan mobilitas berkelanjutan yang tertuang dalam Sustainable Development Goals (SDGs) (UN, 2015).

Di Indonesia, Yogyakarta—sebuah kota yang dikenal sebagai pusat budaya dan pendidikan—memiliki potensi besar untuk mengadopsi sistem *electric bike-sharing* guna mendukung pariwisata, salah satu sektor ekonomi utamanya. Pada tahun 2019, lebih dari 3,7 juta wisatawan mengunjungi Yogyakarta (BPS, 2020), menciptakan tantangan bagi pemerintah kota untuk menyediakan transportasi yang efisien dan ramah lingkungan. Sistem *electric bike-sharing* dapat menjadi solusi untuk memudahkan mobilitas wisatawan di antara destinasi budaya dan sejarah yang tersebar di kota, seperti Keraton Yogyakarta, Candi Prambanan, dan Malioboro, sambil mengurangi dampak negatif terhadap lingkungan. Selain itu, penerapan sistem ini sejalan dengan visi Yogyakarta untuk menjadi kota cerdas dan berkelanjutan (Pemkot Yogyakarta, 2020), yang tidak hanya dapat meningkatkan daya tarik kota sebagai destinasi wisata ramah lingkungan, tetapi juga mendukung pertumbuhan ekonomi lokal melalui pariwisata berkelanjutan.

Namun, keberhasilan implementasi sistem *electric bike-sharing* sangat tergantung pada infrastruktur yang memadai, termasuk stasiun peminjaman sepeda dan fasilitas pengisian daya. Optimalisasi lokasi stasiun, jumlah sepeda listrik, serta rute-rute yang menguntungkan menjadi tantangan krusial yang harus dipertimbangkan. Tantangan ini diperparah oleh variasi permintaan, batasan anggaran, dan kendala operasional. Oleh karena itu, pendekatan optimasi yang efektif diperlukan untuk memaksimalkan cakupan layanan dan efisiensi sistem, sambil meminimalkan biaya investasi dan operasional. Kompleksitas masalah ini semakin meningkat ketika mempertimbangkan sifat stokastik dari permintaan dan variabel operasional lainnya.

Pendekatan pemodelan stokastik dua tahap menjadi relevan dalam konteks ini, di mana keputusan strategis jangka panjang, seperti penentuan lokasi stasiun, dilakukan pada tahap pertama, sementara keputusan operasional, seperti realokasi sepeda, dapat disesuaikan di tahap kedua berdasarkan kondisi permintaan aktual Powell, 2019. Untuk menyelesaikan masalah optimasi yang kompleks ini, penelitian ini mengusulkan penggunaan algoritma *fire hawk optimizer* (FHO), sebuah metode metaheuristik yang terinspirasi dari perilaku elang api dalam berburu dengan menyebarkan api Azizi dkk., 2023. FHO telah menunjukkan kinerja yang baik dalam menyelesaikan masalah optimasi dengan ruang pencarian yang besar dan banyak kendala, seperti yang dihadapi dalam optimasi sistem *electric bike-sharing*.

Penelitian ini bertujuan untuk memperoleh model optimasi penentuan lokasi stasiun pengisian dalam sistem *electric bike-sharing* yang mampu mengakomodasi karakteristik multi-charging dan waktu pengisian singkat yang tidak terdapat dalam model Brandstätter dkk. (2017) untuk sistem *car-sharing* listrik. Pengembangan model dilakukan dengan memodifikasi asumsi kapasitas pengisian daya yang memungkinkan setiap stasiun melayani banyak sepeda listrik per hari Chen dkk., 2020, serta menyesuaikan kendala dan parameter untuk karakteristik khusus sistem *electric bike-sharing*. Dengan mempertimbangkan ketidakpastian permintaan dan kendala operasional, model yang dikembangkan mengintegrasikan pemodelan stokastik dua tahap dan algoritma FHO untuk menghasilkan solusi yang tidak hanya optimal secara teoritis, tetapi juga adaptif dan aplikatif dalam menghadapi dinamika sistem di dunia nyata. Kontribusi dari penelitian ini tidak hanya akan membantu perencanaan dan implementasi sistem berbagi sepeda listrik yang lebih efisien dan berkelanjutan, tetapi juga dapat menjadi kerangka kerja yang dapat diterapkan pada sistem transportasi perkotaan lainnya yang menghadapi ketidakpastian serupa.

1.2. Tujuan dan Manfaat Penelitian

Penelitian ini memiliki beberapa tujuan utama:

1. Memperoleh model optimisasi stokastik dua tahap yang mampu merepresentasikan ketidakpastian permintaan dalam sistem *electric bike-sharing*.
2. Mendapatkan solusi optimal untuk penempatan stasiun, alokasi sepeda, dan penerimaan trip dalam sistem *electric bike-sharing* di Yogyakarta menggunakan algoritma *fire hawk optimizer* (Azizi dkk., 2023).

Penelitian ini diharapkan memberikan beberapa manfaat penting diantaranya:

1. Kontribusi teoritis dalam pengembangan model stokastik untuk sistem transportasi perkotaan untuk pemangku kepentingan seperti pemerintah daerah.
2. Panduan bagi perencana kota dan operator sistem *bike-sharing* dalam mengoptimalkan penempatan stasiun pengisian daya dan meningkatkan aksesibilitas sistem *electric bike-sharing* bagi pengguna (Fishman, 2020).
3. Kontribusi pada pengembangan solusi transportasi perkotaan yang lebih berkelanjutan dan ramah lingkungan.

1.3. Tinjauan Pustaka

Penelitian ini dibangun di atas beberapa aliran literatur yang saling terkait, mencakup sistem *electric bike-sharing*, optimisasi lokasi stasiun, pemodelan stokastik, dan algoritma optimisasi. O'Brien dkk. (2014) menggarisbawahi pentingnya analisis data dalam mengembangkan sistem transportasi berkelanjutan, sementara Fishman (2020) memberikan tinjauan komprehensif tentang perkembangan terbaru dalam sistem *bike-sharing*, termasuk tren elektrifikasi yang menunjukkan potensi untuk memperluas jangkauan perjalanan dan meningkatkan aksesibilitas.

Dalam kebijakan dan dampak lingkungan, penelitian ini sejalan dengan komitmen internasional yang tercantum dalam UNFCCC (2015) dan agenda

pembangunan berkelanjutan UN (2015). IEA (2021) menunjukkan bahwa elektrifikasi transportasi menjadi tren global, dengan MacMichael (2021) memberikan perspektif tentang perkembangan regulasi terkini. Di tingkat lokal, BPS (2020) dan Pemkot Yogyakarta (2020) menyediakan kerangka kebijakan untuk implementasi sistem transportasi berkelanjutan.

Beberapa studi telah menganalisis perilaku pengguna dan preferensi moda transportasi. Campbell dkk. (2016) meneliti faktor-faktor yang mempengaruhi pilihan antara sepeda konvensional dan sepeda listrik di Beijing, menemukan bahwa sepeda listrik lebih disukai untuk perjalanan jarak jauh. Ling dkk. (2017) mempelajari transisi dari pengguna *e-bike* ke pengguna mobil di Cina, sementara Shaheen et al. (2019) memberikan perspektif komprehensif tentang strategi berbagi dalam berbagai moda transportasi mikro. Langford dkk. (2013) dan Fyhri dan Fearnley (2017) menganalisis dampak kesehatan dan perilaku penggunaan, menunjukkan bahwa *e-bikes* dapat meningkatkan frekuensi bersepeda terutama di daerah berbukit.

Optimisasi lokasi stasiun merupakan aspek krusial yang telah mendapat perhatian signifikan dalam literatur. Brandstätter dkk. (2017) mengembangkan model stokastik untuk sistem *car-sharing* dengan asumsi satu stasiun hanya dapat melayani satu kendaraan dalam satu periode waktu. Model yang dikembangkan dalam penelitian ini memperluas pendekatan tersebut dengan mengadaptasi konsep *multi-charging capability* dari Chen dkk. (2020) yang memungkinkan satu stasiun melayani beberapa sepeda secara simultan. Zhu dkk. (2016) memberikan kerangka dasar untuk masalah lokasi stasiun pengisian yang dapat diadaptasi untuk sepeda listrik. Sun dkk. (2018) menekankan pentingnya faktor lingkungan dalam penentuan lokasi stasiun, sedangkan Ma dkk. (2020) menunjukkan dampak penempatan stasiun terhadap pergeseran moda transportasi.

Fondasi teoritis untuk optimisasi stokastik dan pemodelan sistem dibangun dari berbagai sumber. Birge dan Louveaux (2011) dan Powell (2019) menyediakan kerangka kerja untuk program stokastik, sementara Wilson (1996) memberikan dasar teori graf untuk pemodelan jaringan. Billingsley (1995), Ross (2014), dan

Grimmett dan Stirzaker (2020) menyediakan landasan probabilitas dan proses stokastik. Haight (1967) dan Casella dan Berger (2002) memberikan dasar statistik untuk analisis data.

Aspek komputasi dan algoritma optimisasi mendapat dukungan dari beberapa literatur kunci. Boyd dan Vandenberghe (2004) dan Nocedal dan Wright (2006) menyediakan dasar optimisasi numerik, sedangkan Winston dan Goldberg (2004) dan Hastie dkk. (2009) memberikan kerangka analisis operasional. Talbi (2009) dan Glover (1986) membahas pendekatan metaheuristik, yang menjadi dasar pengembangan algoritma modern seperti Fire Hawk Optimizer oleh Azizi dkk. (2023). Implementasi komputasional didukung oleh McKinney (2011) untuk analisis data dan Hunter (2007) untuk visualisasi.

Dari perspektif industri dan implementasi, laporan Bikeplus (2016) tentang program *electric bike-sharing* memberikan wawasan praktis tentang tantangan dan peluang dalam pengembangan sistem operasional. Holland (1992) memberikan kerangka kerja adaptif yang dapat diterapkan dalam optimisasi sistem dinamis seperti *bike-sharing*, sementara Ross (1996) menyediakan dasar untuk pemodelan proses stokastik dalam sistem transportasi.

Meskipun literatur yang ada memberikan dasar yang kuat untuk pemahaman tentang *electric bike-sharing*, masih terdapat beberapa kesenjangan penelitian yang signifikan karena mayoritas penelitian sebelumnya fokus pada sistem *bike-sharing* konvensional atau adaptasi langsung dari model *car-sharing* listrik yang memiliki karakteristik operasional berbeda.

Selain itu, penggunaan algoritma *fire hawk optimizer* yang baru diperkenalkan oleh Azizi dkk. (2023) dalam optimisasi sistem transportasi masih sangat terbatas. Meskipun algoritma ini menunjukkan performa yang menjanjikan dalam masalah optimisasi kompleks, implementasinya dalam optimisasi sistem *electric bike-sharing* belum pernah dilakukan sebelumnya. Penelitian ini bertujuan untuk mengisi kesenjangan tersebut dengan mengembangkan model stokastik yang secara spesifik disesuaikan untuk karakteristik *electric bike-sharing* dan menjadi yang pertama dalam mengaplikasikan algoritma *fire hawk optimizer*

untuk optimisasi lokasi stasiun dalam konteks ini.

1.4. Metodologi Penelitian

Penelitian ini mengadopsi pendekatan multi-fase yang menggabungkan analisis teoretis, pemodelan matematis, dan simulasi komputasi. Metodologi penelitian terdiri dari beberapa fase utama:

1. Pengembangan Model Stokastik Dua Tahap.

- (a) Memperluas model Zhu dkk. (2016) dengan menambahkan komponen ketidakpastian permintaan berdasarkan variasi hari (kerja, Sabtu, Minggu) dan mempertimbangkan karakteristik khusus sepeda listrik seperti waktu pengisian daya dan kapasitas baterai yang berbeda dari kendaraan listrik lainnya.
- (b) Menyesuaikan asumsi kapasitas pengisian daya dari model Chen dkk. (2020) untuk mengakomodasi karakteristik stasiun pengisian sepeda listrik yang dapat melayani *multiple charging* secara simultan, berbeda dengan sistem *car-sharing* yang umumnya terbatas pada *single charging*.
- (c) Mengonstruksi model dengan mempertimbangkan kondisi nyata seperti batasan jarak antar stasiun, karakteristik geografis Yogyakarta, dan pola pergerakan wisatawan.

2. Implementasi algoritma *fire hawk optimizer* (FHO).

Algoritma FHO dipilih karena kemampuannya dalam mencari solusi optimal pada permasalahan dengan dimensi tinggi dan banyak kendala, serta efektivitasnya dalam menemukan solusi global optimal dibandingkan algoritma metaheuristik lainnya (Azizi dkk. (2023)). FHO juga menunjukkan performa yang baik dalam menghindari *local optima* dan konvergensi prematur.

3. Memodelkan sistem *electric bike-sharing*.

4. Implementasi dan Simulasi Komputasi.

- (a) Python dipilih sebagai bahasa pemrograman karena bersifat open source serta karena kemampuannya dalam pemrosesan data yang kompleks dan tersedianya berbagai library pendukung untuk optimisasi.
- (b) NumPy, SciPy, dan Pandas (McKinney (2011)) digunakan untuk manipulasi data dan komputasi numerik yang efisien, terutama dalam penanganan matriks besar dan operasi matematika yang dibutuhkan dalam algoritma optimisasi.
- (c) Matplotlib (Hunter (2007)) dan Folium dipilih untuk visualisasi karena kemampuannya dalam menghasilkan grafik berkualitas tinggi dan peta interaktif yang memudahkan analisis hasil optimisasi lokasi stasiun.

Metodologi ini dirancang untuk menghasilkan model optimisasi yang robust untuk penentuan lokasi stasiun pengisian daya *electric bike-sharing*, serta memberikan wawasan praktis yang berharga untuk pengembangan dan pengelolaan sistem tersebut.

1.5. Sistematika Penulisan

Sistematika penulisan skripsi ini terdiri dari lima bab utama yang disusun sebagai berikut:

1. BAB I PENDAHULUAN

Bab ini berisi gambaran umum tentang penelitian, meliputi latar belakang masalah, rumusan masalah, tujuan dan manfaat penelitian, tinjauan pustaka, dan sistematika penulisan.

2. BAB II DASAR TEORI

Pada bab ini disajikan landasan teoritis yang digunakan dalam penelitian. Topik-topik yang dibahas meliputi konsep *electric bike-sharing*, graf, proses stokastik, dan algoritma heuristik.

3. BAB III OPTIMISASI STOKASTIK

Pada bab ini dijelaskan pendekatan penelitian yang digunakan, termasuk

pengembangan model stokastik dua tahap, implementasi algoritma *fire hawk optimizer*, pengembangan skenario uji, dan metode validasi model.

4. **BAB IV SIMULASI**

Pada bab ini akan dibahas mengenai pembentukan model, termasuk asumsi dan notasi, fungsi tujuan, serta kendala. Bab ini juga menyajikan hasil simulasi numerik menggunakan algoritma *fire hawk optimizer*, disertai dengan analisis dan interpretasi hasil.

5. **BAB V PENUTUP**

Bab ini berisi kesimpulan dari penelitian yang telah dilakukan, serta saran untuk penelitian selanjutnya dalam bidang optimisasi sistem *electric bike-sharing*.

BAB II

DASAR TEORI

Pada bab ini dibahas tentang konsep yang mendasari pembahasan di bab berikutnya. Konsep dasar yang dibahas pada bab ini, yaitu konsep *electric bike-sharing*, graf, proses stokastik, dan algoritma heuristik.

2.1. *Electric Bike-Sharing*

Sistem *electric bike-sharing* adalah salah satu bentuk transportasi berbagi (*shared mobility*) di mana pengguna dapat menyewa sepeda listrik untuk perjalanan singkat di dalam kota.



Gambar 2.1 Stasiun Sepeda Listrik (MacMichael (2021))

Sistem *electric bike-sharing* merupakan evolusi dari sistem *bike-sharing* konvensional yang menggabungkan keuntungan dari mobilitas aktif dengan teknologi *electric assist* untuk memperluas jangkauan dan aksesibilitas layanan (Fishman, 2020). Sepeda listrik dalam sistem ini umumnya dilengkapi dengan motor listrik yang membantu pengendara, terutama saat menghadapi tanjakan atau

perjalanan jarak jauh, sehingga memperluas segmen pengguna potensial (Campbell dkk., 2016). Komponen kunci dari sistem *electric bike-sharing* meliputi:

1. Sepeda Listrik

Umumnya menggunakan sepeda dengan *pedal-assist electric motor* dengan baterai yang dapat diisi ulang (Ling dkk., 2017).

2. Stasiun Pengisian

Infrastruktur yang memungkinkan pengguna untuk mengambil dan mengembalikan sepeda serta mengisi daya baterai (Sun dkk., 2018).

3. Sistem Akses dan Pembayaran

Mekanisme yang memungkinkan pengguna untuk mendaftar, membuka kunci sepeda, dan membayar layanan, biasanya melalui aplikasi smartphone atau kartu RFID (Shaheen et al., 2019).

Dibandingkan dengan sistem *bike-sharing* konvensional, *electric bike-sharing* menawarkan beberapa keunggulan:

1. Dengan kemampuan sepeda listrik untuk menempuh jarak tempuh yang lebih jauh, memungkinkan pengguna untuk melakukan perjalanan yang lebih panjang (Langford dkk., 2013).
2. Potensi untuk menggantikan perjalanan mobil jarak pendek hingga menengah yang berkontribusi pada pengurangan emisi (Bikeplus, 2016).

Namun, sistem ini juga menghadapi tantangan di antaranya:

1. Kebutuhan infrastruktur pengisian daya yang lebih kompleks terutama pada manajemen energi yang digunakan (Sun dkk., 2018).
2. Biaya operasional yang lebih tinggi karena harga sepeda listrik dan pemeliharaan.

3. Manajemen baterai, termasuk pengisian daya dan penggantian (Ling dkk., 2017).

Optimalisasi lokasi stasiun pengisian daya menjadi krusial, mengingat biaya infrastruktur yang signifikan dan dampaknya terhadap aksesibilitas sistem. Model stokastik, seperti yang diusulkan oleh Brandstätter dkk. (2017), memungkinkan perencanaan untuk mempertimbangkan ketidakpastian permintaan dan variabilitas operasional dalam menentukan lokasi stasiun yang optimal.

Untuk merepresentasikan sistem *bike-sharing* ini dibentuk kerangka kerja dengan menggunakan teori graf. Kerangka kerja ini dapat memodelkan konektivitas jaringan stasiun dan rute perjalanan sistem *bike-sharing* dengan merepresentasikan stasiun sebagai titik dan rute trip-trip sebagai busur.

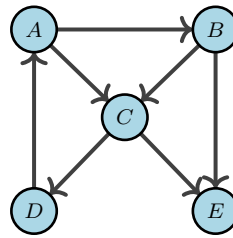
2.2. Graf Berarah

Berikut diberikan definisi graf berarah.

Definisi 2.2.1 *digraf (directed graph) G adalah pasangan himpunan tak kosong berhingga $V(G)$ dengan elemen-elemennya disebut titik (vertex) dan himpunan berhingga $A(G)$ dengan elemennya merupakan pasangan berurutan (u, v) dari dua titik berbeda u dan v di $V(G)$ yang disebut busur (arc). Pasangan himpunan titik $V(G)$ dan himpunan busur $A(G)$ dinotasikan dengan $G = (V(G), A(G))$. Untuk pasangan berurutan (u, v) , titik u disebut titik awal dan titik v disebut titik akhir dari busur tersebut.*

Sebarang digraf dapat divisualisasikan dengan elemen di $V(G)$ digambarkan dalam bentuk titik/lingkaran dan elemen (u, v) di $A(G)$ digambarkan sebagai busur dari u ke v . Pada tulisan ini, penyebutan digraf akan disingkat dengan graf saja.

Contoh 2.2.2 Diberikan graf seperti berikut.



Gambar 2.2 Digraf

Diperhatikan pada Gambar 2.3 terdapat digraf dengan himpunan titik yang terdiri dari $\{A, B, C, D, E\}$ dan himpunan busur yang terdiri dari $\{(A, B), (A, C), (D, A), (C, E), (B, C), (C, D), (E, B)\}$ dengan arah dalam busur ditunjukkan oleh panah.

Definisi 2.2.3 Diberikan graf $G = (V(G), A(G))$. Walk didefinisikan sebagai barisan berhingga dari titik-titik $v_1, v_2, \dots, v_m \in V(G)$ yang setiap dua titik berurutan terhubung oleh busur dan dinotasikan dengan $v_1 \rightarrow v_2 \rightarrow v_3 \rightarrow \dots \rightarrow v_m$. Panjang walk didefinisikan sebagai banyaknya busur yang dilewati dalam walk tersebut. Selanjutnya, path adalah walk dengan semua titik dan busur yang dilewati berbeda.

Untuk memperjelas perbedaan antara *walk* dan *path*, disajikan contoh berikut.

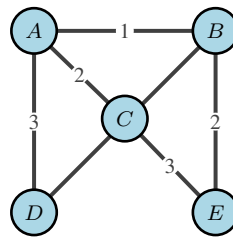
Contoh 2.2.4 Pada Gambar 2.2 dapat dibentuk walk $C \rightarrow D \rightarrow A \rightarrow B \rightarrow C \rightarrow E$ yang merupakan walk dari C menuju E dengan panjang enam berdasarkan jumlah busur dalam walk. Dapat dibentuk juga walk $C \rightarrow D \rightarrow A \rightarrow B \rightarrow E$ yang merupakan path karena titik dan busur dari walk titik C menuju titik E masing-masing dilewati tepat sekali.

Dalam banyak aplikasi, termasuk dalam pemodelan sistem *electric bike-sharing*, seringkali diperlukan pemberian nilai atau bobot pada busur dalam graf. Konsep ini dikenal sebagai graf berbobot yang didefinisikan sebagai berikut.

Definisi 2.2.5 (Wilson (1996)) Bobot dari graf adalah nilai numerik yang diberikan untuk setiap busur pada graf.

Bobot ini sering digunakan untuk merepresentasikan jarak, biaya, waktu, atau properti lain yang relevan dengan hubungan antara dua titik dalam graf. Untuk visualisasi konsep graf berbobot, disajikan contoh berikut.

Contoh 2.2.6 Diberikan diagram graf seperti berikut.



Gambar 2.3 Graf Berbobot

Diperhatikan pada Gambar 2.3, terdapat nilai nonnegatif, yaitu 1, 2, 3 secara acak di tiap busur. Nilai-nilai tersebut adalah bobot dari graf.

Pada permasalahan stokastik lokasi stasiun pengisian daya untuk *electric bike-sharing*, titik potensial pembangunan stasiun pengisian daya dapat dipandang sebagai titik. Trip tiap skenario merupakan busur, rute perjalanan dalam skenario adalah *path*, kontribusi profit tiap trip merepresentasikan bobot, dan digraf merupakan arah dari trip tiap skenario (Brandstätter dkk. (2017)).

2.3. Program Linear Bilangan Bulat

Program linear bilangan bulat merupakan perluasan dari program linear yang memiliki batasan tambahan pada variabel keputusannya. Sebelum membahas lebih lanjut tentang program linear bilangan bulat, penting untuk memahami konsep dasar fungsi linear dan pertidaksamaan linear.

Definisi 2.3.1 (Winston dan Goldberg (2004)) Suatu fungsi $f : \mathbb{R}^n \rightarrow \mathbb{R}$ disebut sebagai **fungsi linear** jika dan hanya jika terdapat vektor $\mathbf{c} = (c_1, c_2, \dots, c_n) \in \mathbb{R}^n$ sehingga untuk setiap $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, berlaku

$$f(\mathbf{x}) = \mathbf{c}^T \mathbf{x} = c_1 x_1 + c_2 x_2 + \dots + c_n x_n.$$

Untuk memperjelas konsep fungsi linear, berikut disajikan contoh fungsi linear dan bukan fungsi linear.

Contoh 2.3.2 Fungsi $f(x_1, x_2) = 3x_1 + 2x_2$ merupakan fungsi linear dari x_1 dan x_2 karena fungsi tersebut dapat ditulis dalam bentuk $c_1x_1 + c_2x_2$ dengan $c_1 = 3$ dan $c_2 = 2$. Sebaliknya, fungsi $f(x_1, x_2) = x_1x_2^2$ bukan merupakan fungsi linear dari x_1 dan x_2 karena fungsi tersebut mengandung perkalian antara variabel (x_1x_2) dan variabel berpangkat (x_2^2) yang tidak sesuai dengan definisi fungsi linear.

Selanjutnya, konsep pertidaksamaan linear juga penting dalam pemrograman linear. Pertidaksamaan linear didefinisikan sebagai berikut.

Definisi 2.3.3 (Winston dan Goldberg (2004)) *Diberikan fungsi linear $f : \mathbb{R}^n \rightarrow \mathbb{R}$ dan $b \in \mathbb{R}$, **pertidaksamaan linear** dapat dinyatakan dalam bentuk $f(\mathbf{x}) \leq b$ atau $f(\mathbf{x}) \geq b$, dengan $\mathbf{x} \in \mathbb{R}^n$*

Untuk lebih memahami konsep pertidaksamaan linear, diberikan contoh berikut.

Contoh 2.3.4 Pertidaksamaan $x_1 + 4x_2 \leq 3$ dan $3x_1 + x_2 \geq 4$ merupakan pertidaksamaan linear. Sebaliknya, $x_1^3x_2 \leq 5$ bukan merupakan pertidaksamaan linear karena mengandung x_1^3 dan perkalian antara variabel x_1 dan x_2 .

Setelah memahami konsep fungsi linear dan pertidaksamaan linear, dapat didefinisikan masalah program linear.

Definisi 2.3.5 (Winston dan Goldberg (2004)) *Masalah program linear (LP) adalah permasalahan optimisasi dengan kriteria berikut:*

1. *Memaksimalkan (atau meminimalkan) fungsi linear dari variabel keputusan. Fungsi tujuan ini disebut dengan fungsi objektif.*
2. *Nilai dari variabel keputusan harus memenuhi kendala. Tiap kendala berbentuk persamaan linear dan atau pertidaksamaan linear.*

3. *Batasan untuk tiap variabel. Untuk setiap variabel x didefinisikan batasan secara spesifik, yaitu x_i nonnegatif ($x_i \geq 0$) untuk setiap $i = 1, 2, \dots, n$ atau tidak diberi batasan.*

*Bentuk rumusan masalah **program linear (LP)** dapat dinyatakan dalam bentuk*

$$\begin{aligned} \max / \min \quad & \mathbf{c}^T \mathbf{x}, \\ \text{dengan kendala} \quad & \mathbf{Ax} \leq \mathbf{b}, \\ & \mathbf{x} \geq 0, \end{aligned}$$

dengan $\mathbf{x} \in \mathbb{R}^n$ adalah vektor variabel keputusan, $\mathbf{c} \in \mathbb{R}^n$ adalah vektor koefisien fungsi tujuan, $\mathbf{A} \in \mathbb{R}^{m \times n}$ adalah matriks koefisien kendala, dan $\mathbf{b} \in \mathbb{R}^m$ adalah vektor batas kanan kendala.

Lebih lanjut diberikan contoh masalah program linear berikut.

Contoh 2.3.6 Berikut adalah contoh program linear:

$$\begin{aligned} \text{Maksimalkan} \quad & z = 3x_1 + 2x_2, \\ \text{dengan kendala} \quad & x_1 + x_2 \leq 10, \\ & 2x_1 + x_2 \leq 15, \\ & x_1, x_2 \geq 0. \end{aligned}$$

Dalam contoh ini:

1. Tujuan masalah di atas adalah memaksimalkan fungsi objektif $z = 3x_1 + 2x_2$ yang akan dimaksimalkan.
2. Terdapat dua kendala: $x_1 + x_2 \leq 10$ dan $2x_1 + x_2 \leq 15$.
3. Terdapat batasan nonnegatif pada variabel: $x_1, x_2 \geq 0$.

Dalam program linear, konsep daerah fisibel sangat penting. Daerah fisibel didefinisikan sebagai berikut.

Definisi 2.3.7 (Winston dan Goldberg (2004)) *Daerah fisibel masalah program linear adalah himpunan dari semua titik yang memenuhi semua kendala dan batasan variabel. Daerah fisibel dapat dinyatakan sebagai berikut*

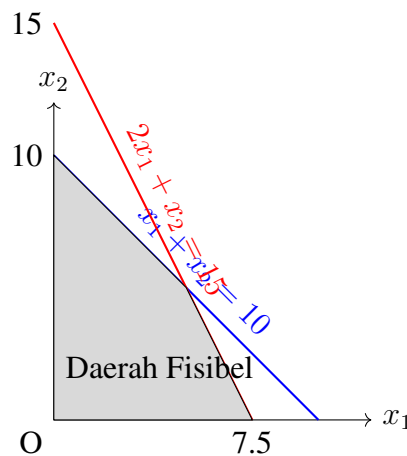
$$\mathcal{F} = \{x \in \mathbb{R}^n : Ax \leq b, x \geq 0\}.$$

Untuk memvisualisasikan konsep daerah fisibel, diperhatikan contoh berikut.

Contoh 2.3.8 Pada Contoh 2.3.6, daerah fisibel adalah area yang dibatasi oleh:

1. garis $x_1 + x_2 = 10$,
2. garis $2x_1 + x_2 = 15$,
3. sumbu x_1 (karena $x_2 \geq 0$), dan
4. sumbu x_2 (karena $x_1 \geq 0$).

Daerah ini membentuk suatu poligon dalam bidang x_1x_2 .



Gambar 2.4 Daerah Fisibel untuk Contoh 2.3.6

Setelah memahami daerah fisibel, dapat didefinisikan solusi optimal dari program linear.

Definisi 2.3.9 (Winston dan Goldberg (2004)) *Untuk masalah memaksimalkan, solusi optimal $\hat{x}$ dari program linear adalah*

$$\hat{x} = \arg \max_{x \in \mathcal{F}} c^T x$$

dengan $\mathcal{F}$ adalah daerah fisibel dan $\arg \max$ (argumen maksimum) menunjukkan nilai x dalam $\mathcal{F}$ yang menghasilkan nilai maksimum dari fungsi objektif $c^T x$.

Untuk lebih memahami konsep solusi optimal, diperhatikan contoh berikut.

Contoh 2.3.10 Dalam program linear pada contoh sebelumnya, solusi optimal akan berada pada salah satu titik sudut dari daerah fisibel. Misalkan solusi optimalnya adalah $(x_1, x_2) = (5, 5)$, maka nilai fungsi objektif maksimalnya adalah $z = 3(5) + 2(5) = 25$.

Sistem *electric bike-sharing* erat berkaitan dengan program bilangan bulat karena sistem ini dibentuk dengan menentukan variabel-variabel keputusan dalam bilangan bulat seperti pembangunan stasiun potensial maupun jumlah armada sepeda yang dibutuhkan. Selanjutnya, didefinisikan program bilangan bulat.

Definisi 2.3.11 (Winston dan Goldberg (2004)) *Masalah **program bilangan bulat (IP)** adalah masalah program linear dengan tambahan kendala bahwa beberapa atau semua variabel harus berupa bilangan bulat. Bentuk umum masalah program bilangan bulat adalah*

$$\begin{aligned} & \max / \min \quad c^T x, \\ & \text{dengan kendala} \quad Ax \leq b, \\ & \quad x \geq 0, \\ & \quad x_i \in \mathbb{Z}, \quad \text{untuk beberapa atau semua } i. \end{aligned}$$

Untuk lebih memahami berbagai jenis program bilangan bulat, diperhatikan contoh berikut.

Contoh 2.3.12 Berikut adalah contoh program bilangan bulat murni.

$$\begin{aligned} \text{Maksimalkan} \quad & z = 3x_1 + 2x_2, \\ \text{dengan kendala} \quad & x_1 + x_2 \leq 6, \\ & x_1, x_2 \geq 0, \\ & x_1, x_2 \in \mathbb{Z}. \end{aligned}$$

Contoh program bilangan bulat campuran.

$$\begin{aligned} \text{Maksimalkan} \quad & z = 3x_1 + 2x_2, \\ \text{dengan kendala} \quad & x_1 + x_2 \leq 6, \\ & x_1, x_2 \geq 0, \\ & x_1 \in \mathbb{Z}. \end{aligned}$$

Contoh IP 0-1.

$$\begin{aligned} \text{Maksimalkan} \quad & z = x_1 - x_2, \\ \text{dengan kendala} \quad & x_1 + 2x_2 \leq 2 \\ & 2x_1 - x_2 \leq 1, \\ & x_1, x_2 = 0 \text{ atau } 1. \end{aligned}$$

Dalam konteks *electric bike-sharing*, program bilangan bulat dapat digunakan untuk menentukan jumlah stasiun yang harus dibangun (variabel bilangan bulat) dan kapasitas masing-masing stasiun (variabel bilangan bulat), sambil memaksimalkan cakupan layanan atau meminimalkan biaya total.

2.4. Proses Stokastik dan Probabilitas

Dalam kejadian apapun di dunia ini dapat dibedakan menjadi kepastian, ketidakpastian, dan kemustahilan. Misalkan, pasti semua sepeda listrik membutuhkan baterai untuk beroperasi, kemungkinan orang lebih menggunakan sepeda listrik pada hari kerja, ataupun tidak mungkin sepeda listrik berubah menjadi mobil dalam perjalanan. Berdasarkan permisalan sederhana ini, dapat didefinisikan konsep *randomness*.

Konsep *randomness* erat kaitannya dengan probabilitas yang merupakan alat matematis untuk mengukur dan menganalisis kejadian-kejadian acak.

Probabilitas memungkinkan untuk mengkuantifikasi ketidakpastian dan membuat prediksi dalam situasi yang tidak deterministik. Untuk memahami probabilitas dengan lebih baik, perlu diketahui beberapa konsep dasar (Ross (2014)).

Untuk memahami bagaimana probabilitas berlaku, perlu didefinisikan ruang sampel.

Definisi 2.4.1 (Ross (2014)) *Ruang sampel (Ω) adalah himpunan semua hasil yang mungkin dari suatu eksperimen acak.*

Setelah memahami ruang sampel, dapat didefinisikan fungsi probabilitas yang memetakan kejadian dalam ruang sampel ke nilai probabilitasnya.

Definisi 2.4.2 (Billingsley (1995)) *Probabilitas P adalah fungsi yang memetakan setiap kejadian A dalam ruang sampel Ω ke suatu bilangan real antara 0 dan 1 yang memenuhi sifat-sifat berikut :*

1. $P(A) \geq 0$ untuk setiap kejadian A .
2. $P(\Omega) = 1$.
3. Untuk kejadian-kejadian yang saling lepas $A_1, A_2, \dots$, berlaku $P(\bigcup_{i=1}^{\infty} A_i) = \sum_{i=1}^{\infty} P(A_i)$.

Dalam program stokastik yang akan dijelaskan kemudian, erat kaitannya dengan variabel-variabel acak. Berikut didefinisikan variabel acak.

Definisi 2.4.3 (Ross (2014)) *Variabel acak adalah fungsi yang memetakan setiap hasil dalam ruang sampel ke suatu bilangan real, dinotasikan dengan $X(c) = x$ dengan $c \in \Omega$ dan $x \in \mathbb{R}$*

Variabel acak terbagi menjadi dua yaitu variabel acak diskrit dan variabel acak kontinu, variabel acak diskrit akan didefinisikan sebagai berikut.

Definisi 2.4.4 (Casella dan Berger (2002)) *Variabel acak diskrit adalah variabel acak yang hanya dapat mengambil dalam himpunan terhingga. Fungsi probabilitas*

dari variabel acak diskrit X didefinisikan sebagai $p(x) = P(X = x)$ untuk setiap nilai x yang mungkin dari X , dan $p(x)$ memenuhi sifat:

1. $p(x) \geq 0$ untuk semua x ,
2. $\sum_x p(x) = 1$.

Selanjutnya variabel acak kontinu akan didefinisikan sebagai berikut.

Definisi 2.4.5 (Ross (1996)) Variabel acak kontinu adalah variabel acak yang dapat mengambil nilai pada suatu interval bilangan real. Variabel acak kontinu X ditandai dengan fungsi kepadatan probabilitas $f(x)$, dengan probabilitas X berada pada interval $[a, b]$ didefinisikan

$$P(a \leq X \leq b) = \int_a^b f(x)dx,$$

dan $f(x)$ memenuhi sifat:

1. $f(x) \geq 0$ untuk semua x ,
2. $\int_{-\infty}^{\infty} f(x)dx = 1$.

Salah satu cara untuk mendapatkan makna dari variabel-variabel acak ini adalah dengan mencari nilai ekspektasinya yang umum dalam formulasi program stokastik. Berikut didefinisikan nilai ekspektasi.

Definisi 2.4.6 (Casella dan Berger (2002)) Nilai harapan atau ekspektasi dari variabel acak X dengan fungsi probabilitas $p(x)$ didefinisikan sebagai

$$E[X] = \sum_x x \cdot p(x), \quad X \text{ variabel acak diskrit},$$

dan

$$E[X] = \int x \cdot f(x) dx, \quad X \text{ variabel acak kontinu}.$$

Dalam konteks permasalahan stokastik lokasi stasiun pengisian daya untuk *electric bike-sharing*, konsep probabilitas dan nilai harapan sangat penting. Permintaan pengguna sepeda listrik dapat dianggap sebagai variabel acak dan dapat digunakan nilai harapan untuk memperkirakan permintaan rata-rata pada setiap stasiun.

Definisi 2.4.7 (Ross (1996)) *Proses stokastik adalah kumpulan variabel acak $X(t)$, $t \in T$ yang diindekskan oleh parameter t yang merepresentasikan waktu.*

Dalam sistem *electric bike-sharing*, permintaan sepeda listrik di setiap stasiun dapat dimodelkan sebagai proses stokastik yang mana permintaan bervariasi seiring waktu dan dipengaruhi oleh berbagai faktor acak seperti cuaca, hari dalam seminggu, atau acara khusus di sekitar lokasi sebagai contoh berikut.

Contoh 2.4.8 Dalam sistem *electric bike-sharing*, permintaan sepeda listrik di setiap stasiun dapat dimodelkan sebagai proses stokastik. Misalkan $X(t)$ mewakili jumlah permintaan sepeda listrik pada waktu t di suatu stasiun tertentu. Proses stokastik ini mungkin terlihat seperti berikut.

1. $X(9 : 00) = 5$ (5 permintaan pada pukul 9:00),
2. $X(10 : 00) = 8$ (8 permintaan pada pukul 10:00), dan
3. $X(11 : 00) = 3$ (3 permintaan pada pukul 11:00).

Dalam pemodelan kedatangan pelanggan atau permintaan, sering digunakan distribusi Poisson. Distribusi Poisson sering digunakan untuk memodelkan kedatangan pelanggan dalam sistem antrian yang dapat diterapkan dalam konteks permintaan sepeda listrik di stasiun-stasiun pengisian daya (Grimmett dan Stirzaker (2020)).

Definisi 2.4.9 (Haight (1967)) *Distribusi Poisson adalah distribusi probabilitas diskret yang mengekspresikan probabilitas sejumlah kejadian terjadi dalam interval waktu atau ruang tertentu jika kejadian-kejadian tersebut terjadi dengan*

rata-rata yang diketahui dan independen terhadap waktu sejak kejadian terakhir. Fungsi massa probabilitas dari distribusi Poisson didefinisikan dengan

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}, k = 0, 1, 2, \dots$$

Selanjutnya, akan diberikan contoh dari penerapan distribusi Poisson.

Contoh 2.4.10 Misalkan rata-rata kedatangan pelanggan di suatu stasiun *electric bike-sharing* adalah empat orang per jam. Jika kedatangan pelanggan mengikuti distribusi Poisson, maka probabilitas tepat k pelanggan datang dalam satu jam diberikan oleh

$$P(X = k) = \frac{e^{-\lambda} \lambda^k}{k!}$$

dengan $\lambda = 4$ (rata-rata kedatangan per jam) dan k adalah jumlah kedatangan yang ingin dihitung probabilitasnya. Oleh karena itu, probabilitas tepat tiga pelanggan datang dalam satu jam adalah

$$P(X = 3) = \frac{e^{-4} 4^3}{3!} \approx 0.195.$$

Ini berarti terdapat tepat tiga pelanggan akan datang dalam satu jam di stasiun tersebut dengan probabilitas sebesar 0.195.

2.5. Teori Optimisasi

Optimisasi adalah proses menemukan solusi terbaik dari suatu masalah di bawah kondisi tertentu. Dalam *electric bike-sharing*, optimisasi berperan penting dalam menentukan lokasi stasiun pengisian daya yang optimal, alokasi sepeda, dan manajemen sistem secara keseluruhan. Berikut beberapa konsep utama yang relevan dalam penulisan ini.

Definisi 2.5.1 (Nocedal dan Wright (2006)) Diberikan masalah optimisasi

$$\min_{\mathbf{x} \in X} f(\mathbf{x}),$$

dengan $X \subseteq \mathbb{R}^n$ adalah himpunan fisibel dan $f : X \rightarrow \mathbb{R}$ adalah fungsi tujuan. Suatu titik $\mathbf{x}^* \in X$ disebut solusi global jika $f(\mathbf{x}^*) \leq f(\mathbf{x})$ untuk semua $\mathbf{x} \in X$.

Dalam masalah penentuan lokasi stasiun pengisian daya, solusi global direpresentasikan dengan konfigurasi terbaik dari variabel keputusan sebagai contoh, konfigurasi yang dimaksud meliputi penentuan lokasi-lokasi spesifik untuk stasiun pengisian daya guna memaksimalkan keuntungan atau meminimalkan biaya secara keseluruhan. Dalam penyelesaian model yang kompleks seperti ini dapat digunakan algoritma metaheuristik sebagai alat untuk menemukan solusi terbaik dan untuk menemukan solusi terbaik ini terlebih dahulu dicari dalam ruang pencarian.

Definisi 2.5.2 (Glover (1986)) *Ruang pencarian S adalah himpunan semua solusi yang mungkin untuk suatu masalah optimisasi. Setiap elemen $s \in S$ merepresentasikan satu konfigurasi susunan nilai variabel-variabel keputusan atau solusi potensial dari masalah yang sedang diselesaikan.*

Ruang pencarian memberikan gambaran tentang semua kemungkinan solusi yang dapat dieksplorasi oleh algoritma optimisasi. Kemudian, untuk lebih memperjelas struktur ruang pencarian diperlukan fungsi *neighbourhood*.

Definisi 2.5.3 (Glover (1986)) *Fungsi neighborhood $N : S \rightarrow 2^S$ adalah fungsi yang memetakan setiap solusi $s \in S$ ke subset dari S yang disebut neighborhood dari s . Elemen-elemen dalam $N(s)$ biasanya merupakan solusi yang dapat dicapai dari s melalui modifikasi kecil atau operasi tertentu.*

Berikut diberikan contoh dari fungsi *neighbourhood*.

Contoh 2.5.4 Misalkan sebuah sistem bike-sharing memiliki dua stasiun, masing-masing dengan kapasitas maksimal 5 sepeda. Suatu solusi direpresentasikan oleh pasangan bilangan (x, y) , dengan x adalah jumlah sepeda di stasiun 1 dan y adalah jumlah sepeda di stasiun 2.

Fungsi *neighborhood* $N(s)$ untuk solusi $s = (x, y)$ dapat dibentuk sebagai berikut.

$$N(x, y) = \{(x', y') \mid |x' + y' - (x + y)| \leq 1, 0 \leq x' \leq 5, 0 \leq y' \leq 5\}$$

1. Solusi *neighbor* adalah solusi yang dapat dicapai dengan memindahkan satu sepeda antar stasiun atau menambah/mengurangi satu sepeda dari sistem.
2. Batasan $|x' + y' - (x + y)| \leq 1$ memastikan perubahan total sepeda tidak lebih dari 1.
3. Batasan $0 \leq x' \leq 5$ dan $0 \leq y' \leq 5$ memastikan kapasitas stasiun tidak dilampaui.

Diambil 2 pasangan kapasitas sepeda di stasiun 1 dan 2 sebagai berikut.

1. Jika $s = (3, 2)$, maka $N(s) = \{(2, 2), (3, 1), (3, 3), (4, 2), (2, 3), (4, 1)\}$,
2. Jika $s = (5, 0)$, maka $N(s) = \{(4, 0), (4, 1), (5, 1)\}$.

Fungsi *neighbourhood* ini memungkinkan algoritma optimisasi terutama metaheuristik untuk menjelajahi ruang pencarian secara efisien.

Definisi 2.5.5 (Glover (1986)) *Metaheuristik adalah prosedur pencarian yang menggunakan strategi untuk mengeksplorasi ruang pencarian. Didefinisikan tupel (S, f, N, M) dengan:*

1. S adalah ruang pencarian,
2. $f : S \rightarrow \mathbb{R}$ adalah fungsi tujuan yang akan dioptimalkan,
3. $N : S \rightarrow 2^S$ adalah fungsi neighborhood, dan
4. M adalah memori yang menyimpan informasi pencarian.

Prosedur metaheuristik secara umum mencari nilai $s^ \in S$ sedemikian sehingga $f(s^*)$ optimal.*

Algoritma *Fire Hawk Optimizer* yang digunakan dalam penelitian ini termasuk dalam kategori metaheuristik.

Contoh 2.5.6 Misalkan dibentuk tupel (S, f, N, M) sebagai berikut

1. Himpunan S adalah ruang pencarian yang terdiri dari semua konfigurasi yang mungkin untuk penempatan stasiun dan alokasi sepeda. Misalnya, $\mathbf{s} \in S$ mungkin direpresentasikan sebagai vektor $\mathbf{s} = (x_1, \dots, x_n, y_1, \dots, y_n)$, dengan x_i adalah lokasi stasiun ke- i dan y_i adalah jumlah sepeda di stasiun tersebut.
2. Fungsi $f : S \rightarrow \mathbb{R}$, merepresentasikan total keuntungan yang dinyatakan dalam

$$f(\mathbf{s}) = \sum_{i=1}^n (p_i y_i - c_i x_i) - \sum_{u \in U} \alpha_u d_u(\mathbf{s})$$

dengan p_i adalah profit per sepeda di stasiun i , c_i adalah biaya pembangunan stasiun i , α_u adalah probabilitas skenario permintaan u , dan $d_u(\mathbf{s})$ adalah fungsi yang mengukur ketidakpuasan permintaan untuk konfigurasi $\mathbf{s}$ dalam skenario u .

3. Fungsi $N : S \rightarrow 2^S$ adalah fungsi neighborhood yang mendefinisikan solusi-solusi yang dapat dicapai dari suatu solusi dengan modifikasi. Misalnya,

$$N(\mathbf{s}) = \{\mathbf{s}' \in S \mid \mathbf{s}' \text{ didapat dengan memilih stasiun} \\ \text{atau mengubah alokasi sepeda}\}$$

4. Notasi M merupakan struktur memori yang menyimpan informasi pencarian, seperti solusi terbaik yang ditemukan sejauh ini, frekuensi pencarian ke area tertentu dalam ruang pencarian, atau tren perubahan fungsi tujuan.

Tujuan dari metaheuristik ini adalah menemukan nilai $\mathbf{s}^* \in S$ yang memaksimalkan $f(\mathbf{s})$. Seperti halnya algoritma *fire hawk optimizer* terdiri dari struktur ini untuk mengeksplorasi ruang pencarian S , mengevaluasi solusi f , menggunakan N untuk bergerak antar solusi, dan memanfaatkan M untuk menyimpan dan menggunakan informasi selama proses pencarian.

BAB III

Optimisasi Stokastik

3.1. Program Stokastik

Dalam konteks optimisasi, terdapat dua jenis model utama, yaitu model deterministik dan model stokastik. Perbedaan mendasar antara keduanya terletak pada cara keduanya menangani ketidakpastian dalam parameter. Model deterministik mengasumsikan bahwa semua parameter diketahui secara pasti, meskipun dalam kenyataannya, nilai-nilai tersebut sering kali hanya berupa estimasi. Model ini memperlakukan parameter sebagai nilai tetap dan pasti, yang memudahkan proses optimisasi namun mungkin tidak sepenuhnya merefleksikan kondisi dunia nyata, di mana ketidakpastian selalu ada. Sebaliknya, model stokastik dirancang untuk mengakomodasi ketidakpastian dengan memodelkan setidaknya satu parameter sebagai variabel acak yang mengikuti distribusi probabilitas tertentu. Pendekatan stokastik memungkinkan analisis yang lebih realistis terhadap kondisi dunia nyata, terutama ketika menghadapi variabel yang tidak sepenuhnya dapat diprediksi. (Birge dan Louveaux (2011)).

Definisi 3.1.1 (Birge dan Louveaux (2011)) *Program stokastik adalah model optimisasi di mana setidaknya satu dari parameter merupakan variabel acak.*

Dengan mempertimbangkan ketidakpastian dalam model, program stokastik bertujuan untuk menemukan solusi yang optimal tidak hanya untuk satu skenario tunggal, tetapi juga untuk berbagai skenario yang mungkin terjadi. Pendekatan ini sangat relevan dalam konteks sistem *electric bike-sharing*, di mana faktor-faktor seperti permintaan pengguna, ketersediaan sepeda, dan pasokan daya dapat berfluktuasi secara acak. Dengan mengakomodasi variasi ini, program stokastik dapat menghasilkan keputusan yang lebih adaptif dan robust, sehingga sistem dapat beroperasi secara optimal meskipun terdapat perubahan atau

ketidakpastian di lapangan.

Selanjutnya, akan diberikan contoh permasalahan sederhana pemodelan program stokastik sebagai berikut.

Contoh 3.1.2 Diberikan masalah optimisasi stokastik sederhana dengan variabel keputusan $\mathbf{x}$ dan variabel acak $\mathbf{c}$ yang memiliki fungsi objektif:

$$\max E[\mathbf{c} \cdot \mathbf{x}]$$

dengan kendala

$$\mathbf{c} \in \mathbb{R}, \quad 0 \leq \mathbf{x} \leq 3, \quad \mathbf{x} \in \mathbb{Z}^n, \quad n \in \mathbb{N}.$$

Notasi $E[\cdot]$ merepresentasikan nilai harapan, dengan fungsi objektif menggunakan ekspektasi $E[\mathbf{c} \cdot \mathbf{x}]$ karena $\mathbf{c}$ merupakan variabel acak yang mengikuti suatu distribusi probabilitas. Penggunaan ekspektasi ini memungkinkan perhitungan nilai rata-rata dari semua kemungkinan nilai $\mathbf{c}$, sehingga akan menghasilkan solusi yang optimal secara rata-rata.

Misalkan $\mathbf{c}$ memiliki tiga kemungkinan nilai dengan distribusi diskrit,

1. $P(c = 2) = 0.2$,
2. $P(c = 3) = 0.5$, dan
3. $P(c = 4) = 0.3$.

Maka nilai ekspektasi dari $\mathbf{c}$ dapat dihitung sebagai

$$E[\mathbf{c}] = 2 \cdot 0.2 + 3 \cdot 0.5 + 4 \cdot 0.3 = 3.1,$$

sehingga fungsi objektif dapat ditulis menjadi

$$E[\mathbf{c} \cdot \mathbf{x}] = 3.1x.$$

Dengan demikian, meskipun nilai $\mathbf{c}$ bersifat tidak pasti, masalah optimisasi dapat diselesaikan dengan menggunakan nilai ekspektasinya untuk mendapatkan solusi yang optimal secara rata-rata.

3.1.1. Program Bilangan Bulat Stokastik

Program bilangan bulat stokastik mengembangkan konsep program stokastik dengan membatasi beberapa atau semua variabel keputusan menjadi bilangan bulat. Pendekatan ini sangat umum dalam aplikasi praktis, di mana keputusan harus dibuat dalam unit diskrit yang tidak dapat dibagi, seperti jumlah sepeda listrik yang harus disediakan dalam sistem *electric bike-sharing*.

Definisi 3.1.3 (*Birge dan Louveaux (2011)*) Program bilangan bulat stokastik adalah jenis program stokastik di mana solusi untuk salah satu atau lebih variabel keputusan harus berupa bilangan bulat.

Program bilangan bulat stokastik sangat relevan dengan *electric bike-sharing* karena banyak keputusan yang harus diambil melibatkan variabel diskrit, seperti jumlah sepeda atau stasiun pengisian daya. Selain itu, ketidakpastian dalam permintaan dan biaya operasional menambah kompleksitas pada masalah optimisasi. Berikut diilustrasikan contoh penerapan program bilangan bulat stokastik.

Contoh 3.1.4 Dimisalkan perusahaan *electric bike-sharing* yang harus memutuskan jumlah sepeda listrik untuk dipesan menghadapi permintaan yang tidak pasti. Misalkan x adalah jumlah sepeda yang dipesan, d adalah permintaan (variabel acak), dan p adalah keuntungan per sepeda yang disewakan. Oleh karena itu, masalah dapat diformulasikan sebagai

$$\max E[\min(x, d) \cdot p] - cx$$

dengan kendala

$$x \geq 0, \quad x \in \mathbb{Z}^n, \quad n \in \mathbb{N}.$$

Konstanta c menyatakan biaya tetap pemesanan satu unit sepeda listrik. Model ini mengoptimalkan keuntungan yang diharapkan dengan mempertimbangkan ketidakpastian dalam permintaan dan membuat keputusan tentang jumlah pesanan yang optimal sebagai variabel keputusan bilangan bulat.

Model semacam ini sangat relevan dengan *electric bike-sharing* di mana perusahaan harus memutuskan berapa banyak sepeda listrik yang harus dibeli di hadapan ketidakpastian mengenai permintaan pengguna dan biaya operasional yang bervariasi.

3.1.2. Program Stokastik Dua Tahap

Program stokastik dua tahap adalah salah satu jenis program stokastik yang memodelkan keputusan yang harus dibuat dalam dua tahap. Pada tahap pertama, keputusan diambil sebelum ketidakpastian (variabel acak) terungkap. Pada tahap kedua, setelah ketidakpastian diketahui, variabel keputusan tambahan dibuat untuk menyesuaikan atau merespons terhadap realisasi ketidakpastian tersebut (Birge dan Louveaux (2011)).

Definisi 3.1.5 (Birge dan Louveaux (2011)) *Program stokastik dua tahap atau two stage stochastic programming adalah model optimisasi dengan keputusan dapat dibagi dalam 2 fase atau tahap. Tahap pertama mempertimbangkan keputusan yang dibuat sebelum nilai dari variabel acak diketahui, dan tahap kedua memberikan respon setelah nilai parameter stokastik diketahui.*

Untuk memperjelas konsep program stokastik dua tahap diilustrasikan contoh bagaimana keputusan dibagi menjadi dua tahap dan bagaimana ketidakpastian mempengaruhi proses pengambilan keputusan.

Contoh 3.1.6 Misalkan sebuah operator *electric bike-sharing* di Yogyakarta memiliki 2 jenis sepeda listrik, reguler dan premium. Setiap malam, operator harus memutuskan berapa jam operasional dari masing-masing jenis sepeda yang harus disiapkan di stasiun utama untuk keesokan harinya. Satu jam tambahan juga akan disediakan untuk fleksibilitas. Jika keesokan hari cuaca cerah (kemungkinan 70%), akan ada permintaan untuk 3 jam sepeda reguler dan 1 jam sepeda premium. Jika cuaca hujan (kemungkinan 30%), permintaan akan menjadi 1 jam sepeda reguler dan 3 jam sepeda premium. Operator ingin memaksimalkan ekspektasi total jam sepeda yang digunakan.

Permasalahan ini adalah contoh program stokastik dua tahap. Pada tahap pertama, operator memutuskan alokasi awal jam operasional sepeda sebelum mengetahui kondisi cuaca. Tahap kedua terjadi keesokan hari ketika cuaca diketahui, dan operator dapat menyesuaikan alokasi jam operasional yang tersisa.

Variabel keputusan tahap pertama adalah jumlah jam operasional sepeda reguler (x_1) dan premium (x_2) yang disiapkan. Untuk tahap kedua, w_1^s dan w_2^s adalah jumlah jam operasional sepeda reguler dan premium yang ditambahkan setelah cuaca diketahui, dengan $s = 1$ untuk cerah dan $s = 2$ untuk hujan. Variabel z_1^s dan z_2^s adalah jumlah jam operasional sepeda reguler dan premium yang digunakan pada hari dengan cuaca tipe s .

Model program linear stokastik dua tahap dapat dinyatakan dalam bentuk

$$\max 0,70(z_1^1 + z_2^1) + 0,30(z_1^2 + z_2^2)$$

dengan kendala

$$x_1 + x_2 = 3,$$

$$x_1 + w_1^1 \geq z_1^1,$$

$$x_2 + w_2^1 \geq z_2^1,$$

$$w_1^1 + w_2^1 = 1,$$

$$z_1^1 \leq 3,$$

$$z_2^1 \leq 1,$$

$$x_1 + w_1^2 \geq z_1^2,$$

$$x_2 + w_2^2 \geq z_2^2,$$

$$w_1^2 + w_2^2 = 1,$$

$$z_1^2 \leq 1,$$

$$z_2^2 \leq 3,$$

$$\text{semua variabel} \geq 0.$$

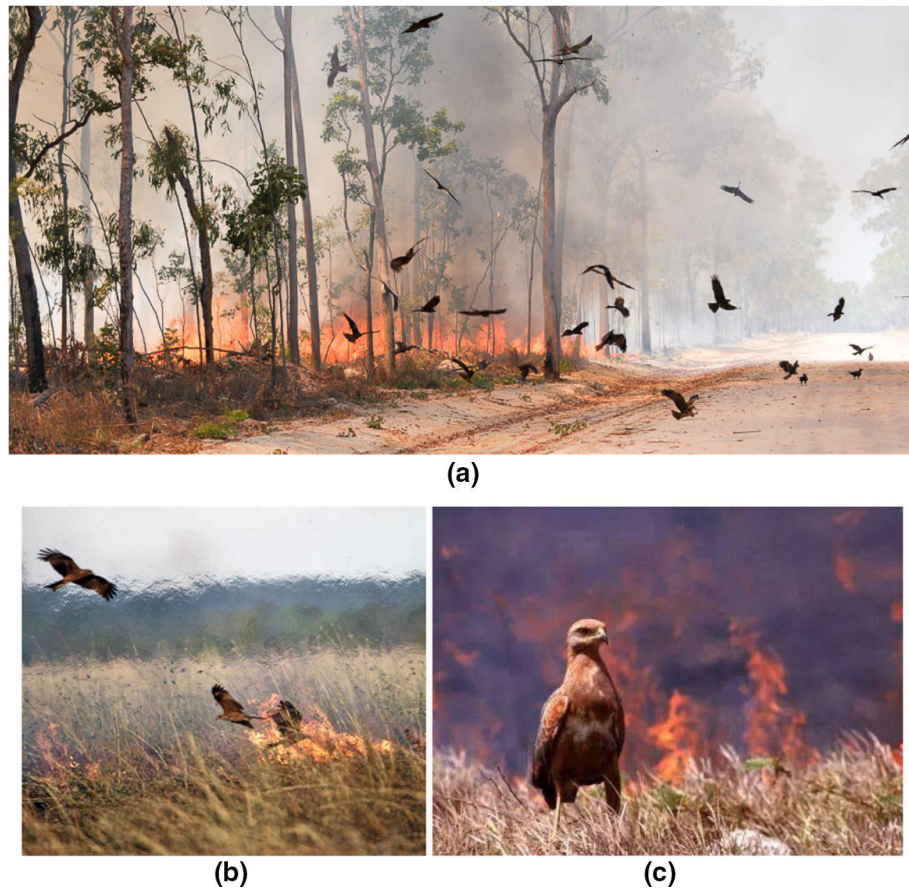
Dengan menggunakan *software* optimisasi Lingo 19.0 diperoleh solusi 2

jam operasional untuk sepeda reguler dan 1 jam untuk premium di malam hari. Jika cuaca cerah, operator menambahkan 1 jam operasional untuk sepeda reguler, sehingga jam operasional yang digunakan adalah 3 jam reguler dan 1 jam premium. Jika cuaca hujan, operator menambahkan 1 jam operasional untuk sepeda premium, sehingga jam operasional yang digunakan adalah 2 jam reguler dan 2 jam premium. Strategi ini menghasilkan ekspektasi penggunaan sebesar 3,7 jam per hari.

Contoh ini mengilustrasikan bagaimana operator *electric bike-sharing* dapat mengoptimalkan alokasi jam operasional sepeda dengan mempertimbangkan ketidakpastian cuaca dan pola permintaan untuk efisiensi operasional dan kepuasan pengguna.

3.2. *Fire Hawk Optimizer*

Suku asli Australia memanfaatkan api sebagai alat untuk mengendalikan dan menjaga keseimbangan ekosistem lokal, yang telah menjadi bagian dari budaya dan tradisi mereka selama bertahun-tahun. Sering kali, kebakaran yang dimulai dengan sengaja atau secara alami terjadi, seperti karena petir, menyebar karena faktor manusia, dan faktor-faktor lain yang meningkatkan kerentanan lanskap alami dan satwa liar. Selain itu, elang siul, elang hitam, dan elang coklat juga menjadi salah satu penyebab utama penyebaran api di alam liar. Burung-burung ini dikenal sebagai elang api atau *fire hawk* yang selalu mencoba untuk menyebarkan api dengan cara membawa kayu yang terbakar dengan menggunakan paruh ataupun cakar mereka. Aktivitas elang ini dicatat sebagai fenomena yang merusak alam (Azizi dkk. (2023)).



Gambar 3.1 Foto Perilaku Elang Api Terhadap Api (Azizi dkk. (2023))

Aktivitas elang ini diilustrasikan pada Gambar 3.1. Sebagai mekanisme untuk mengendalikan dan menangkap mangsa mereka, burung-burung ini mengambil kayu yang terbakar dan menjatuhkannya di tempat yang tidak terbakar untuk memantik api (a). Api yang terpantik ini akan menakuti mangsa seperti tikus, ular, ataupun hewan lain, dan memaksa mereka mundur (b) ke tempat di mana mereka dapat dimangsa dengan mudah oleh burung-burung elang api ini (c).

Algoritma *Fire-Hawk Optimizer* meniru perilaku dari elang api yaitu perilaku mereka mengatur dan menyebarkan api untuk menangkap mangsanya. Lebih rincinya cara kerja algoritma tersebut sebagai berikut. Pertama, ditentukan jumlah kandidat solusi (X) sebagai vektor posisi dari *fire hawk* dan mangsa. Lalu, ditentukan juga inisialisasi secara acak nilai awal vektor posisi di ruang pencarian, yaitu didefinisikan

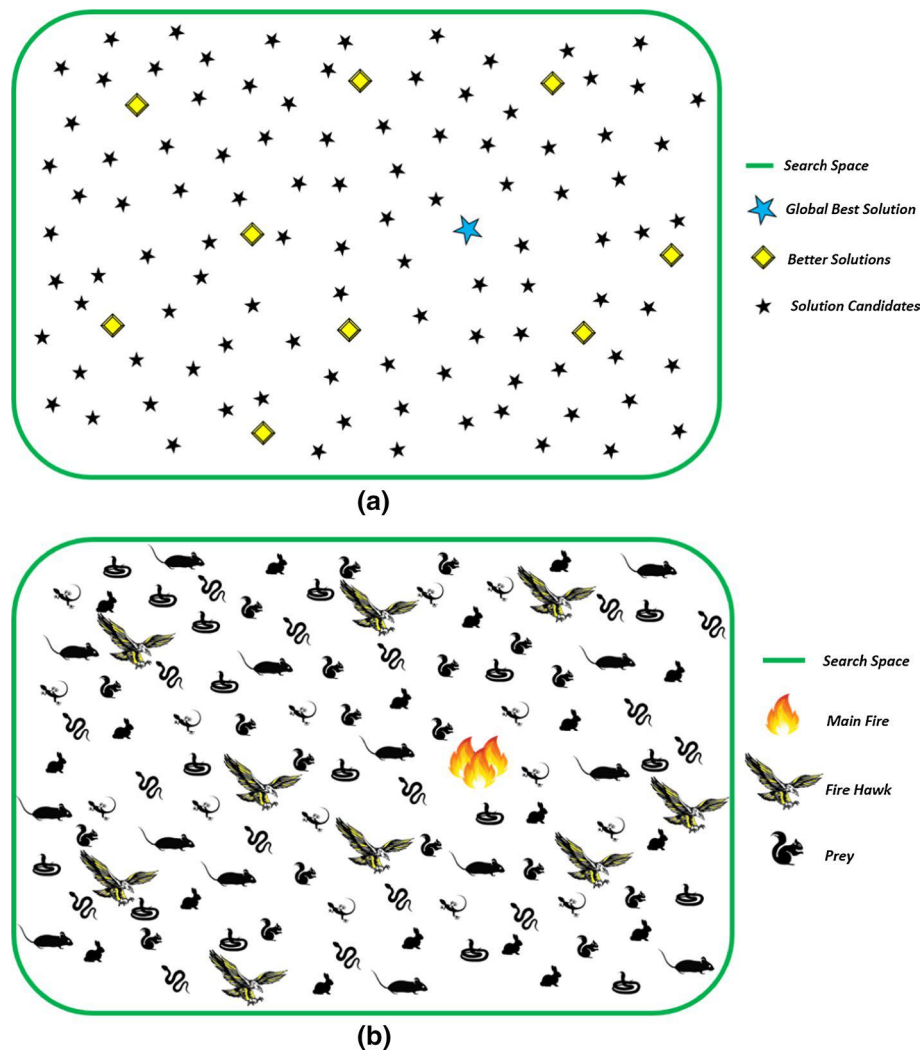
$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_i \\ \vdots \\ X_N \end{bmatrix} = \begin{bmatrix} x_1^1 & x_1^2 & \cdots & x_1^a & \cdots & x_1^d \\ x_2^1 & x_2^2 & \cdots & x_2^a & \cdots & x_2^d \\ \vdots & \vdots & & \vdots & & \vdots \\ x_b^1 & x_b^2 & \cdots & x_b^a & \cdots & x_b^d \\ \vdots & \vdots & & \vdots & & \vdots \\ x_N^1 & x_N^2 & \cdots & x_N^a & \cdots & x_N^d \end{bmatrix}, \quad (3.1)$$

dan

$$x_i^j(0) = x_{i,\min}^j + \text{rand}(x_{i,\max}^j - x_{i,\min}^j), \quad i = 1, 2, \dots, N, \text{ dan } j = 1, 2, \dots, d. \quad (3.2)$$

Notasi X_i merepresentasikan kandidat solusi ke i di ruang pencarian, d merepresentasikan dimensi dari permasalahan, N adalah jumlah total kandidat solusi di ruang pencarian, x_i^j adalah variabel keputusan ke j dari kandidat solusi ke i , $x_i^j(0)$ merepresentasikan posisi awal dari kandidat solusi, $x_{i,\min}^j$ dan $x_{i,\max}^j$ adalah batas bawah dan batas atas dari decision variable ke j untuk kandidat solusi ke i , dan rand adalah fungsi untuk memilih bilangan acak yang terdistribusi seragam dalam rentang $[0,1]$.

Untuk menentukan lokasi *fire hawk* di ruang pencarian, fungsi objektif dievaluasi untuk memilih kandidat solusi dari masalah optimisasi yang dikerjakan. Beberapa kandidat solusi dengan nilai fungsi objektif terbaik direpresentasikan sebagai *fire hawk*, sedangkan kandidat solusi lain dianggap sebagai mangsa. *Fire hawk* yang terpilih dimanfaatkan untuk menyebarkan api di sekitar mangsa di ruang pencarian untuk memudahkan perburuan. Selain itu, solusi global terbaik diasumsikan sebagai api utama yang pertama kali dimanfaatkan oleh *fire hawk* untuk menyebarkan api di ruang pencarian. Skema di atas diilustrasikan pada Gambar 3.2.



Gambar 3.2 Skema Penentuan Elang Api dan Mangsa di Ruang Pencarian (Azizi dkk. (2023))

Secara matematis skema tersebut dapat ditulis sebagai

$$PR = \begin{bmatrix} PR_1 \\ PR_2 \\ \vdots \\ PR_k \\ \vdots \\ PR_m \end{bmatrix}, \quad (3.3)$$

dan

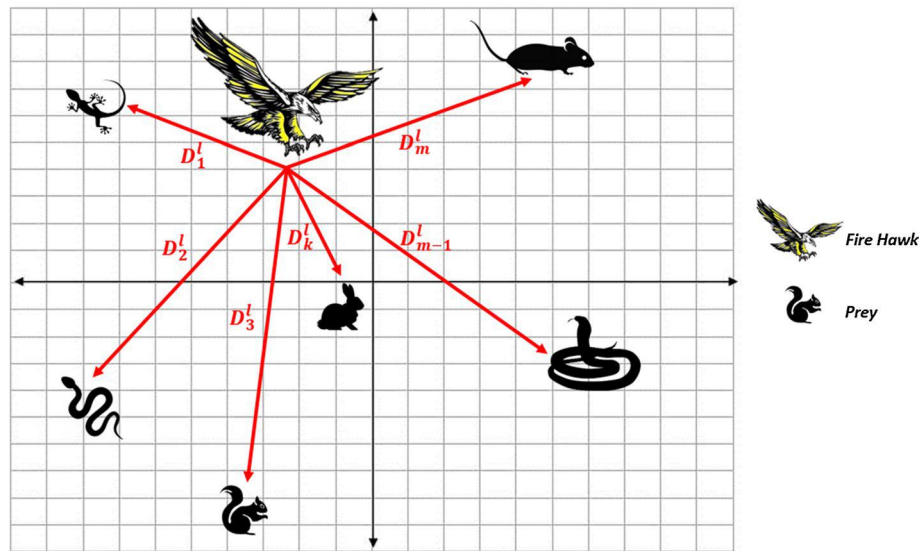
$$FH = \begin{bmatrix} FH_1 \\ FH_2 \\ \vdots \\ FH_l \\ \vdots \\ FH_n \end{bmatrix}. \quad (3.4)$$

Notasi PR_k adalah mangsa ke- k di ruang pencarian dengan m jumlah keseluruhan mangsa, dan FH_l adalah *fire hawk* ke l dengan n jumlah keseluruhan *fire hawk* di ruang pencarian.

Fase berikutnya dari algoritma ini adalah menghitung jumlah jarak antara *fire hawk* dan mangsa, sehingga jarak mangsa terpendek terhadap tiap *fire hawk* dapat ditentukan. Jadi, teritori yang efektif untuk burung-burung ini dapat ditentukan. Dengan kata lain teritori *fire hawk* dengan fungsi objektif terbaik ditentukan dari mangsa-mangsa terdekatnya, teritori *fire hawk* dengan fungsi objektif terbaik kedua ditentukan berdasar mangsa didekatnya yang tidak termasuk dalam teritori *fire hawk* sebelumnya, dan seterusnya. Gambar 3.3 menunjukkan ilustrasi dari perspektif ini, dengan D_k^l didefinisikan sebagai

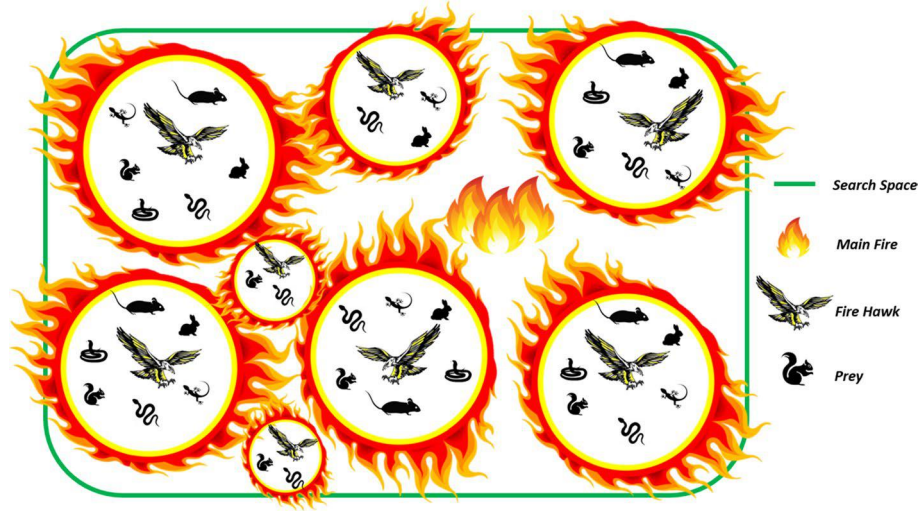
$$D_k^l = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}, \quad l = 1, 2, \dots, n, \text{ dan } k = 1, 2, \dots, m. \quad (3.5)$$

Notasi D_k^l menyatakan jumlah jarak antara *fire hawk* ke- l dan mangsa ke- k , m adalah jumlah mangsa di ruang pencarian, n adalah jumlah *fire hawk* di ruang pencarian, dan (x_1, y_1) dan (x_2, y_2) merepresentasikan koordinat dari *fire hawk* di ruang pencarian.



Gambar 3.3 Skema Perhitungan Jarak Antara Elang Api dan Mangsa (Azizi dkk. (2023))

Setelah melakukan prosedur yang telah dijelaskan di atas untuk mengukur jarak total antara *fire hawk* dan mangsa, teritori burung-burung ini dapat ditentukan berdasarkan mangsa terdekat di sekitarnya. Dengan mengklasifikasikan *fire hawk* dan mangsa, proses pencarian dari algoritma dapat diatur. Perlu diperhatikan bahwa *fire hawk* dengan nilai fungsi objektif yang lebih baik akan memilih mangsa terdekat terbaik dalam ruang pencarian untuk teritori tertentu. Kemudian, *fire hawk* lain mencari mangsa terdekat lainnya di ruang pencarian, yang menunjukkan bahwa *fire hawk* terkuat melakukan perburuan yang lebih baik daripada *fire hawk* lain. Gambar 3.4 menunjukkan skema penentuan teritori *fire hawk* di ruang pencarian.

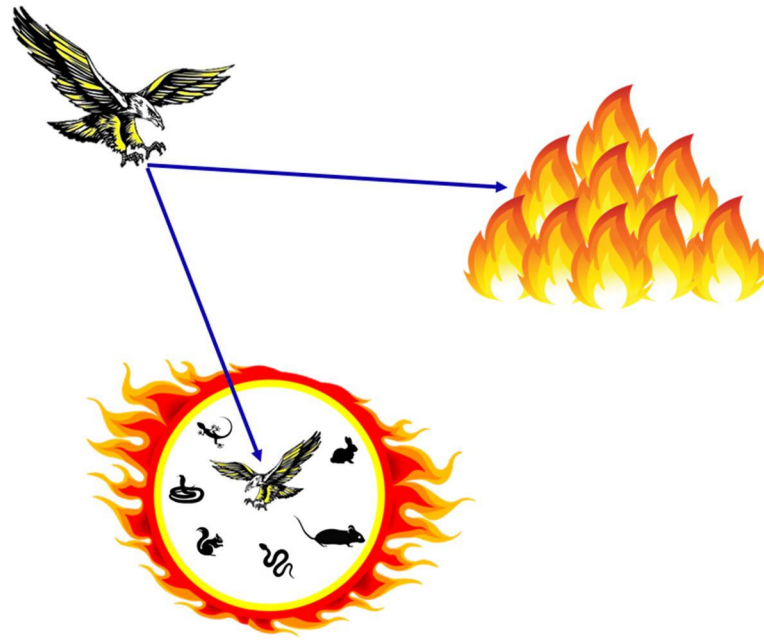


Gambar 3.4 Skema Penentuan Teritori Elang Api di Ruang Pencarian (Azizi dkk. (2023))

Pada fase berikutnya dari algoritma, *fire hawk* mengumpulkan kayu yang terbakar dari api utama untuk memantik api di area yang dipilih. Pada fase ini, tiap burung membawa kayu yang terbakar lalu menjatuhkannya di teritori tertentu untuk memaksa mangsanya buru-buru melarikan diri. Sedangkan beberapa burung akan menggunakan kayu yang terbakar dari teritori *fire hawk* yang lain sehingga kedua perilaku ini dapat dimanfaatkan untuk memperbarui prosedur dari *loop* pencarian utama FHO, yang ditunjukkan pada persamaan berikut:

$$FH_l^{new} = FH_l + (r_1 \times GB - r_2 \times FH_{near}), l = 1, 2, \dots, n. \quad (3.6)$$

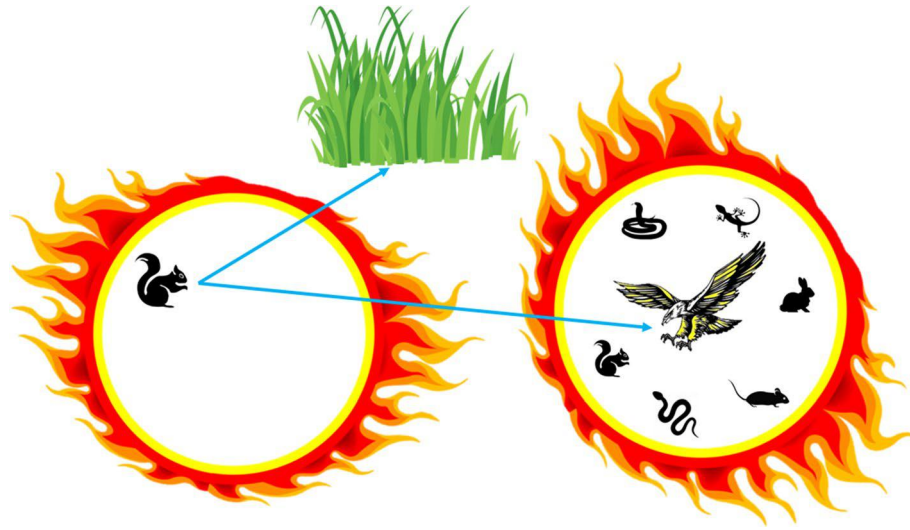
Notasi FH_l^{new} menyatakan vektor posisi baru dari *fire hawk* (FH_l) ke- l , GB adalah solusi global terbaik di ruang pencarian yang dianggap sebagai api utama, FH_{near} adalah salah satu *fire hawk* lain di ruang pencarian, dan r_1 dan r_2 adalah angka acak yang terdistribusi seragam dalam rentang $(0, 1)$ untuk menentukan pergerakan dari *fire hawk* terhadap api utama dan teritori *fire hawk* lain. Gambar 3.5 menunjukkan skema pembaruan posisi *fire hawk*.



Gambar 3.5 Skema Proses Pembaruan Posisi Elang Api di Ruang Pencarian (Azizi dkk. (2023))

Fase algoritma selanjutnya, pergerakan dari mangsa di dalam teritori dari tiap *fire hawk* dianggap sebagai kunci dari perilaku hewan untuk proses pembaruan posisi. Ketika kayu yang terbakar dijatuhkan oleh seekor *fire hawk*, mangsa memutuskan untuk bersembunyi, melarikan diri, ataupun berlari ke arah *fire hawk* secara tidak sengaja. Perilaku ini dapat dianggap sebagai proses pembaruan posisi yang dituliskan dengan persamaan berikut:

$$PR_q^{new} = PR_q + (r_3 \times FH_l - r_4 \times SP_l), \quad l = 1, 2, \dots, n, \text{ dan } q = 1, 2, \dots, r. \quad (3.7)$$



Gambar 3.6 Skema Proses Pembaruan Posisi Mangsa di Luar Teritori Elang Api (Azizi dkk. (2023))

Notasi PR_q^{new} menyatakan vektor posisi baru dari mangsa ke- q (PR_q) dikelilingi oleh *fire hawk* ke- l (FH_l), GB adalah solusi global terbaik di ruang pencarian dianggap sebagai api utama, SP_l adalah tempat aman di dalam teritori *fire hawk* ke- l , dan r_3 dan r_4 adalah bilangan acak yang terdistribusi seragam dalam rentang $(0, 1)$ untuk menentukan pergerakan dari mangsa menuju ke *fire hawk* dan tempat aman. Gambar 3.6 menunjukkan skema pembaruan posisi mangsa.

Selain itu, mangsa melakukan pergerakan menuju teritori *fire hawk* lain ketika ada kemungkinan mangsa akan mendekat ke *fire hawk* di dekat semak-semak atau bahkan bersembunyi di tempat yang lebih aman di luar teritori *fire hawk* yang membuat mereka terjebak. Perilaku ini dapat dianggap sebagai proses pembaruan posisi yang dapat dituliskan dalam persamaan berikut.

$$PR_q^{new} = PR_q + (r_5 \times FH_{Alter} - r_6 \times SP), \quad l = 1, 2, \dots, n \text{ dan } q = 1, 2, \dots, r. \quad (3.8)$$

Notasi PR_q^{new} menyatakan vektor posisi baru dari mangsa ke- q (PR_q) dikelilingi oleh *fire hawk* ke- l (FH_l), FH_{Alter} adalah salah satu *fire hawk* lain di

ruang pencarian, SP adalah tempat aman di luar teritori *fire hawk* ke- l , r_5 dan r_6 adalah bilangan acak yang terdistribusi seragam dalam rentang $(0, 1)$ untuk menentukan pergerakan dari mangsa menuju *fire hawk* lain dan tempat aman di luar teritori.

Berdasarkan fakta bahwa tempat aman di alam biasanya berupa lokasi di mana sebagian besar hewan berkumpul agar tetap aman dan nyaman selama terdapat ancaman, SP_l dan SP dapat diformulasikan sebagai berikut:

$$SP_1 = \frac{\sum_{q=1}^r PR_q}{r}, \quad q = 1, 2, \dots, r \text{ dan } l = 1, 2, \dots, n. \quad (3.9)$$

$$SP = \frac{\sum_{k=1}^m PR_k}{m}, \quad k = 1, 2, \dots, m. \quad (3.10)$$

Notasi PR_q menyatakan mangsa ke- q yang dikelilingi oleh *fire hawk* ke- l (FH_l) dan PR_k adalah mangsa ke- k di ruang pencarian.

Perlu diperhatikan bahwa teritori dari tiap *fire hawk* diasumsikan sebagai daerah melingkar untuk tujuan visualisasi skema, sehingga definisi dari teritori tergantung jarak secara keseluruhan dari mangsa dan *fire hawk*. Dengan kata lain ketika mangsa diposisikan di teritori *fire hawk* tertentu, diasumsikan mangsa dipengaruhi oleh *fire hawk* tersebut dan bukan yang lain, sehingga jumlah dari mangsa dan jarak mereka terhadap *fire hawk* ditentukan oleh batas dari teritori *fire hawk*. Sedangkan, kemungkinan mangsa berada di luar teritori mereka juga dianggap proses pembaruan posisi terkait fakta bahwa mangsa harus dipengaruhi oleh *fire hawk* dari teritori *fire hawk* lain. Jumlah mangsa tiap *loop* adalah total jumlah kandidat solusi dikurangi dengan jumlah *fire hawk* yang ditentukan secara acak menggunakan *brownian motion* dengan distribusi normal sebagai salah satu distribusi terkenal yang digunakan dalam prosedur pengacakan.

Kriteria pemberhentian pada algoritma FHO ini menggunakan kekonvergenan iterasi (Talbi (2009)). Konvergensi terjadi ketika perbaikan solusi

antara iterasi berturut-turut menjadi sangat kecil atau tidak signifikan. Lebih rincinya algoritma dihentikan ketika pertidaksamaan berikut dipenuhi, yaitu

$$\frac{|f(GB_{t+1}) - f(GB_t)|}{|f(GB_t)|} < \epsilon.$$

Notasi $f(GB_t)$ menyatakan nilai fungsi objektif dari solusi global terbaik (*Global Best*) pada iterasi ke- t , $f(GB_{t+1})$ adalah nilai fungsi objektif dari GB pada iterasi berikutnya, dan ϵ adalah ambang batas toleransi yang sangat kecil (misalnya, 10^{-6}). Jika kondisi ini terpenuhi untuk sejumlah iterasi berturut-turut (misalnya, 5 atau 10 iterasi), algoritma dianggap telah konvergen dan dapat dihentikan. Kriteria ini memastikan bahwa algoritma berhenti ketika peningkatan kualitas solusi menjadi minimal yang mengindikasikan bahwa solusi optimal atau mendekati optimal telah ditemukan.

Aspek umum dari FHO termasuk pelanggaran batas dari solusi kandidat di samping kriteria penghentian juga dipertimbangkan dalam model matematika dari algoritma ini. Dalam hal ini, ketentuan matematika juga diimplementasikan dalam algoritma FHO yang mana kontrol batas untuk variabel keputusan yang dilanggar ditentukan, sementara sejumlah evaluasi fungsi objektif atau iterasi yang telah ditentukan sebelumnya atau bahkan kriteria lain seperti kekonvergenan dapat digunakan sebagai kriteria penghentian. Disajikan *pseudo-code* dan *flowchart* dari algoritma FHO di bawah ini.

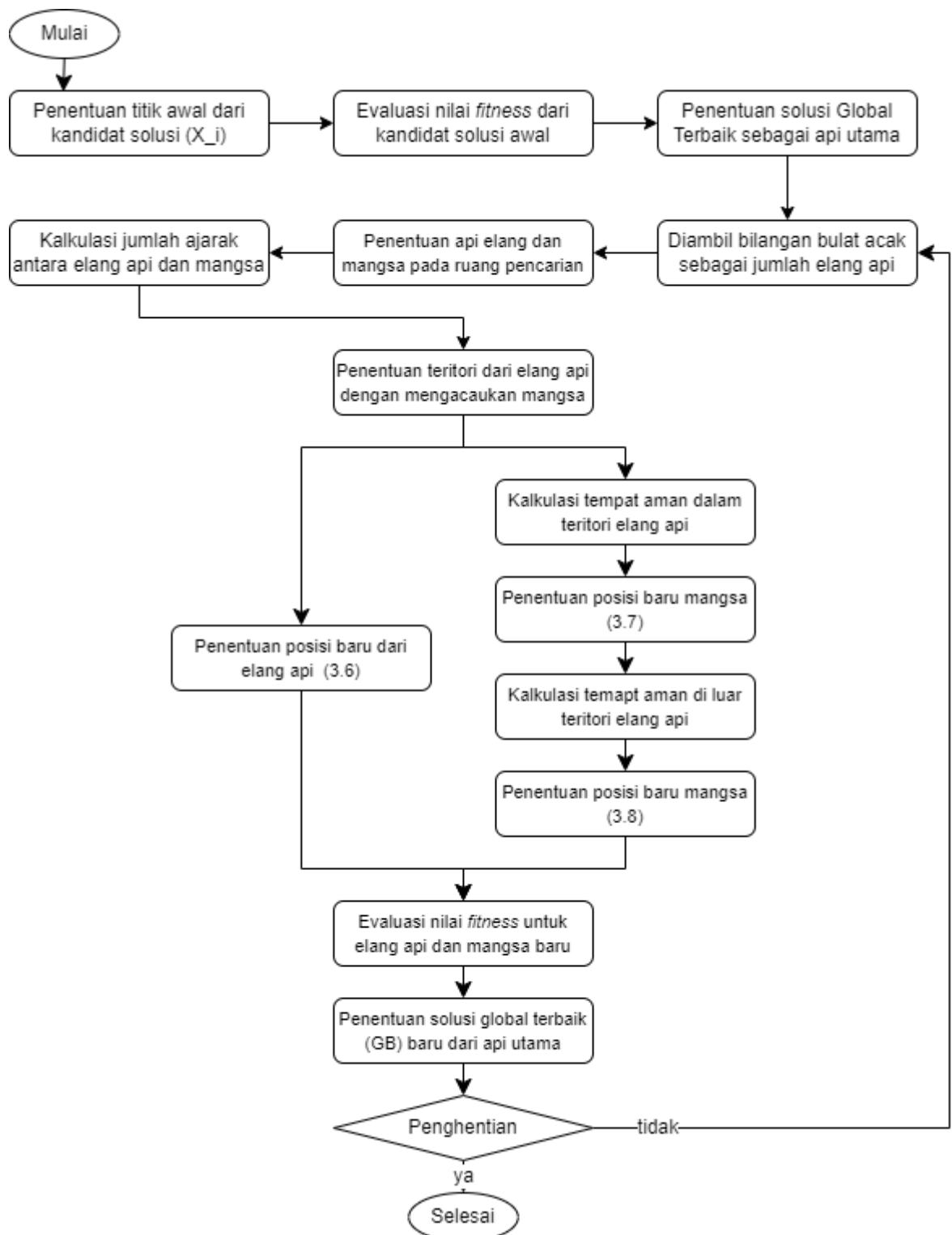
Untuk lebih memudahkan penjelasan dari algoritma *Fire Hawk Optimizer* diilustrasikan flowchart algoritma *Fire Hawk Optimizer* pada Gambar 3.7 dan Contoh 3.2.1

Algorithm 1 Fire Hawk Optimizer (FHO)

```

1: procedure FIRE HAWK OPTIMIZER
2:   Ditentukan posisi awal dari kandidat solusi ( $X_i$ ) di ruang pencarian dengan
    $N$  kandidat
3:   Evaluasi nilai fitness untuk kandidat solusi awal
4:   Ditentukan solusi global terbaik (GB) sebagai ai utama
5:   while Iterasi  $\leq$  iterasi maksimum  $\wedge$  konvergensi counter  $\leq$  iterasi
   konvergensi maksimum do
6:     Dibentuk  $n$  bilangan bulat untuk menentukan jumlah elang api
7:     Ditentukan elang api (FH) dan mangsa-mangsa (PR) di ruang pencarian
8:     Dikalkulasikan total jarak antara elang api dan mangsa-mangsa
9:     Ditentukan teritori elang api dengan menyebarkan mangsa-mangsa
10:    for  $l = 1 : n$  do
11:      Ditentukan posisi baru dari elang api berdasarkan persamaan (3.6)
12:      for  $q = 1 : r$  do
13:        Dikalkulasikan tempat aman di dalam teritori elang api pada
        persamaan (3.9)
14:        Ditentukan posisi baru dari mangsa-mangsa pada persamaan
        (3.7)
15:        Dikalkulasikan tempat aman di luar teritori elang api pada
        persamaan (3.10)
16:        Ditentukan posisi baru dari mangsa-mangsa pada persamaan
        (3.8)
17:      end for
18:    end for
19:    Evaluasi nilai fitness untuk elang api dan mangsa-mangsa baru
20:    Ditentukan solusi global terbaik (GB) sebagai api utama
21:  end while
22:  return GB
23: end procedure

```



Gambar 3.7 Flowchart FHO

Contoh 3.2.1 Akan digunakan algoritma FHO untuk mencari nilai (x, y) yang meminimumkan $f(x, y) = x^2 + y^2$, dengan $-5 \leq x \leq 5$ dan $-5 \leq y \leq 5$.

1. Inisialisasi

Misalkan digunakan 5 kandidat solusi di $\mathbb{R}^2$, yaitu

$$X = \begin{bmatrix} -2.1 & 3.4 \\ 1.5 & -0.8 \\ 4.2 & 2.7 \\ -3.6 & -1.9 \\ 0.3 & 4.5 \end{bmatrix}.$$

2. Evaluasi dan Seleksi

Selanjutnya ditentukan nilai fungsi untuk setiap kandidat sebagai berikut

$$f(-2.1, 3.4) = (-2.1)^2 + (3.4)^2 = 4.41 + 11.56 = 15.97,$$

$$f(1.5, -0.8) = (1.5)^2 + (-0.8)^2 = 2.25 + 0.64 = 2.89,$$

$$f(4.2, 2.7) = (4.2)^2 + (2.7)^2 = 17.64 + 7.29 = 24.93,$$

$$f(-3.6, -1.9) = (-3.6)^2 + (-1.9)^2 = 12.96 + 3.61 = 16.57,$$

$$f(0.3, 4.5) = (0.3)^2 + (4.5)^2 = 0.09 + 20.25 = 20.34.$$

Selanjutnya, dipilih 2 *fire hawk* terbaik kandidat solusi dengan nilai fungsi terendah dan 3 mangsa:

$$FH = \begin{bmatrix} 1.5 & -0.8 \\ -2.1 & 3.4 \end{bmatrix},$$

$$PR = \begin{bmatrix} -3.6 & -1.9 \\ 0.3 & 4.5 \\ 4.2 & 2.7 \end{bmatrix}.$$

3. Penentuan teritori

Berikutnya dihitung jarak antara setiap *fire hawk* dan mangsa. Untuk $FH_1 = [1.5, -0.8]$, diperoleh

$$D_1^1 = \sqrt{(1.5 - (-3.6))^2 + (-0.8 - (-1.9))^2} = 5.22,$$

$$D_2^1 = \sqrt{(1.5 - 0.3)^2 + (-0.8 - 4.5)^2} = 5.39,$$

$$D_3^1 = \sqrt{(1.5 - 4.2)^2 + (-0.8 - 2.7)^2} = 4.54.$$

Untuk $FH_2 = [-2.1, 3.4]$, didapat

$$D_1^2 = \sqrt{(-2.1 - (-3.6))^2 + (3.4 - (-1.9))^2} = 5.59,$$

$$D_2^2 = \sqrt{(-2.1 - 0.3)^2 + (3.4 - 4.5)^2} = 2.69,$$

$$D_3^2 = \sqrt{(-2.1 - 4.2)^2 + (3.4 - 2.7)^2} = 6.37.$$

4. Pembaruan Posisi *Fire hawk*

Misalkan $r_1 = 0.3$, $r_2 = 0.5$, dan $GB = [1.5, -0.8]$ (*fire hawk* terbaik).

Untuk FH_1 , didapat

$$\begin{aligned} FH_1^{new} &= [1.5, -0.8] + (0.3 \times [1.5, -0.8] - 0.5 \times [-2.1, 3.4]) \\ &= [1.5, -0.8] + [0.45, -0.24] - [-1.05, 1.7] \\ &= [3.0, -2.74]. \end{aligned}$$

Untuk FH_2 , diperoleh

$$\begin{aligned} FH_2^{new} &= [-2.1, 3.4] + (0.3 \times [1.5, -0.8] - 0.5 \times [1.5, -0.8]) \\ &= [-2.1, 3.4] + [0.45, -0.24] - [0.75, -0.4] \\ &= [-2.4, 3.56]. \end{aligned}$$

5. Pembaruan Posisi Mangsa

Misalkan $r_3 = 0.4$, $r_4 = 0.6$. Ditentukan tempat aman untuk teritori FH_1 (misalkan PR_1 dan PR_3 ada di teritori ini):

$$SP_1 = \frac{[-3.6, -1.9] + [4.2, 2.7]}{2} = [0.3, 0.4]$$

Pembaruan posisi PR_1 :

$$\begin{aligned} PR_1^{new} &= [-3.6, -1.9] + (0.4 \times [1.5, -0.8] - 0.6 \times [0.3, 0.4]) \\ &= [-3.6, -1.9] + [0.6, -0.32] - [0.18, 0.24] \\ &= [-3.18, -2.46] \end{aligned}$$

6. Evaluasi dan Iterasi

Dievaluasi posisi baru dan ulangi proses dari langkah 2 sampai kriteria penghentian terpenuhi (misalnya, jumlah iterasi maksimum atau konvergensi tercapai).

7. Solusi

Setelah beberapa iterasi, algoritma akan konvergen ke titik (0,0), yang merupakan minimum global dari fungsi $f(x, y) = x^2 + y^2$.

Diselesaikan juga permasalahan sederhana sistem *electric bike-sharing* pada Contoh 3.1.6 menggunakan algoritma *Fire Hawk Optimizer*

Contoh 3.2.2 Akan digunakan permasalahan pada Contoh 3.1.6 dengan menggunakan algoritma *Fire Hawk Optimizer*.

1. Inisialisasi

Ditentukan jumlah kandidat solusi (X) sebagai vektor posisi dari *fire hawk* dan mangsa, serta ditentukan inisialisasi secara acak sebagai nilai awal vektor posisi di ruang pencarian sebagai berikut,

$$X = \begin{bmatrix} 2 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 2 \\ 0 & 3 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 2 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 \\ 2 & 1 & 1 & 0 & 0 & 1 & 3 & 0 & 1 & 0 \\ 1 & 2 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 3 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 2 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 2 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 3 & 0 & 1 & 0 & 1 & 0 & 2 & 0 & 0 & 0 \\ 0 & 3 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 2 \end{bmatrix}.$$

2. Evaluasi dan Seleksi

Fungsi objektif dievaluasi untuk memilih kandidat solusi dari permasalahan. Beberapa kandidat solusi dengan nilai fungsi objektif terbaik direpresentasikan sebagai *fire hawk*, sedangkan kandidat solusi lain dianggap sebagai mangsa. Sehingga didapat *fire hawk* dan mangsa sebagai berikut

$$FH = \begin{bmatrix} 3 & 0 & 1 & 0 & 1 & 0 & 2 & 0 & 0 & 0 \\ 2 & 1 & 1 & 0 & 0 & 1 & 3 & 0 & 1 & 0 \end{bmatrix},$$

dan

$$PR = \begin{bmatrix} 3 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 2 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 3 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 2 \\ 0 & 3 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 2 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 2 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 2 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 2 \\ 1 & 2 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 \end{bmatrix}.$$

Notasi PR menyatakan mangsa di ruang pencarian, dan FH_t adalah *fire hawk* di ruang pencarian.

3. Penentuan teritori

Selanjutnya, dihitung jumlah jarak antara *fire hawk* dan mangsa untuk menentukan teritori yang efektif. Pada generasi awal ini, terdapat 2 *fire hawk* sehingga dibentuk 2 teritori. Sebagai contoh untuk teritori pertama yaitu teritori elang api dengan nilai fungsi objektif terbaik didapatkan jarak sebagai berikut.

$$D_1^1 = 2,8284271247461903,$$

$$D_2^1 = 3,1622776601683795,$$

$$D_3^1 = 3,7416573867739413,$$

$$D_4^1 = 3,7416573867739413,$$

$$D_5^1 = 3,872983346207417,$$

$$D_6^1 = 4,0.$$

4. Update Posisi *Fire Hawk*

Selanjutnya, posisi dari *fire hawk* diperbarui sebagai berikut

$$FH_l^{new} = FH_l + (r_1 \times GB - r_2 \times FH_{near}), l = 1, 2, \dots, n.$$

Notasi FH_l^{new} menyatakan vektor posisi baru dari *fire hawk* (FH_l) ke- l , GB adalah solusi global terbaik di ruang pencarian yang dianggap sebagai api utama, FH_{near} adalah salah satu *fire hawk* lain di ruang pencarian, dan r_1 dan r_2 adalah bilangan acak yang terdistribusi seragam dalam rentang $(0, 1)$.

Misalkan $r_1 = 0.3$, $r_2 = 0.5$, $GB = [3, 0, 0, 1, 0, 1, 0, 0, 0, 0]$, dan $FH_{near} = [2, 1, 1, 0, 0, 1, 3, 0, 1, 0]$, maka untuk FH_1 , didapat

$$\begin{aligned} FH_1^{new} &= [3, 0, 1, 0, 1, 0, 2, 0, 0, 0] + (0.3 \times [3, 0, 0, 1, 0, 1, 0, 0, 0, 0] \\ &\quad - 0.5 \times [2, 1, 1, 0, 0, 1, 3, 0, 1, 0]) \\ &= [3, 0, 1, 0, 1, 0, 2, 0, 0, 0] + [0.9, 0, 0, 0.3, 0, 0.3, 0, 0, 0, 0] \\ &\quad - [1, 0.5, 0.5, 0, 0, 0.5, 1.5, 0, 0.5, 0] \\ &= [2.9, -0.5, 0.5, 0.3, 1, -0.2, 0.5, 0, -0.5, 0] \end{aligned}$$

5. Tempat Aman dan Update Posisi Mangsa

Tempat aman dari teritori 1 didapatkan dengan menghitung rata-rata dari semua mangsa di teritori 1 sebagai berikut

$$SP_1 = \begin{bmatrix} 1.66666667 \\ 1.33333333 \\ 0. \\ 1. \\ 0.33333333 \\ 0.66666667 \\ 0.33333333 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}.$$

Ketika kayu yang terbakar dijatuhkan, mangsa memutuskan untuk bersembunyi, melarikan diri, atau berlari ke arah *fire hawk* secara tidak sengaja sehingga terdapat pembaruan posisi mangsa. Sebagai contoh pada mangsa ke 1 jika di dalam teritori didapatkan posisi berikut

$$PR_1^{new} = PR_1 + (r_3 \times FH_1 - r_4 \times SP_1)$$

. Misalkan $r_3 = 0.4$ dan $r_4 = 0.6$, maka didapat

$$\begin{aligned} PR_1^{new} &= [3, 0, 0, 1, 0, 1, 0, 0, 0, 0] + (0.4 \times [3, 0, 1, 0, 1, 0, 2, 0, 0, 0]) \\ &\quad - 0.6 \times [1.6667, 1.3333, 0.3333, 0.6667, 0.6667, 0.3333, 0, 0.6667, 0, 1]) \\ &= [3, 0, 0, 1, 0, 1, 0, 0, 0, 0] + [1.2, 0, 0.4, 0, 0.4, 0, 0.8, 0, 0, 0] - \\ &\quad [1, 0.8, 0.2, 0.4, 0.4, 0.2, 0, 0.4, 0, 0.6] \\ &= [3.2, -0.8, 0.2, 0.6, 0, 0.8, 0.8, -0.4, 0, -0.6] \end{aligned}$$

6. Solusi

Proses di atas dilakukan sebanyak 100 generasi dan didapatkan keputusan terbaik

$$X = \begin{bmatrix} 2 & 1 & 1 & 0 & 0 & 1 & 3 & 1 & 1 & 2 \end{bmatrix},$$

yaitu 2 jam operasional untuk sepeda reguler dan 1 jam untuk premium di malam hari. Jika cuaca cerah, operator menambahkan 1 jam operasional untuk sepeda reguler, sehingga jam operasional yang digunakan adalah 3 jam reguler dan 1 jam premium. Jika cuaca hujan, operator menambahkan 1 jam operasional untuk sepeda premium, sehingga jam operasional yang digunakan adalah 2 jam reguler dan 2 jam premium. Strategi ini menghasilkan ekspektasi penggunaan sebesar 3,7 jam per hari.

BAB IV

SISTEM *ELECTRIC BIKE-SHARING* DI KOTA YOGYAKARTA

Dalam bab ini akan dibahas pembentukan model sistem *electric bike-sharing* mulai dari asumsi-asumsi, notasi, fungsi tujuan, serta kendala. Permasalahan utama dalam masalah optimalisasi ini yaitu memaksimalkan keuntungan berdasarkan pemilihan stasiun peminjaman, trip yang diterima, dan jumlah sepeda listrik yang perlu dibeli. Permasalahan ini akan dimodelkan menggunakan model *two-stage stochastic* dengan kendala pada tahap pertama terfokus pada perencanaan jangka panjang yaitu lokasi stasiun dan kapasitas sistem (sepeda dan pengisian daya) dan kendala pada tahap kedua berkaitan dengan operasi harian sistem. Model *two-stage stochastic* yang disusun ini akan diselesaikan dengan algoritma *fire-hawk optimizer*.

Dalam studi kasus ini, lokasi sistem *electric bike-sharing* yang digunakan dalam simulasi berlokasi di Yogyakarta dengan Tugu Yogyakarta sebagai titik tengah sistem. Selain itu skenario yang digunakan meliputi hari kerja (*weekdays*), Sabtu (*Saturday*), dan Minggu (*Sunday*), dengan probabilitas skenario Minggu yang paling besar, diikuti Sabtu, kemudian hari kerja memiliki probabilitas paling kecil.

4.1. Pemodelan Sistem *Electric Bike-Sharing*

Sistem *electric bike-sharing* muncul sebagai solusi transportasi perkotaan yang menjanjikan, menawarkan mobilitas yang fleksibel dan ramah lingkungan. Untuk merancang dan mengoperasikan sistem ini secara efisien memerlukan pendekatan canggih untuk mengatasi berbagai tantangan, termasuk ketidakpastian permintaan, manajemen pengisian daya, dan optimalisasi jaringan (Brandstätter dkk., 2017).

4.1.1. Kompleksitas Sistem *Electric Bike-Sharing*

Sistem *electric bike-sharing* memiliki beberapa karakteristik unik yang membuatnya lebih kompleks dibandingkan sistem *bike-sharing* tradisional:

1. Manajemen Baterai

Sistem menggunakan metode pengisian daya langsung (*direct charging*) yang mana sepeda listrik diisi dayanya saat terparkir di stasiun, bukan menggunakan sistem penukaran baterai (*battery swapping*).

2. Variabilitas Permintaan

Pola permintaan sangat bervariasi tergantung pada waktu, cuaca, dan acara khusus, memerlukan pendekatan yang fleksibel.

3. Optimalisasi Jaringan

Penempatan stasiun yang optimal harus mempertimbangkan tidak hanya aksesibilitas tetapi juga ketersediaan infrastruktur pengisian daya.

4. Keseimbangan Sistem

Menjaga keseimbangan antara ketersediaan sepeda dan slot pengisian di seluruh jaringan merupakan tantangan yang berkelanjutan.

4.1.2. Pendekatan Pemodelan Stokastik Dua Tahap

Untuk mengatasi kompleksitas ini, diadopsi pendekatan pemodelan stokastik dua tahap. Pendekatan ini digunakan karena cocok untuk digunakan dalam pengaplikasian permasalahan transportasi yang melibatkan ketidakpastian.

1. Tahap Pertama

Dalam tahap pertama ini keputusan yang diambil melibatkan perencanaan strategis jangka panjang yaitu terkait lokasi stasiun dan kapasitas sistem.

a. Lokasi Stasiun

Penentuan stasiun pengisian dibangun dengan mempertimbangkan beberapa faktor seperti kepadatan populasi, dan infrastruktur yang ada.

b. Kapasitas Sistem

Penentuan jumlah sepeda listrik yang akan dibeli dan kapasitas pengisian di setiap stasiun.

Keputusan ini dibuat sebelum mempertimbangkan permintaan, berdasarkan proyeksi dan analisis skenario. Kemudian setelah ketidakpastian diketahui, dapat ditentukan keputusan pada tahap kedua.

2. Tahap Kedua

Dalam tahap kedua keputusan yang diambil berkaitan dengan operasi harian sistem.

a. Alokasi Sepeda

Penentuan distribusi sepeda di seluruh jaringan untuk memenuhi permintaan.

b. Penerimaan Trip

Pengambilan trip mana yang akan diterima berdasarkan ketersediaan sepeda dan status pengisian daya.

c. Manajemen Pengisian Daya

Manajemen pengisian daya mengikuti beberapa protokol, setiap sepeda yang kembali dari trip langsung masuk ke mode pengisian daya, sepeda dalam status pengisian tidak tersedia untuk trip baru, dan waktu pengisian bergantung pada sisa kapasitas baterai saat kembali.

Keputusan ini dibuat secara *real-time* atau *near-real-time* setelah permintaan aktual diketahui. Keputusan pada tahap pertama dan tahap kedua diintegrasikan berdasarkan komponen utama model.

4.1.3. Komponen Utama Model

Model yang dikembangkan mengintegrasikan beberapa komponen penting meliputi:

1. Representasi Jaringan

Dalam hal ini digunakan graf berbobot untuk merepresentasikan jaringan stasiun dan rute perjalanan.

2. Pemodelan Permintaan

Pendekatan berbasis skenario akan diadopsi untuk menangkap variabilitas permintaan.

3. Dinamika Sistem

Graf lokasi *time-expanded* digunakan untuk memodelkan pergerakan sepeda, status pengisian daya, dan alokasi trip dengan setiap titik merepresentasikan kombinasi lokasi stasiun dan waktu diskrit, sementara *arc* menggambarkan aktivitas seperti pengisian daya (*waiting arc*), perjalanan (*travel arc*), serta penempatan awal dan akhir sepeda.

4. Kendala Anggaran dan Operasional

Batasan finansial dan teknis akan dipertimbangkan dalam pengambilan keputusan.

4.1.4. Struktur Model

Model diformulasikan dengan program linear bilangan bulat stokastik, struktur ini dapat mengintegrasikan komponen-komponen di atas untuk masalah optimisasi sistem *electric bike-sharing*.

1. Fungsi Tujuan

Tujuan adalah memaksimalkan ekspektasi keuntungan, menyeimbangkan pendapatan dari trip yang diterima dengan biaya operasional dan investasi.

2. Kendala Tahap Pertama

Kendala meliputi kendala anggaran, kapasitas sistem, dan lokasi stasiun.

3. Kendala Tahap Kedua

Kendala mencakup keseimbangan alur sepeda bergerak, manajemen pengisian daya, dan pemenuhan permintaan.

4. Variabel Keputusan

Kombinasi variabel biner dan kontinu untuk merepresentasikan keputusan diskrit (seperti pembangunan stasiun) dan kontinu (seperti alokasi sepeda).

4.2. Asumsi dan Notasi

Selanjutnya, dijelaskan mengenai asumsi-asumsi model, yang sebagian besar berdasarkan Brandstätter dkk. (2017) dengan beberapa modifikasi.

1. Perkiraan Permintaan

Diasumsikan tersedia perkiraan permintaan pelanggan untuk berbagai skenario (hari kerja) dengan probabilitas terkait. Setiap skenario mencakup set perkiraan trip yang mempertimbangkan aspek spasial dan temporal permintaan.

2. Informasi Trip

Setiap trip memiliki informasi lengkap tentang lokasi asal, tujuan, waktu mulai, dan waktu selesai. Informasi ini memungkinkan estimasi konsumsi baterai dan kontribusi profit untuk setiap trip.

3. Pengisian Daya

Setiap trip yang diterima mengharuskan sepeda listrik memulai perjalanan dalam keadaan baterai terisi penuh. Karakteristik pengisian daya diasumsikan linear terhadap waktu.

4. Kapasitas Stasiun

Setiap lokasi potensial stasiun pengisian memiliki kapasitas maksimum slot pengisian.

5. Jarak Berjalan

Pengguna diasumsikan bersedia berjalan dari/ke stasiun pengisian dalam radius tertentu dari lokasi asal/tujuan mereka.

6. Homogenitas Armada

Operator menggunakan armada sepeda listrik yang homogen untuk

mempermudah perencanaan dan perawatan.

7. Fokus Model

Model tidak mempertimbangkan aktivitas operasional staf seperti relokasi sepeda secara manual.

8. Waktu Perjalanan

Waktu perjalanan antara stasiun dianggap deterministik dan konstan.

9. Independensi Skenario

Permintaan dianggap independen antar skenario.

10. Keterbatasan Anggaran

Terdapat batasan anggaran untuk pembangunan stasiun dan pembelian sepeda.

11. Biaya operasional

Terdapat biaya operasional stasiun sejumlah stasiun yang dibangun dan biaya operasional sepeda sejumlah sepeda yang dibeli.

12. Reliabilitas Sepeda

Sepeda diasumsikan selalu dalam kondisi operasional yang baik dan tidak mempertimbangkan kemungkinan kerusakan atau perawatan.

13. Perilaku Pengguna

Pengguna diasumsikan selalu mengembalikan sepeda ke stasiun pengisian dan tidak meninggalkannya di lokasi lain.

14. Kebijakan Harga

Model mengasumsikan kebijakan harga yang tetap dan tidak mempertimbangkan variasi harga berdasarkan waktu atau permintaan.

Asumsi-asumsi ini membantu dalam menyederhanakan kompleksitas sistem *electric bike-sharing*, memungkinkan untuk membangun model matematis yang dapat diselesaikan sambil tetap menangkap esensi dari permasalahan *electric bike-sharing*. Penting untuk dicatat bahwa asumsi-asumsi ini mungkin perlu

dimodifikasi atau dilonggarkan, tergantung pada karakteristik spesifik dari sistem yang sedang dimodelkan.

Selanjutnya ditentukan parameter dan variabel yang digunakan berdasarkan asumsi yang telah disebutkan.

Kelompok	Notasi	Deskripsi
Himpunan dan Indeks	Ω	Himpunan skenario
	ω	Himpunan trip
	K^ω	Himpunan perjalanan dalam skenario ω
	S	Jumlah stasiun potensial
	H	Inisial jumlah sepeda yang akan dibeli
	T	Periode perencanaan
	$i, j \in S$	Lokasi potensial stasiun
	$t \in T$	Periode waktu, $T = \{0, \dots, T_{\max}\}$
	$h \in H$	Himpunan sepeda, $H = \{1, \dots, H_{\max}\}$
Parameter	π_ω	Probabilitas skenario ω
	p_k	Kontribusi keuntungan perjalanan k
	F_i	Biaya pembangunan stasiun di lokasi i
	F_{bike}	Biaya pembelian satu sepeda
	φ_i	Biaya operasional stasiun i
	ϕ	Biaya operasional per sepeda
	C_i	Kapasitas stasiun i
	W	Anggaran yang tersedia
	o_k, d_k	Asal dan tujuan perjalanan k
Parameter	s_k, e_k	Waktu mulai dan selesai perjalanan k
	b_k	Konsumsi baterai perjalanan k
	ρ	Laju pengisian daya per unit waktu
	r	Jarak maksimum yang diizinkan antar stasiun

Kelompok	Notasi	Deskripsi
Variabel Tahap Pertama	y_i	$y_i = \begin{cases} 1, & \text{jika stasiun } i \text{ dibangun} \\ 0, & \text{jika stasiun } i \text{ tidak dibangun} \end{cases},$ $y_i \in \{0, 1\}, i \in S$
	z_h	$z_h = \begin{cases} 1, & \text{jika sepeda } h \text{ dibeli} \\ 0, & \text{jika sepeda } h \text{ tidak dibeli} \end{cases},$ $z_h \in \{0, 1\}, h \in H$
Variabel Tahap Kedua	x_k^ω	Apakah perjalanan k diterima dalam skenario ω
	$x_k^{h,\omega}$	Apakah perjalanan k ditugaskan ke sepeda h dalam skenario ω
	$f_a^{h,\omega}$	Apakah sepeda h melintasi <i>arc</i> a dalam skenario ω
Graf dengan ekspansi waktu	$G^\omega = (V^\omega, A^\omega)$	Graf lokasi dengan ekspansi waktu dalam skenario ω
	A_I^ω	Himpunan <i>arc</i> alokasi awal dalam skenario ω
	A_W^ω	Himpunan <i>arc</i> tunggu dalam skenario ω
	A_C^ω	Himpunan <i>arc</i> pengumpulan akhir dalam skenario ω
	A_T^ω	Himpunan <i>arc</i> perjalanan dalam skenario ω
	r^ω	Titik akar dalam skenario ω
	s^ω	Titik tujuan dalam skenario ω

4.3. Formulasi Permasalahan

Dalam subbab ini akan diformulasikan masalah program linear bilangan bulat stokastik dua tahap sistem *electric bike-sharing*

1. Fungsi Objektif

Berdasarkan struktur model yang telah dipaparkan, dibentuk fungsi tujuan dengan memaksimalkan keuntungan yang diharapkan dengan menyeimbangkan pendapatan dari trip yang diterima dengan biaya operasional dan investasi. Fungsi objektif dari model ini memaksimalkan ekspektasi profit, yaitu

$$\max \sum_{\omega \in \Omega} \pi_{\omega} \left(\sum_{k \in K_{\omega}} p_k x_k^{\omega} \right) - \sum_{i \in S} \varphi_i y_i - \sum_{h \in H} \phi z_h. \quad (4.1)$$

Fungsi objektif (4.1) terbagi menjadi 3 bagian yaitu:

- a. Ekspektasi pendapatan: $\sum_{\omega \in \Omega} \pi_{\omega} \left(\sum_{k \in K_{\omega}} p_k x_k^{\omega} \right)$ jumlahan ini merupakan jumlahan dari keuntungan p_k dari setiap trip x_k dengan pembobotan probabilitas skenario π_{ω} .
- b. Biaya operasional stasiun: $\sum_{i \in S} \varphi_i y_i$ jumlahan ini merupakan jumlahan dari biaya operasi φ_i untuk setiap stasiun y_i yang dibangun.
- c. Biaya operasional sepeda: $\sum_{h \in H} \phi z_h$ jumlahan ini merupakan jumlahan biaya operasi ϕ dari setiap sepeda yang dibeli z_h .

2. Kendala Tahap Pertama

- a. Kendala anggaran:

$$\sum_{i \in S} F_i y_i + \sum_{h \in H} F_{\text{bike}} z_h \leq W, \quad (4.2)$$

Kendala ini memastikan total biaya pembangunan stasiun ($F_i y_i$) dan pembelian sepeda ($F_{\text{bike}} z_h$) tidak melebihi budget yang tersedia W .

- b. Kendala radius pembangunan stasiun:

$$y_i = 0 \text{ if } \exists j \in S, i \neq j : d(i, j) \leq r \text{ and } y_j \neq 0, \quad (4.3)$$

Kendala ini memastikan stasiun yang dibangun dengan jarak minimum r diantara stasiun. Jika tidak ada stasiun dalam radius r atau tidak ada

stasiun terdekat yang dibangun ($y_j = 0$), maka stasiun i harus dibangun ($y_i = 1$).

3. Kendala Tahap Kedua

a. Kendala penerimaan trip:

$$\sum_{h \in H} x_k^{h,\omega} = x_k^\omega \quad \forall \omega \in \Omega, k \in K_\omega, \quad (4.4)$$

kendala ini memastikan jika trip k diterima di skenario ω ($x_k^\omega = 1$), harus ditetapkan untuk satu sepeda saja.

b. Kendala alokasi sepeda:

$$x_k^{h,\omega} \leq z_h \quad \forall h \in H, \forall \omega \in \Omega, \forall k \in K_\omega, \quad (4.5)$$

kendala ini memastikan trip hanya bisa ditetapkan ke sepeda jika sepeda tersebut telah dibeli ($z_h = 1$).

c. Kendala kapasitas stasiun:

$$\sum_{h \in H} \sum_{a \in \delta^+(i_t) \cap (A_W^\omega \cup A_C^\omega)} f_a^{h,\omega} \leq C_i y_i \quad \forall \omega \in \Omega, \forall i_t \in V^\omega \setminus r^\omega, s^\omega, \quad (4.6)$$

Kendala ini memastikan jumlah sepeda pada sebuah stasiun (entah dalam waktu tunggu atau sudah terparkir) tidak boleh melebihi kapasitas stasiun C_i jika stasiun dibangun ($y_i = 1$).

d. Kendala konservasi arus:

$$f^{h,\omega}[\delta^-(i_t)] = f^{h,\omega}[\delta^+(i_t)] \quad \forall h \in H, \forall \omega \in \Omega, \forall i_t \in V^\omega \setminus r^\omega, s^\omega, \quad (4.7)$$

Kendala ini memastikan jumlah sepeda yang memasuki node (stasiun dalam waktu tertentu) harus sama dengan jumlah stasiun yang meninggalkan node tersebut, untuk mempertahankan konservasi aliran.

e. Kendala alokasi awal:

$$f^{h,\omega}[A_I^\omega] = z_h \quad \forall h \in H, \forall \omega \in \Omega, \quad (4.8)$$

Kendala ini memastikan tiap sepeda yang dibeli ($z_h = 1$) dialokasikan di stasiun awal dari skenario.

f. Kendala waktu pengisian daya:

$$\begin{aligned} f_a^{h,\omega} &\leq f_{a'}^{h,\omega} \quad \forall h \in H, \forall \omega \in \Omega, \forall k \in K^\omega, \forall a = (i_{s_k}, j_{e_k}) \in A_T^\omega(k), \\ &\quad \forall a' = (j_t, j_{t'}) \in A_W^\omega, t \geq e_k, t' \leq e_k + \left\lceil \frac{b_k}{\rho} \right\rceil, \end{aligned} \quad (4.9)$$

Kendala ini memastikan setelah trip, sepeda tetap pada stasiun untuk pengisian daya. Kendala ini memaksa sepeda untuk tetap di stasiun akhir untuk waktu yang diperlukan untuk pengisian ulang daya baterai yang telah terpakai pada trip ($\left\lceil \frac{b_k}{\rho} \right\rceil$).

4. Domain variabel

$$y_i \in (0, 1) \quad \forall i \in S, \quad (4.10)$$

$$z_h \in (0, 1) \quad \forall h \in H, \quad (4.11)$$

$$x_k^\omega \in (0, 1) \quad \forall k \in K^\omega, \forall \omega \in \Omega, \quad (4.12)$$

$$x_k^{h,\omega} \in (0, 1) \quad \forall h \in H, \forall \omega \in \Omega, \forall k \in K^\omega, \quad (4.13)$$

$$f_a^{h,\omega} \in (0, 1) \quad \forall h \in H, \forall \omega \in \Omega, \forall a \in A^\omega. \quad (4.14)$$

Kendala-kendala ini mendefinisikan semua variabel keputusan dalam bentuk biner (0 atau 1), yang menunjukkan bahwa stasiun dibangun atau tidak, sepeda dibeli atau tidak, trip diterima atau tidak, trip ditetapkan pada sepeda tertentu atau tidak, dan kendala operasional seperti waktu pengisian daya dan kapasitas stasiun.

Berdasarkan konstruksi di atas, permasalahan SCSLP dapat dituliskan

sebagai berikut:

$$\max \sum_{\omega \in \Omega} \pi_{\omega} \left(\sum_{k \in K_{\omega}} p_k x_k^{\omega} \right) - \sum_{i \in S} \varphi_i y_i - \sum_{h \in H} \phi z_h. \quad (4.15)$$

dengan kendala: **First-stage**

$$\sum_{i \in S} F_i y_i + \sum_{h \in H} F_{\text{bike}} z_h \leq W, \quad (4.16)$$

$$y_i = 0 \text{ if } \exists j \in S, i \neq j : d(i, j) \leq r \text{ and } y_j \neq 0. \quad (4.17)$$

Second-stage:

$$\sum_{h \in H} x_k^{h, \omega} = x_k^{\omega} \quad \forall \omega \in \Omega, k \in K_{\omega}, \quad (4.18)$$

$$x_k^{h, \omega} \leq z_h \quad \forall h \in H, \forall \omega \in \Omega, \forall k \in K_{\omega}, \quad (4.19)$$

$$\sum_{h \in H} \sum_{a \in \delta^+(i_t) \cap (A_W^{\omega} \cup A_C^{\omega})} f_a^{h, \omega} \leq C_i y_i \quad \forall \omega \in \Omega, \forall i_t \in V^{\omega} \setminus r^{\omega}, s^{\omega}, \quad (4.20)$$

$$f^{h, \omega}[\delta^-(i_t)] = f^{h, \omega}[\delta^+(i_t)] \quad \forall h \in H, \forall \omega \in \Omega, \forall i_t \in V^{\omega} \setminus r^{\omega}, s^{\omega}, \quad (4.21)$$

$$f^{h, \omega}[A_I^{\omega}] = z_h \quad \forall h \in H, \forall \omega \in \Omega, \quad (4.22)$$

$$f_a^{h, \omega} \leq f_{a'}^{h, \omega} \quad \forall h \in H, \forall \omega \in \Omega, \forall k \in K^{\omega}, \forall a = (i_{s_k}, j_{e_k}) \in A_T^{\omega}(k), \quad (4.23)$$

$$\forall a' = (j_t, j_{t'}) \in A_W^{\omega}, t \geq e_k, t' \leq e_k + \left\lceil \frac{b_k}{\rho} \right\rceil.$$

Domain Variabel:

$$y_i \in (0, 1) \quad \forall i \in S, \quad (4.24)$$

$$z_h \in (0, 1) \quad \forall h \in H, \quad (4.25)$$

$$x_k^{\omega} \in (0, 1) \quad \forall k \in K^{\omega}, \forall \omega \in \Omega, \quad (4.26)$$

$$x_k^{h, \omega} \in (0, 1) \quad \forall h \in H, \forall \omega \in \Omega, \forall k \in K^{\omega}, \quad (4.27)$$

$$f_a^{h, \omega} \in (0, 1) \quad \forall h \in H, \forall \omega \in \Omega, \forall a \in A^{\omega}. \quad (4.28)$$

4.4. Simulasi

Selanjutnya akan dibahas penyelesaian masalah two-stage stochastic untuk memaksimalkan profit dari sistem *bike-sharing* berbasis stasiun. Masalah optimisasi ini melibatkan lokasi serta biaya pembangunan stasiun yang optimal,

trip yang diterima beserta profit kontribusi tiap tripnya, dan juga jumlah dan harga sepeda yang akan dibeli. Penyelesaian model ini akan menggunakan algoritma metaheuristik yaitu *Fire Hawk Optimizer*.

Selanjutnya wilayah yang digunakan dalam simulasi berlokasi di Yogyakarta dengan pusat lokasi berada di Tugu Yogyakarta. Lokasi potensial untuk pembangunan stasiun dan juga trip-trip tiap skenario di-generate secara acak. Kemudian, biaya pembangunan stasiun, profit kontribusi, dan juga harga sepeda tidak menggunakan mata uang khusus tapi di-generate semasuk akal mungkin seperti harga sepeda dan pembangunan stasiun yang terlampaui tinggi selisihnya.

4.4.1. Generating Data

Untuk mensimulasikan sistem *electric bike-sharing* di Yogyakarta, digunakan data yang dihasilkan secara acak dengan beberapa parameter dan batasan yang realistis. Berikut adalah rincian data yang dihasilkan.

1. Lokasi Stasiun

Dalam kasus ini diambil 15 titik potensial untuk pembangunan stasiun pengisian daya (i). Lokasi ini mencakup beberapa tempat strategis di Yogyakarta, termasuk Stasiun Tugu, Malioboro Mall, Pasar Beringharjo, Bank Indonesia Yogyakarta, Keraton Yogyakarta, dan Universitas Gadjah Mada, serta beberapa lokasi acak lainnya.

2. Kapasitas Stasiun

Setiap stasiun potensial memiliki kapasitas (C_i) yang bervariasi antara 5 hingga 10 slot sepeda.

3. Skenario

Dalam kasus ini dibuat 3 skenario (Ω) berdasarkan hari (Senin-Jumat) yaitu hari kerja, Sabtu, dan Minggu dengan masing-masing skenario memiliki probabilitas (π_ω) yaitu 0.30, 0.34, dan 0.36 untuk mensimulasikan variasi permintaan.

4. Trip

Setiap skenario memiliki 30 trip (K^ω) yang dibangkitkan secara acak dengan mempertimbangkan lokasi stasiun, jarak maksimum baterai, dan waktu perjalanan.

5. Waktu Perencanaan

Periode perencanaan (T_{max}) ditetapkan selama 15 unit waktu.

6. Anggaran

Anggaran total (W) ditetapkan sebesar 1.200.000.000.

7. Biaya Pembukaan Stasiun

Biaya pembukaan stasiun (F_i) dihitung berdasarkan komponen tetap (α) antara 10.000.000 hingga 50.000.000, dan komponen variabel (β) antara 2.000.000 hingga 5.000.000 per kapasitas.

8. Sepeda Listrik

Jumlah maksimum sepeda listrik yang dapat dibeli tidak ditentukan secara eksplisit, tetapi dibatasi oleh anggaran dan kapasitas stasiun.

9. Biaya Operasional

Biaya operasional per sepeda (ϕ) dan per stasiun (φ_i) tidak ditentukan secara eksplisit dalam data yang dihasilkan.

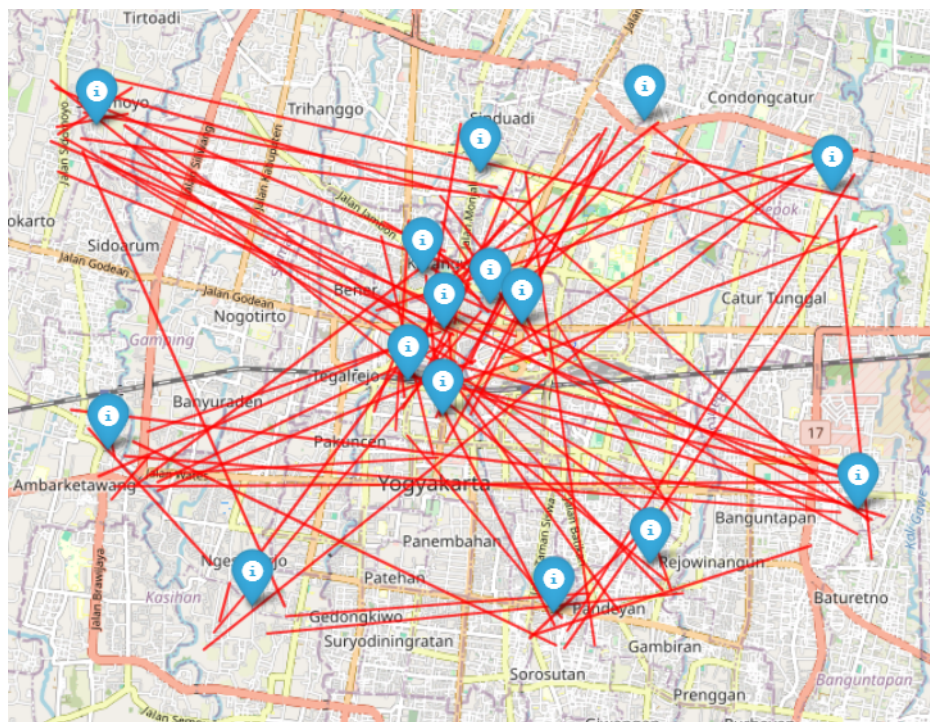
10. Profit

Profit per trip dihitung berdasarkan durasi perjalanan, dengan rate 10.000.000 per unit waktu.

11. Konsumsi Baterai

Konsumsi baterai untuk setiap trip dihitung berdasarkan jarak perjalanan relatif terhadap jarak maksimum baterai (50 km).

Data ini digunakan untuk membuat peta interaktif yang menunjukkan lokasi potensial stasiun dan trip yang mungkin terjadi. Peta ini disimpan sebagai file HTML untuk visualisasi lebih lanjut.



Gambar 4.1 Peta Lokasi Potensial Pembangunan Stasiun

4.5. Prosedur Algoritma *Fire Hawk Optimizer*

Algoritma *Fire Hawk Optimizer* (FHO) diaplikasikan untuk menyelesaikan permasalahan optimisasi lokasi stasiun pengisian daya dalam sistem *electric bike-sharing*. Berikut adalah langkah-langkah utama dari algoritma FHO:

4.5.1. Inisialisasi

Langkah pertama dilakukan dengan menentukan jumlah kandidat solusi (X) sebagai vektor posisi dari elang api dan mangsa. Setiap kandidat solusi merepresentasikan konfigurasi potensial dari sistem *electric bike-sharing*, termasuk lokasi stasiun pengisian daya, alokasi sepeda listrik, dan trip yang diterima. Nilai awal ditentukan secara acak dalam batas-batas yang ditentukan oleh kendala sistem, seperti anggaran dan kapasitas maksimum stasiun.

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_i \\ \vdots \\ X_N \end{bmatrix} = \begin{bmatrix} x_1^1 & x_1^2 & \cdots & x_1^a & \cdots & x_1^d \\ x_2^1 & x_2^2 & \cdots & x_2^a & \cdots & x_2^d \\ \vdots & \vdots & & \vdots & & \vdots \\ x_b^1 & x_b^2 & \cdots & x_b^a & \cdots & x_b^d \\ \vdots & \vdots & & \vdots & & \vdots \\ x_N^1 & x_N^2 & \cdots & x_N^a & \cdots & x_N^d \end{bmatrix}, \quad (4.29)$$

dan

$$x_i^j(0) = x_{i,\min}^j + \text{rand}(x_{i,\max}^j - x_{i,\min}^j), \quad i = 1, 2, \dots, N, \text{ dan } j = 1, 2, \dots, d, \quad (4.30)$$

dengan

1. X_i merepresentasikan kandidat solusi ke i di ruang pencarian,
2. d merepresentasikan dimensi dari permasalahan (misalnya, jumlah lokasi potensial stasiun plus jumlah sepeda yang dapat dialokasikan),
3. N adalah jumlah total kandidat solusi di ruang pencarian,
4. x_i^j adalah variabel keputusan ke j dari kandidat solusi ke i (misalnya, keputusan untuk membangun stasiun di lokasi tertentu atau jumlah sepeda yang dialokasikan),
5. $x_i^j(0)$ merepresentasikan posisi awal dari kandidat solusi,
6. $x_{i,\min}^j$ dan $x_{i,\max}^j$ adalah batas bawah dan batas atas dari variabel keputusan ke j untuk kandidat solusi ke i ,
7. rand adalah bilangan acak yang terdistribusi seragam dalam rentang $[0,1]$,
dan
8. $i = 1, 2, \dots, N$ dan $j = 1, 2, \dots, d$.

4.5.2. Evaluasi dan Seleksi

Fungsi objektif ini merepresentasikan total profit yang diharapkan dari operasi sistem *electric bike-sharing*, dengan mempertimbangkan pendapatan dari perjalanan yang diterima, biaya pembangunan stasiun, dan biaya operasional.

Beberapa kandidat solusi dengan nilai fungsi objektif terbaik direpresentasikan sebagai elang api, sedangkan kandidat solusi lain dianggap sebagai mangsa. Elang api yang dipilih dimanfaatkan untuk menyebarkan api di sekitar mangsa di ruang pencarian untuk memudahkan perburuan. Hal ini dapat diinterpretasikan sebagai eksplorasi area di sekitar solusi-solusi terbaik untuk menemukan konfigurasi sistem *electric bike-sharing* yang lebih optimal.

Solusi terbaik global diasumsikan sebagai api utama yang pertama kali dimanfaatkan oleh elang api untuk menyebarkan api di ruang pencarian. Ini merepresentasikan konfigurasi terbaik sistem *electric bike-sharing* yang ditemukan sejauh ini. Solusi-solusi yang direpresentasikan sebagai elang api dan mangsa dinyatakan dinyatakan sebagai berikut.

$$PR = \begin{bmatrix} PR_1 \\ PR_2 \\ \vdots \\ PR_k \\ \vdots \\ PR_m \end{bmatrix}, \quad (4.31)$$

dan

$$FH = \begin{bmatrix} FH_1 \\ FH_2 \\ \vdots \\ FH_l \\ \vdots \\ FH_n \end{bmatrix}, \quad (4.32)$$

dengan

1. notasi PR_k adalah mangsa ke- k di ruang pencarian berdasarkan jumlah angka dari mangsa m . Mangsa-mangsa ini merepresentasikan solusi-solusi yang kurang optimal untuk sistem *electric bike-sharing*,
2. notasi FH_l adalah elang api ke l berdasarkan jumlah dari n elang api di ruang pencarian. Elang-elang api merepresentasikan solusi-solusi terbaik yang ditemukan untuk sistem *electric bike-sharing*,
3. jumlah total mangsa dinotasikan m (solusi kurang optimal), dan
4. jumlah total elang api dinotasikan n (solusi terbaik).

4.5.3. Penentuan Teritori

Pada fase berikutnya, dihitung jumlah jarak antara elang api dan mangsa untuk menentukan teritori yang efektif. Jarak ini merepresentasikan perbedaan antara solusi-solusi dalam ruang pencarian sistem *electric bike-sharing*. Jarak mangsa terpendek terhadap tiap elang api ditentukan, dengan mangsa terdekat dari elang api pertama memiliki fungsi objektif terbaik. Teritori dari burung lain dipertimbangkan dari rata-rata jarak mangsa yang tersisa.

$$D_k^l = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}, \quad l = 1, 2, \dots, n, \text{ dan } k = 1, 2, \dots, m, \quad (4.33)$$

dengan

1. notasi D_k^l adalah jumlah jarak antara elang api ke- l dan mangsa ke- m dalam ruang solusi sistem *electric bike-sharing*,
2. jumlah mangsa di ruang pencarian dinotasikan m ,
3. jumlah elang api di ruang pencarian dinotasikan n ,
4. Koordinat dari elang api dan mangsa di ruang pencarian direpresentasikan dengan (x_1, y_1) dan (x_2, y_2) . Untuk sistem *electric bike-sharing*, koordinat ini bisa merepresentasikan berbagai aspek konfigurasi sistem, seperti lokasi stasiun, alokasi sepeda, dan trip yang diterima, dan
5. $l = 1, 2, \dots, n$, dan $k = 1, 2, \dots, m$.

4.5.4. Pembaruan Posisi Elang Api

Elang api mengumpulkan kayu yang terbakar dari api utama untuk memantik api di area yang dipilih. Dalam algoritma optimisasi, hal ini merepresentasikan proses eksplorasi solusi baru berdasarkan solusi terbaik yang ada. Tiap burung membawa kayu yang terbakar lalu menjatuhkannya di teritori tertentu untuk memaksa mangsanya melarikan diri. Beberapa burung akan menggunakan kayu yang terbakar dari teritori elang api yang lain. Proses ini direpresentasikan dalam persamaan berikut:

$$FH_l^{new} = FH_l + (r_1 \times GB - r_2 \times FH_{near}), \quad l = 1, 2, \dots, n, \quad (4.34)$$

dengan

1. notasi FH_l^{new} adalah new vektor posisi dari elang api (FH_l) ke- l , merepresentasikan solusi baru untuk konfigurasi sistem *electric bike-sharing*,
2. notasi GB adalah solusi terbaik global di ruang pencarian yang dianggap sebagai api utama, merepresentasikan konfigurasi terbaik sistem yang ditemukan sejauh ini,

3. notasi FH_{near} adalah salah satu elang api lain di ruang pencarian, merepresentasikan solusi terbaik lainnya, dan
4. notasi r_1 dan r_2 adalah angka acak yang terdistribusi seragam dalam rentang $(0, 1)$.

4.5.5. Pembaruan Posisi Mangsa

Pergerakan mangsa di dalam teritori dari tiap elang api dianggap sebagai kunci dari perilaku hewan untuk proses pembaruan posisi. Dalam optimisasi sistem *electric bike-sharing*, hal ini merepresentasikan penyesuaian solusi-solusi yang kurang optimal berdasarkan solusi-solusi terbaik yang ada. Ketika kayu yang terbakar dijatuhkan, mangsa memutuskan untuk bersembunyi, melarikan diri, atau berlari ke arah elang api secara tidak sengaja. Proses ini direpresentasikan dalam persamaan berikut:

$$PR_q^{new} = PR_q + (r_3 \times FH_l - r_4 \times SP_l), \quad l = 1, 2, \dots, n, \text{ dan } q = 1, 2, \dots, r, \quad (4.35)$$

dan

$$PR_q^{new} = PR_q + (r_5 \times FH_{Alter} - r_6 \times SP), \quad l = 1, 2, \dots, n, \text{ dan } q = 1, 2, \dots, r, \quad (4.36)$$

dengan

1. notasi PR_q^{new} adalah vektor posisi baru dari mangsa ke- q (PR_q) yang dimangsa oleh elang api ke- l (FH_l), merepresentasikan solusi baru yang dihasilkan dari solusi yang kurang optimal,
2. notasi SP_l adalah tempat aman di dalam teritori elang api ke- l , merepresentasikan rata-rata dari solusi-solusi dalam teritori tertentu,
3. notasi SP adalah tempat aman di luar teritori elang api ke- l , merepresentasikan rata-rata global dari semua solusi,
4. notasi FH_{Alter} adalah salah satu elang api lain di ruang pencarian, merepresentasikan solusi terbaik lainnya,

5. notasi r_3, r_4, r_5, r_6 adalah bilangan acak yang berdistribusi seragam dalam rentang $(0, 1)$, dan
6. $l = 1, 2, \dots, n$, dan $q = 1, 2, \dots, r$.

4.5.6. Tempat Aman

Tempat aman adalah lokasi di mana sebagian besar hewan berkumpul untuk tempat berlindung dari ancaman. Dalam optimisasi, hal ini merepresentasikan area di ruang solusi yang cenderung menghasilkan solusi yang baik. SP_l dan SP diformulasikan sebagai berikut:

$$SP_l = \frac{\sum_{q=1}^r PR_q}{r}, \quad l = 1, 2, \dots, n, \text{ dan } q = 1, 2, \dots, r, \quad (4.37)$$

dan

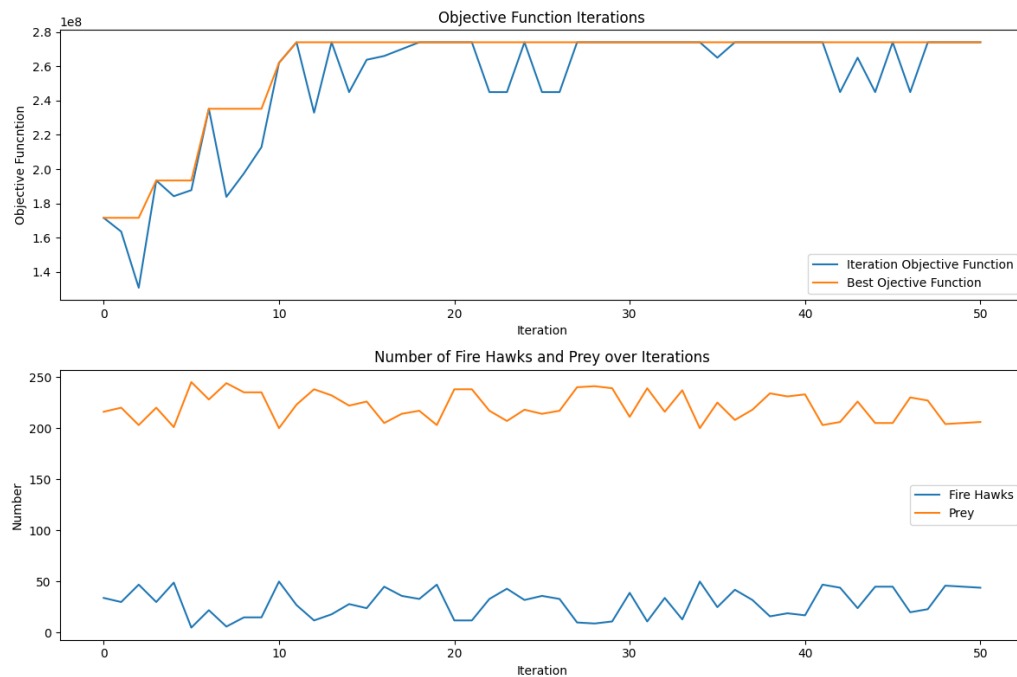
$$SP = \frac{\sum_{k=1}^m PR_k}{m}, \quad k = 1, 2, \dots, m, \quad (4.38)$$

dengan

1. notasi PR_q adalah mangsa ke- q yang dikelilingi oleh elang api ke- l (FH_l), merepresentasikan solusi yang kurang optimal dalam teritori tertentu.
2. Notasi PR_k adalah mangsa ke- k di ruang pencarian, merepresentasikan semua solusi yang kurang optimal,
3. notasi r adalah jumlah mangsa dalam teritori tertentu,
4. notasi m adalah jumlah total mangsa di seluruh ruang pencarian, dan
5. $l = 1, 2, \dots, n$, $q = 1, 2, \dots, r$, dan $k = 1, 2, \dots, m$

4.5.7. Proses Iterasi dan Kriteria Penghentian

Simulasi *Fire Hawk Optimizer* terhadap formulasi yang telah dipaparkan di atas dengan hasil sebagai berikut.



Gambar 4.2 Iterasi Algoritma Elang Api Optimizer

Algoritma *Fire Hawk Optimizer* melakukan proses iterasi untuk terus memperbaiki solusi seperti yang terlihat pada Gambar 4.2. Pada setiap iterasi:

1. posisi elang api dan mangsa diperbarui menggunakan persamaan yang telah didefinisikan sebelumnya,
2. nilai fungsi objektif dari setiap solusi baru dievaluasi,
3. solusi terbaik global (*GB*) diperbarui jika ditemukan solusi yang lebih baik, dan
4. proses penentuan teritori, pembaruan posisi elang api, dan pembaruan posisi mangsa diulangi.

Iterasi akan terus berlanjut hingga salah satu kriteria penghentian terpenuhi. Kriteria penghentian dapat berupa (Talbi (2009)):

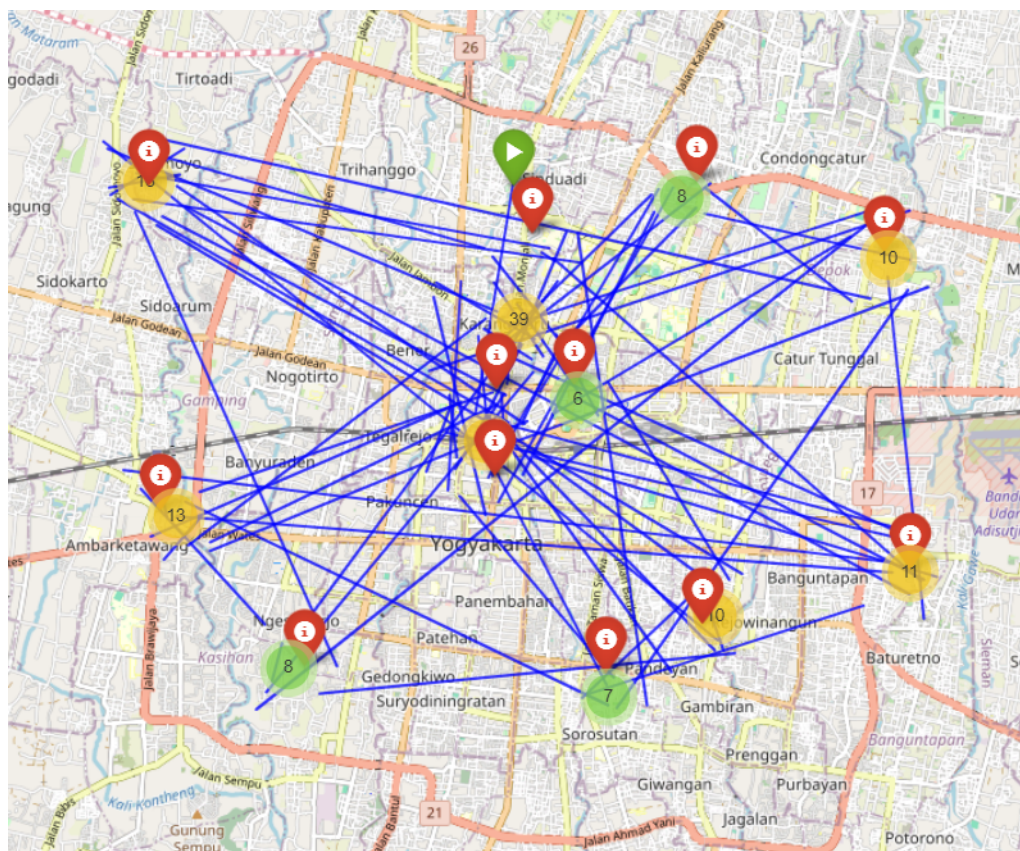
1. mencapai jumlah iterasi maksimum yang telah ditentukan sebelumnya,
2. mencapai batas waktu komputasi yang telah ditetapkan, dan

3. mencapai tingkat konvergensi tertentu, dimana perbaikan solusi antar iterasi menjadi sangat kecil atau tidak signifikan.

Setelah kriteria penghentian terpenuhi, algoritma akan berhenti dan mengembalikan solusi terbaik yang ditemukan sebagai konfigurasi optimal untuk sistem *electric bike-sharing*. Solusi ini mencakup lokasi stasiun pengisian daya yang harus dibangun, alokasi sepeda listrik yang optimal, dan trip yang harus diterima.

4.5.8. Output Simulasi

Simulasi dilakukan dengan menggunakan *Python* dan diperoleh solusi optimal dengan menggunakan algoritma *Fire Hawk Optimizer*. Gambar 4.3 menunjukkan lokasi stasiun yang dibuka dan trip yang diterima, dijelaskan juga hasil dari output, termasuk nilai untuk setiap variabel dan hasil fungsi objektif.



Gambar 4.3 Peta Stasiun yang Dibangun dan Trip yang Diterima

Variabel dan Hasil

1. **Stasiun yang Dibangun ((y))** Solusi optimal menunjukkan bahwa dari 15 lokasi potensial, 12 stasiun dipilih untuk dibangun, yaitu stasiun [1, 2, 3, 6, 7, 8, 9, 10, 11, 12, 13, 14]. Lokasi-lokasi ini meliputi (tidak dalam urutan).

- (a) Sidomoyo,
- (b) Monjali,
- (c) Kentungan,
- (d) Seturan,
- (e) Tugu,
- (f) Cik di Tiro,
- (g) Malioboro,
- (h) Simpang Empat Pelem,
- (i) Ngastiharjo,
- (j) Mergangsan,
- (k) Rejowinangun, dan
- (l) Bandara Adisucipto.

Hal ini menandakan bahwa 80% dari lokasi potensial dianggap strategis untuk mendukung operasi sistem *bike-sharing*. Pemilihan lokasi ini mencakup berbagai area penting di Yogyakarta, termasuk pusat kota, area wisata, kawasan pendidikan, area bisnis dan pemukiman, serta titik-titik transportasi penting.

2. **Sepeda yang Dibeli (b)**

Model merekomendasikan pembelian 55 sepeda listrik. Jumlah ini mewakili keseimbangan optimal antara memenuhi permintaan yang diperkirakan dan mengelola biaya inventaris. Berikut lokasi terakhir sepeda listrik dari seluruh skenario.

- (a) Sidomoyo : 8 sepeda,
- (b) Monjali : 1 sepeda,
- (c) Ketungan : 2 sepeda,
- (d) Seturan : 7 sepeda,
- (e) Tugu : 4 sepeda,
- (f) Cik di Tiro : 5 sepeda,
- (g) Malioboro : 7 sepeda,
- (h) Simpang Empat Pelem : 5 sepeda,
- (i) Ngastiharjo : 3 sepeda,
- (j) Mergangsan : 8 sepeda,
- (k) Rejowinangun : 7 sepeda, dan
- (l) Bandara Adisucipto : 4 sepeda.

3. Trip yang Diterima (x)

Tingkat penerimaan trip bervariasi di antara skenario:

- (a) Skenario 0 (Weekdays): 25/30 trip diterima (83.33%)
- (b) Skenario 1 (Saturday): 28/30 trip diterima (93.33%)
- (c) Skenario 2 (Sunday): 27/30 trip diterima (90.00%)

Tingkat penerimaan yang tinggi ini menunjukkan bahwa sistem yang diusulkan mampu memenuhi sebagian besar permintaan yang diproyeksikan, dengan kinerja terbaik pada hari Sabtu.

Secara keseluruhan, solusi ini menunjukkan desain sistem *bike-sharing* yang efisien terhadap permintaan dengan potensi keuntungan yang signifikan. Namun, masih ada ruang untuk penyempurnaan lebih lanjut, terutama dalam hal peningkatan utilisasi sepeda dan kemungkinan optimasi lebih lanjut dari lokasi stasiun.

BAB V

PENUTUP

Pada bab terakhir ini dituliskan kesimpulan dan saran-saran yang dapat diambil berdasarkan materi-materi yang telah dibahas pada bab-bab sebelumnya.

5.1. Kesimpulan

Berdasarkan pembahasan yang telah dilakukan sebelumnya, penulis menarik beberapa kesimpulan berikut:

1. Masalah *electric bike-sharing system* dapat dimodelkan dengan masalah stokastik dua tahap.
2. Permasalahan-permasalahan optimisasi seperti *electric bike-sharing system* dapat diselesaikan dengan algoritma *metaheuristic* yang salah satunya yaitu *Fire Hawk Optimizer*. Simulasi *electric bike-sharing system* di kota Yogyakarta dengan parameter-parameter yang telah dipaparkan sebelumnya memberikan gambaran bahwa *electric bike-sharing system* ini dapat terealisasi dengan nilai keuntungan sebesar 274.030.876 dengan stasiun-stasiun yang dibangun yaitu, Sidomoyo, Monjali, Kentungan, Seturan, Tugu, Cik di Tiro, Malioboro, Simpang Empat Pelem, Ngastiharjo, Mergangsan, Rejowinangun, Bandara Adisucipto serta 55 sepeda listrik yang dibeli dan 80 trip yang diterima.

5.2. Saran

Formulasi *two-stage stochastic* pada permasalahan *electric bike-sharing system* dan penyelesaiannya dengan menggunakan algoritma *Fire Hawk Optimizer* serta beberapa kesimpulan yang telah dituliskan di atas, penulis ingin menyampaikan beberapa saran sebagai berikut.

1. Penelitian selanjutnya dapat dikembangkan dengan membuat formulasi yang lebih efisien sehingga proses komputasi tidak memakan daya ataupun waktu yang lama.
2. Perlu dilakukan penelitian lebih lanjut terutama *benchmarking* metode penyelesaian permasalahan *electric bike-sharing system* seperti dengan algoritma *metaheuristic* lain misal *particle swarm optimization*, *simulated annealing*, algoritma genetia, ataupun yang lain.

DAFTAR PUSTAKA

- Azizi, M. and Talatahari, S. and Gandomi, A. H., 2023, Fire Hawk Optimizer: a novel metaheuristic algorithm, *Artificial Intelligence Review*, 56(1), 287–363.
- Bikeplus, 2016. *Shared Electric Bike Programme Report*. Carplus Trust, n.p..
- Billingsley, P., 1995, *Probability and Measure*, 3rd ed., John Wiley & Sons, New York.
- Birge, J. R., Louveaux, F., 2011, *Introduction to stochastic programming*, Springer Science & Business Media, London.
- Boyd, S., Vandenberghe, L., 2004, *Convex optimization*, Cambridge University Press, Cambridge.
- BPS, 2020, *Statistik Kepariwisata Daerah Istimewa Yogyakarta 2019*, Badan Pusat Statistik Provinsi D.I. Yogyakarta.
- Brandstätter, G. and Kahr, M. and Leitner, M., 2017, Determining optimal locations for charging stations of electric car-sharing systems under stochastic demand, *Transportation Research Part B: Methodological*, 104, 17–35.
- Campbell, A. A., Cherry, C. R., Ryerson, M. S., Yang, X., 2016. Factors influencing the choice of shared bicycles and shared electric bikes in Beijing. *Transportation Research Part C: Emerging Technologies*, 67, 399-414.
- Casella, G. and Berger, R. L., 2002, *Statistical Inference*, Duxbury.
- Chen, T. D. and Kockelman, K. M. and Khan, M., 2020, The electric vehicle charging station location problem: A parking-based assignment method for Seattle, *Transportation Research Part D: Transport and Environment*, 78, 102132.

- Fishman, E., 2020, Bikeshare: A Review of Recent Literature, *Transport Reviews*, 40(6), 744–767.
- Fyhri, A., Fearnley, N., 2017. Effects of e-bikes on bicycle use and mode share. *Transportation Research Part D: Transport and Environment*, 36, 45-52.
- Glover, F., 1986. *Future paths for integer programming and links to artificial intelligence*. Computers & operations research, 13(5), 533-549.
- Grimmett, G. and Stirzaker, D., 2020, *Probability and Random Processes*, Oxford University Press, New York.
- Haight, F. A., 1967, *Handbook of the Poisson Distribution*, John Wiley & Sons, New York.
- Hastie, T. and Tibshirani, R. and Friedman, J., 2009, *The elements of statistical learning: data mining, inference, and prediction*, Springer Science & Business Media, New York.
- Holland, J. H., 1992, *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*, MIT Press, Cambridge, MA.
- Hunter, J. D., 2007, Matplotlib: A 2D graphics environment, *Computing in Science & Engineering*, 9(3), 90–95.
- IEA, 2021, *Global EV Outlook 2021*, International Energy Agency, Paris.
- Langford, B. C., Cherry, C. R., Bassett Jr, D. R., Fitzhugh, E. C., Dhakal, N., 2013. Comparing physical activity of pedal-assist electric bikes with walking and conventional bicycles. *Journal of Transport & Health*, 4(1), 86-93.
- Ling, Z., Cherry, C. R., Yang, H., Jones, L. R., 2017. From e-bike to car: A study on factors influencing motorization of e-bike users across China. *Transportation Research Part D: Transport and Environment*, 41, 50-63.

- Ma, X., Yuan, Y., Van Oort, N., Hoogendoorn, S., 2020. Bike-sharing systems' impact on modal shift: A case study in Delft, the Netherlands. *Journal of Cleaner Production*, 259, 120846.
- MacMichael, S., 2021, Plan for compulsory e-bike insurance in Spain faces fierce opposition, *road.cc*, <https://road.cc/content/news/mandatory-bike-insurance-spain-291997> (diakses pada 20 September 2024).
- McKinney, W., 2011, pandas: a foundational Python library for data analysis and statistics, *Python for High Performance and Scientific Computing*, 14(9), 1–9.
- Nocedal, J., Wright, S., 2006. *Numerical optimization*. Springer Science & Business Media, New York, NY.
- O'Brien, O. and Cheshire, J. and Batty, M., 2014, Mining bicycle sharing data for generating insights into sustainable transport systems, *Journal of Transport Geography*, 34, 262–273.
- Pemkot Yogyakarta, 2020, *Rencana Pembangunan Jangka Menengah Daerah Kota Yogyakarta 2017-2022*, Pemerintah Kota Yogyakarta.
- Powell, W. B., 2019, A unified framework for stochastic optimization, *European Journal of Operational Research*, 275(3), 795–821.
- Ross, S. M., 1996, *Stochastic Processes*, John Wiley & Sons, New York.
- Ross, S. M., 2014, *Introduction to Probability Models*, Academic Press, Burlington, MA.
- Shaheen, S., Cohen, A., Chan, N., Bansal, A., 2019, Sharing strategies: carsharing, shared micromobility (bikesharing and scooter sharing), transportation network companies, microtransit, and other innovative mobility modes, *Transportation, Land Use, and Environmental Planning*, 237–262.

- Sun, Y., Mobasheri, A., Hu, X., Wang, W., 2018. Investigating impacts of environmental factors on the cycling behavior of bicycle-sharing users. *Sustainability*, 9(6), 1060.
- Talbi, E. G., 2009, Metaheuristics: from design to implementation, *John Wiley & Sons*, Hoboken, NJ, 37–39.
- UNFCCC, 2015, *Paris Agreement*, United Nations, Paris.
- United Nations General Assembly, 2015, Transforming our world: The 2030 agenda for sustainable development, *United Nations General Assembly*, A/RES/70/1. Available at: <https://sustainabledevelopment.un.org/post2015/transformingourworld>.
- Wilson, R. J., 1996, *Introduction to Graph Theory*, Longman, Harlow.
- Winston, W. L. and Goldberg, J. B., 2004, *Operations research: applications and algorithms*, Vol. 3, Thomson Brooks/Cole, Belmont.
- Zhu, L. and Gonder, J. and Lin, Z., 2016, Charging station location problem of plug-in electric vehicles, *Journal of Transport Geography*, 52, 11–22.

LAMPIRAN A

DATA

Tabel 1.1 Data Stasiun

station_id	capacity	opening_cost	latitude	longitude
0	8	64475368	-7.7891	110.3614
1	7	29062142	-7.7931	110.3656
2	8	53604018	-7.7824	110.3750
3	5	54169124	-7.7830	110.3657
4	5	70770109	-7.7801	110.3712
5	9	79681206	-7.7764	110.3632
6	6	68409814	-7.816507590225311	110.37882377262206
7	9	58627082	-7.764545623765107	110.36998599777583
8	9	87155237	-7.758517422697779	110.38959483165141
9	10	56627802	-7.804273754947625	110.4149690263108
10	5	64476799	-7.758885258655926	110.32442053616289
11	9	55915390	-7.7667612206468375	110.41185548735434
12	10	64731553	-7.810665575510653	110.39030027249711
13	5	57354989	-7.797234234968725	110.32582055624444
14	7	52547174	-7.815663727363731	110.34298468522442

Tabel 1.2 Weekday Data Skenario dan Profit

Scenario	Trip ID	Destination	Profit
Weekdays	0	(-7.792931179673451, 110.36593183784399)	30000000
Weekdays	1	(-7.775647815981623, 110.36724251595128)	20000000
Weekdays	2	(-7.800520210506863, 110.41480096138257)	20000000
Weekdays	3	(-7.769890027550567, 110.41249566091625)	20000000
Weekdays	4	(-7.812769098888491, 110.39224477396209)	20000000
Weekdays	5	(-7.776569094401348, 110.37288896664307)	30000000
Weekdays	6	(-7.805526526949287, 110.41676671215377)	10000000
Weekdays	7	(-7.770438134482901, 110.36159240469725)	30000000
Weekdays	8	(-7.805665405269387, 110.41645859073208)	20000000
Weekdays	9	(-7.805541747688057, 110.38990179513809)	10000000
Weekdays	10	(-7.761427938777749, 110.32886417224998)	30000000
Weekdays	11	(-7.800388527425106, 110.32471770304838)	20000000
Weekdays	12	(-7.801710289960986, 110.32659754105705)	30000000
Weekdays	13	(-7.800040916629955, 110.41611241279443)	20000000
Weekdays	14	(-7.754633697351422, 110.32873446979093)	30000000
Weekdays	15	(-7.796324586945069, 110.36600172032915)	20000000
Weekdays	16	(-7.763738531763849, 110.32534399418394)	30000000
Weekdays	17	(-7.771354263023364, 110.41488578879236)	30000000
Weekdays	18	(-7.803140288211153, 110.32618939992823)	10000000
Weekdays	19	(-7.764528569476906, 110.3198398775548)	30000000
Weekdays	20	(-7.793381694707156, 110.36545130279752)	30000000
Weekdays	21	(-7.783222844979383, 110.37109768424821)	30000000
Weekdays	22	(-7.8141712124257905, 110.38299766811353)	20000000
Weekdays	23	(-7.805631999110501, 110.39071673422443)	30000000
Weekdays	24	(-7.785064224320225, 110.36569285174483)	10000000
Weekdays	25	(-7.820291802236042, 110.38370114302596)	10000000
Weekdays	26	(-7.788425603038559, 110.3618555932266)	10000000
Weekdays	27	(-7.781567806650796, 110.36698710627229)	30000000
Weekdays	28	(-7.760737076884888, 110.32272475287424)	30000000
Weekdays	29	(-7.76346096101143, 110.39499553264821)	10000000

Tabel 1.3 Saturday Data Skenario dan Profit

Scenario	Trip ID	Destination	Profit
Saturday	0	(-7.75871107773189, 110.39134815553926)	20000000
Saturday	1	(-7.818683948045408, 110.38043422876682)	10000000
Saturday	2	(-7.819162310093343, 110.37643519677164)	10000000
Saturday	3	(-7.793072124436985, 110.36907039943321)	10000000
Saturday	4	(-7.815685814372376, 110.34690484321186)	30000000
Saturday	5	(-7.786874516259785, 110.36954395081311)	30000000
Saturday	6	(-7.779588526556111, 110.37001583609423)	30000000
Saturday	7	(-7.814312023834777, 110.39423259841826)	10000000
Saturday	8	(-7.786823838943322, 110.36906116005858)	30000000
Saturday	9	(-7.795136348660938, 110.33049748925366)	10000000
Saturday	10	(-7.753613497144761, 110.32635997594211)	10000000
Saturday	11	(-7.820813850628063, 110.3371846465614)	20000000
Saturday	12	(-7.7607050501917065, 110.32083135006967)	30000000
Saturday	13	(-7.794864016923349, 110.32354010961637)	20000000
Saturday	14	(-7.7811530039246355, 110.37023645423197)	30000000
Saturday	15	(-7.7792830658680945, 110.37132394967563)	20000000
Saturday	16	(-7.757683191947154, 110.32297643633814)	30000000
Saturday	17	(-7.78220799361297, 110.37915371308083)	30000000
Saturday	18	(-7.798025226732104, 110.32777940593454)	10000000
Saturday	19	(-7.817076175571514, 110.37979932423605)	30000000
Saturday	20	(-7.777854216274366, 110.37343442033261)	10000000
Saturday	21	(-7.791149772363954, 110.3702851624637)	10000000
Saturday	22	(-7.785429750117955, 110.36018833237053)	30000000
Saturday	23	(-7.787393884777669, 110.36406566304566)	20000000
Saturday	24	(-7.811308255015415, 110.34260012329163)	30000000
Saturday	25	(-7.813860746612476, 110.3918175913882)	10000000
Saturday	26	(-7.81301638041292, 110.34033159647704)	30000000
Saturday	27	(-7.767798790112538, 110.41504464936352)	20000000
Saturday	28	(-7.782394523032584, 110.3670622693528)	10000000
Saturday	29	(-7.7838892603336225, 110.37427530337268)	30000000

Tabel 1.4 Sunday Data Skenario dan Profit

Scenario	Trip ID	Destination	Profit
Sunday	0	(-7.754752211101029, 110.31954432750135)	30000000
Sunday	1	(-7.801083456349155, 110.41582559786396)	10000000
Sunday	2	(-7.786741537607359, 110.36497806889548)	20000000
Sunday	3	(-7.77526140556621, 110.36610537165039)	10000000
Sunday	4	(-7.79793487443431, 110.36460546575)	10000000
Sunday	5	(-7.779393206442219, 110.37283933382679)	30000000
Sunday	6	(-7.765079428939965, 110.40791648049658)	30000000
Sunday	7	(-7.766483431597694, 110.36441998748231)	20000000
Sunday	8	(-7.772728610153513, 110.4081503856958)	10000000
Sunday	9	(-7.786007661386255, 110.3632971806047)	10000000
Sunday	10	(-7.759109290071894, 110.3277243573456)	10000000
Sunday	11	(-7.762074840637407, 110.4150550488201)	10000000
Sunday	12	(-7.801513788859254, 110.32921371250177)	10000000
Sunday	13	(-7.805540404637923, 110.39088484416312)	20000000
Sunday	14	(-7.783618490223531, 110.36677941388547)	10000000
Sunday	15	(-7.77096639688936, 110.41728666003176)	30000000
Sunday	16	(-7.767945092577382, 110.3727601660306)	30000000
Sunday	17	(-7.781030790251795, 110.37113705794233)	30000000
Sunday	18	(-7.777079017004526, 110.3704256347233)	30000000
Sunday	19	(-7.783838142517056, 110.36690204433873)	30000000
Sunday	20	(-7.783720084159466, 110.36639890659504)	20000000
Sunday	21	(-7.767531353710349, 110.36535251914106)	10000000
Sunday	22	(-7.760215186126595, 110.38419678669128)	20000000
Sunday	23	(-7.8204016238634235, 110.37917718252952)	30000000
Sunday	24	(-7.783875491808733, 110.3759394151611)	10000000
Sunday	25	(-7.792707835752402, 110.32539392047775)	10000000
Sunday	26	(-7.798172099226585, 110.3650017074385)	10000000
Sunday	27	(-7.759285126664732, 110.31979258704025)	10000000
Sunday	28	(-7.791043820302062, 110.36105716198279)	30000000
Sunday	29	(-7.8035972514521195, 110.4128934623853)	10000000

Tabel 1.5 Weekday Data Skenario Lainnya

Scenario	Trip ID	Start Time	End Time	Battery Consumption	Probability
Weekdays	0	5	8	0.062217669587698386	0.3
Weekdays	1	0	2	0.10701353654407111	0.3
Weekdays	2	0	2	0.11670451517626136	0.3
Weekdays	3	2	4	0.08936807656614155	0.3
Weekdays	4	6	8	0.20747699405596637	0.3
Weekdays	5	11	14	0.024080815488423665	0.3
Weekdays	6	2	3	0.10142192376549508	0.3
Weekdays	7	7	10	0.054632223929739564	0.3
Weekdays	8	7	9	0.12814905906809582	0.3
Weekdays	9	6	7	0.019610493595530708	0.3
Weekdays	10	9	12	0.2154004464729302	0.3
Weekdays	11	7	9	0.08832422022992438	0.3
Weekdays	12	5	8	0.19008947396952916	0.3
Weekdays	13	0	2	0.11255325564876342	0.3
Weekdays	14	6	9	0.0231471433421849	0.3
Weekdays	15	11	13	0.09379349819987594	0.3
Weekdays	16	2	5	0.17889024517920643	0.3
Weekdays	17	9	12	0.13330727442589083	0.3
Weekdays	18	11	12	0.014907193928993319	0.3
Weekdays	19	11	14	0.2132098621812017	0.3
Weekdays	20	4	7	0.03701767027148605	0.3
Weekdays	21	7	10	0.06093130705466953	0.3
Weekdays	22	4	6	0.07946732982412702	0.3
Weekdays	23	8	11	0.05693256159547741	0.3
Weekdays	24	5	6	0.08278664879613755	0.3
Weekdays	25	4	5	0.12421131504639817	0.3
Weekdays	26	1	2	0.010783588598240176	0.3
Weekdays	27	5	8	0.04084386509049799	0.3
Weekdays	28	1	4	0.11855108116108053	0.3
Weekdays	29	10	11	0.05977798087267876	0.3

Tabel 1.6 Saturday Data Skenario Lainnya

Scenario	Trip ID	Start Time	End Time	Battery Consumption	Probability
Saturday	0	1	3	0.0704929393661057	0.34
Saturday	1	11	12	0.1408308207340015	0.34
Saturday	2	4	5	0.07657157502211483	0.34
Saturday	3	4	5	0.0762383045030205	0.34
Saturday	4	7	10	0.12974420975932455	0.34
Saturday	5	8	11	0.0900320649903682	0.34
Saturday	6	3	6	0.014910720550740638	0.34
Saturday	7	11	12	0.10955778541781142	0.34
Saturday	8	11	14	0.019547159811202117	0.34
Saturday	9	10	11	0.0812661837131828	0.34
Saturday	10	9	10	0.18976945572060308	0.34
Saturday	11	7	9	0.20400287179010476	0.34
Saturday	12	8	11	0.11797275813721532	0.34
Saturday	13	2	4	0.04809157471702315	0.34
Saturday	14	1	4	0.10891074821327364	0.34
Saturday	15	10	12	0.023010295084486324	0.34
Saturday	16	7	10	0.10993672671294834	0.34
Saturday	17	10	13	0.0604823726783922	0.34
Saturday	18	1	2	0.18857674489125542	0.34
Saturday	19	1	4	0.06228447748438064	0.34
Saturday	20	4	5	0.050296854884397176	0.34
Saturday	21	11	12	0.11051186098286486	0.34
Saturday	22	0	3	0.017195518088264857	0.34
Saturday	23	9	11	0.023689490470052817	0.34
Saturday	24	0	3	0.11284957581239363	0.34
Saturday	25	8	9	0.08603452037998166	0.34
Saturday	26	5	8	0.010175224265666812	0.34
Saturday	27	6	8	0.055283163869471144	0.34
Saturday	28	9	10	0.002346555740345217	0.34
Saturday	29	10	13	0.13178957872592567	0.34

Tabel 1.7 Sunday Data Skenario Lainnya

Scenario	Trip ID	Start Time	End Time	Battery Consumption	Probability
Sunday	0	8	11	0.11069695699910045	0.36
Sunday	1	5	6	0.12681471196583274	0.36
Sunday	2	3	5	0.08151117212498699	0.36
Sunday	3	8	9	0.0950354466301667	0.36
Sunday	4	4	5	0.058674640322312115	0.36
Sunday	5	10	13	0.035824898097683106	0.36
Sunday	6	5	8	0.09061648088181894	0.36
Sunday	7	5	7	0.08849772530931686	0.36
Sunday	8	2	3	0.048885312447122145	0.36
Sunday	9	5	6	0.060853959747676746	0.36
Sunday	10	6	7	0.11042185401585164	0.36
Sunday	11	10	11	0.13271147208429795	0.36
Sunday	12	3	4	0.09772358246522068	0.36
Sunday	13	3	5	0.0452649216520342	0.36
Sunday	14	2	3	0.02348088579620471	0.36
Sunday	15	11	14	0.11782245056011577	0.36
Sunday	16	3	6	0.0038417288463018185	0.36
Sunday	17	8	11	0.02799553068716554	0.36
Sunday	18	1	4	0.02975269065720625	0.36
Sunday	19	4	7	0.10138149694701239	0.36
Sunday	20	7	9	0.009730430740964337	0.36
Sunday	21	8	9	0.03074333233255257	0.36
Sunday	22	3	5	0.05669477116352015	0.36
Sunday	23	10	13	0.12327249182452929	0.36
Sunday	24	1	2	0.029401556030399765	0.36
Sunday	25	8	9	0.012037544070469473	0.36
Sunday	26	10	11	0.07528869871973946	0.36
Sunday	27	3	4	0.018731418750935168	0.36
Sunday	28	5	8	0.01597100666376714	0.36
Sunday	29	7	8	0.20327934160326394	0.36

Tabel 1.8 Trip yang Diterima tiap Skenario

Hari Kerja	Sabtu	Minggu
0	0	0
1	2	1
2	3	2
3	4	4
4	5	5
5	6	6
6	7	7
7	8	8
8	9	9
9	10	10
10	11	11
11	12	12
13	13	13
14	14	14
15	15	15
16	16	16
17	17	18
18	18	19
20	19	20
21	20	21
23	21	22
24	22	23
25	23	24
27	24	25
28	25	27
	26	28
	27	29
	28	

LAMPIRAN B

LAMPIRAN PENYELESAIAN SISTEM *ELECTRIC* *BIKE-SHARING* SEDERHANA DENGAN LINGO

MODEL:

! Objective function;

MAX = 0.70 * (Z11 + Z21) + 0.30 * (Z12 + Z22);

! Constraints;

X1 + X2 = 3;

X1 + W11 >= Z11;

X2 + W12 >= Z12;

W11 + W12 = 1;

Z11 <= 3;

Z12 <= 1;

X1 + W21 >= Z21;

X2 + W22 >= Z22;

W21 + W22 = 1;

Z21 <= 1;

Z22 <= 3;

! Non-negativity constraints;

@FREE(X1); @FREE(X2);

@BND(0, W11, 1); @BND(0, W12, 1);

@BND(0, W21, 1); @BND(0, W22, 1);

@BND(0, Z11, 3); @BND(0, Z12, 1);

@BND(0, Z21, 1); @BND(0, Z22, 3);

END

```

Global optimal solution found.
Objective value:                3.700000
Infeasibilities:                0.000000
Total solver iterations:        1
Elapsed runtime seconds:        0.03

Model Class:                    LP

Total variables:                10
Nonlinear variables:            0
Integer variables:              0

Total constraints:              12
Nonlinear constraints:          0

Total nonzeros:                26
Nonlinear nonzeros:            0

```


Variable	Value	Reduced Cost
Z11	3.000000	-0.4000000
Z21	1.000000	-0.7000000
Z12	1.000000	-0.3000000
Z22	2.000000	0.000000
X1	2.000000	0.000000
X2	1.000000	0.000000
W11	1.000000	0.000000
W12	0.000000	0.3000000
W21	0.000000	0.000000
W22	1.000000	-0.3000000

Row	Slack or Surplus	Dual Price
1	3.700000	1.000000
2	0.000000	0.3000000
3	0.000000	-0.3000000
4	0.000000	0.000000
5	0.000000	0.3000000
6	0.000000	0.000000
7	0.000000	0.000000
8	1.000000	0.000000
9	0.000000	-0.3000000
10	0.000000	0.000000
11	0.000000	0.000000
12	1.000000	0.000000

Gambar 2.1 Hasil Model Sistem *Electric Bike-Sharing* Sederhana

LAMPIRAN C

LAMPIRAN PENYELESAIAN SISTEM *ELECTRIC* *BIKE-SHARING* SEDERHANA DENGAN *FIRE HAWK* *OPTIMIZER*

```
1 # -*- coding: utf-8 -*-
2 """skripsi_generating_data.py
3
4 Automatically generated by Colab.
5
6 Original file is located at
7 https://colab.research.google.com/drive/1
   Z9f1G6_zx_vka3b8XYzsEm-HgW7NBfC8
8 """
9
10 import pandas as pd
11 import random
12 import numpy as np
13 from geopy.distance import geodesic
14 import folium
15 import matplotlib.pyplot as plt
16 import pickle
17
18 # Parameters
19 num_stations_options = [15]
20 capacity_min = 5
21 capacity_max = 10
22 alpha_min, alpha_max = 10000000, 50000000
23 beta_min, beta_max = 2000000, 5000000
24 num_scenarios_options = [3]
25 trips_per_scenario_options = [30]
26 Tmax_options = [15]
```

```

27 max_walking_time = 10
28 budget_fraction = 0.8
29
30 # Tugu Yogyakarta as the center
31 tugu_yogyakarta = (-7.7829, 110.3671)
32
33 strategic_locations = [
34     (-7.7891, 110.3614), # Yogyakarta Tugu Train Station
35     (-7.7931, 110.3656), # Malioboro Mall
36     (-7.7824, 110.3750), # Beringharjo Market
37     (-7.7830, 110.3657), # Bank Indonesia Yogyakarta
38     (-7.7801, 110.3712), # Yogyakarta Palace
39     (-7.7764, 110.3632), # UGM ( Universitas Gadjah Mada)
40 ]
41
42 # Scenario probabilities
43 scenario_probabilities = {
44     0: 0.30, # Weekdays
45     1: 0.34, # Saturday
46     2: 0.36 # Sunday
47 }
48
49 def generate_random_coordinates(center, num_points, radius=0.05,
    min_distance=1.0):
50     coords = []
51     while len(coords) < num_points:
52         lat = center[0] + random.uniform(-radius, radius)
53         long = center[1] + random.uniform(-radius, radius)
54         new_coord = (lat, long)
55
56         if all(geodesic(new_coord, existing_coord).kilometers >=
            min_distance for existing_coord in coords +
            strategic_locations):
57             coords.append(new_coord)
58

```

```

59     return [coord[0] for coord in coords], [coord[1] for coord
60           in coords]
61
62 num_stations = random.choice(num_stations_options)
63 num_random_stations = num_stations - len(strategic_locations)
64 random_lats, random_longs = generate_random_coordinates(
65     tugu_yogyakarta, num_random_stations)
66
67 combined_lats = [loc[0] for loc in strategic_locations] +
68     random_lats
69 combined_longs = [loc[1] for loc in strategic_locations] +
70     random_longs
71
72 CHARGING_TIME = 2 # hours
73 MAX_TRIP_DURATION = 3 # hours
74 MAX_BATTERY_DISTANCE = 50 # km
75
76 # Modify station generation
77 stations_data = []
78 for i in range(num_stations):
79     capacity = random.randint(capacity_min, capacity_max)
80     alpha = random.randint(alpha_min, alpha_max)
81     beta = random.randint(beta_min, beta_max)
82     opening_cost = alpha + beta * capacity
83     min_bikes = max(1, int(capacity * 0.2))
84     max_bikes = int(capacity * 0.8)
85     charging_slots = random.randint(1, min(5, capacity))
86     stations_data.append((i, capacity, opening_cost,
87         combined_lats[i], combined_longs[i], min_bikes,
88         max_bikes, charging_slots))
89
90 stations_df = pd.DataFrame(stations_data, columns=['station_id',
91     'capacity', 'opening_cost', 'latitude', 'longitude', '
92     min_bikes', 'max_bikes', 'charging_slots'])
93
94 # Constants
95 WALKING_DISTANCE_THRESHOLD = 0.5

```

```

87
88 # Generate Scenarios
89 num_scenarios = random.choice(num_scenarios_options)
90 Tmax = random.choice(Tmax_options)
91
92 # Function to calculate trip distance
93 def calculate_trip_distance(origin, destination):
94     return geodesic(origin, destination).kilometers
95
96 # Modify scenario generation
97 def generate_initial_scenarios(num_scenarios, num_trips_options,
98     stations_df, Tmax, scenario_probabilities):
99     scenarios_data = []
100     for scenario in range(num_scenarios):
101         num_trips = random.choice(num_trips_options)
102         for trip_id in range(num_trips):
103             while True:
104                 origin_station = stations_df.sample(n=1).iloc[0]
105                 dest_station = stations_df.sample(n=1).iloc[0]
106
107                 origin_lat = origin_station['latitude'] + random
108                     .uniform(-0.006, 0.006)
109                 origin_long = origin_station['longitude'] +
110                     random.uniform(-0.006, 0.006)
111                 dest_lat = dest_station['latitude'] + random.
112                     uniform(-0.006, 0.006)
113                 dest_long = dest_station['longitude'] + random.
114                     uniform(-0.006, 0.006)
115
116                 origin = (origin_lat, origin_long)
117                 destination = (dest_lat, dest_long)
118
119                 trip_distance = calculate_trip_distance(origin,
120                     destination)

```

```

116         if origin != destination and trip_distance <=
            MAX_BATTERY_DISTANCE:
117             sk = random.randint(0, Tmax -
                MAX_TRIP_DURATION - 1)
118             ek = random.randint(sk + 1, min(Tmax, sk +
                MAX_TRIP_DURATION))
119             trip_duration = ek - sk
120
121         if ek <= Tmax:
122             profit = 10000000 * trip_duration
123             battery_consumption = trip_distance /
                MAX_BATTERY_DISTANCE
124             probability = scenario_probabilities[
                scenario]
125             scenarios_data.append((scenario, trip_id
                , origin, destination, sk, ek,
                profit, trip_distance,
                battery_consumption, probability))
                # Add probability here
126         break
127
128     return scenarios_data
129
130 scenarios_data = generate_initial_scenarios(num_scenarios,
        trips_per_scenario_options, stations_df, Tmax,
        scenario_probabilities)
131 scenarios_df = pd.DataFrame(scenarios_data, columns=['scenario',
        'trip_id', 'origin', 'destination', 'start_time', '
        end_time', 'profit', 'distance', 'battery_consumption', '
        probability'])
132
133 def calculate_walking_time(coord1, coord2):
134     distance_km = geodesic(coord1, coord2).kilometers
135     walking_speed_kmph = 5
136     walking_time_min = (distance_km / walking_speed_kmph) * 60
137     return walking_time_min

```

```

138
139 distance_matrix = pd.DataFrame(index=stations_df['station_id'],
    columns=stations_df['station_id'])
140 for i in stations_df['station_id']:
141     for j in stations_df['station_id']:
142         if i != j:
143             distance = calculate_trip_distance(
144                 (stations_df.loc[i, 'latitude'], stations_df.loc
145                  [i, 'longitude']),
146                 (stations_df.loc[j, 'latitude'], stations_df.loc
147                  [j, 'longitude'])
148             )
149             distance_matrix.loc[i, j] = distance
150         else:
151             distance_matrix.loc[i, j] = 0
152
153 def create_potential_stations_and_trips_map(stations_df,
154     scenarios_df):
155     m = folium.Map(location=[tugu_yogyakarta[0], tugu_yogyakarta
156     [1]], zoom_start=14)
157
158     # Add station markers
159     for idx, row in stations_df.iterrows():
160         folium.Marker(
161             location=[row['latitude'], row['longitude']],
162             popup=f"Station■ID:■{row['station_id']},■Capacity:■{
163                 row['capacity']},■Opening■Cost:■IDR■{row['
164                 opening_cost']:,}",
165             icon=folium.Icon(color='blue', icon='info-sign')
166         ).add_to(m)
167
168     # Add trip lines
169     for _, trip in scenarios_df.iterrows():
170         folium.PolyLine(
171             locations=[trip['origin'], trip['destination']],
172             color="red",

```

```

167         weight=2,
168         opacity=0.8,
169         popup=f"Scenario:■{trip['scenario']},■Trip■ID:■{trip
           ['trip_id']}"
170     ).add_to(m)
171
172     return m
173
174     # Save the data
175     stations_df.to_csv('stations_data.csv', index=False)
176     scenarios_df.to_csv('scenarios_data.csv', index=False)
177     distance_matrix.to_csv('distance_matrix.csv')
178
179     with open('other_data.pkl', 'wb') as f:
180         pickle.dump({
181             'num_stations': num_stations,
182             'num_scenarios': num_scenarios,
183             'max_trips': max(scenarios_df.groupby('scenario')['
               trip_id'].count()),
184             'Tmax': Tmax,
185             'budget_fraction': budget_fraction,
186             'charging_time': CHARGING_TIME,
187             'max_trip_duration': MAX_TRIP_DURATION,
188             'max_battery_distance': MAX_BATTERY_DISTANCE,
189             'scenario_probabilities': scenario_probabilities
190         }, f)
191
192     potential_stations_and_trips_map =
193         create_potential_stations_and_trips_map(stations_df,
194           scenarios_df)
195
196     potential_stations_and_trips_map.save("
       potential_stations_and_trips_map.html")

```

LAMPIRAN D

LAMPIRAN SKRIP PROGRAM GENERATING DATA

```
1 # -*- coding: utf-8 -*-
2 """skripsi-example
3
4 Automatically generated by Colab.
5
6 Original file is located at
7 https://colab.research.google.com/drive/13
8   DrLktDF7SXFcqLXMuWfJjyW4s6X6p9r
9 """
10 import numpy as np
11 import random
12 import matplotlib.pyplot as plt
13
14 class SimpleFHO:
15     def __init__(self, pop_size, max_generations):
16         self.pop_size = pop_size
17         self.max_generations = max_generations
18         self.best_solution = self.initialize_solution() #
19             Initialize with a valid solution
20         self.best_fitness = float('-inf')
21         self.fitness_history = []
22
23     def initialize_solution(self):
24         while True:
25             solution = np.zeros(10, dtype=int)
26             solution[:2] = np.random.randint(0, 4, size=2) # x1
27                 and x2
28             solution[2:6] = np.random.randint(0, 2, size=4) # w
29                 variables
```

```

27         solution[6:] = np.random.randint(0, 4, size=4) # z
           variables
28         if self.constraints(solution):
29             return solution
30
31     def objective_function(self, solution):
32         -, -, -, -, -, -, z11, z12, z21, z22 = solution
33         return 0.7 * (z11 + z21) + 0.3 * (z12 + z22)
34
35     def constraints(self, solution):
36         x1, x2, w11, w12, w21, w22, z11, z12, z21, z22 =
           solution
37
38         if x1 + x2 != 3:
39             return False
40         if x1 + w11 < z11 or x2 + w12 < z12:
41             return False
42         if w11 + w12 != 1:
43             return False
44         if z11 > 3 or z12 > 1:
45             return False
46         if x1 + w21 < z21 or x2 + w22 < z22:
47             return False
48         if w21 + w22 != 1:
49             return False
50         if z21 > 1 or z22 > 3:
51             return False
52         if any(v < 0 for v in solution):
53             return False
54
55         return True
56
57     def territories(self, Fire_Hawks, Preys):
58         territories = {i: np.array([]) for i in range(len(
           Fire_Hawks))}
59         preys_left = Preys.copy()

```

```

60     for i in range(len(Fire_Hawks)):
61         D = np.linalg.norm(Fire_Hawks[i] - preys_left, axis
62                             =1)
63         sorted_preys_idx = np.argsort(D)
64         alpha = np.random.randint(1, len(preys_left)) if len
65             (preys_left) > 1 else 1
66         my_preys = sorted_preys_idx[:alpha]
67         territories[i] = preys_left[my_preys]
68         preys_left = preys_left[sorted_preys_idx[alpha:]]
69         if len(preys_left) == 0:
70             break
71     if len(preys_left) > 0:
72         territories[len(Fire_Hawks)-1] = np.concatenate((
73             territories[len(Fire_Hawks)-1], preys_left))
74     return territories
75
76 def optimize(self):
77     print("Initializing■population...")
78     Pop = np.array([self.initialize_solution() for _ in
79                     range(self.pop_size)])
80     fitness = np.array([self.objective_function(sol) for sol
81                         in Pop])
82
83     print(f"Initial■population■size:■{len(Pop)}")
84     print(f"Sample■solution:■{Pop[0]}")
85     print(f"Sample■fitness:■{fitness[0]}")
86
87     best_idx = np.argmax(fitness)
88     self.best_solution = Pop[best_idx]
89     self.best_fitness = fitness[best_idx]
90     print(f"Initial■best■solution:■{self.best_solution}")
91     print(f"Initial■best■fitness:■{self.best_fitness}")
92
93     for generation in range(self.max_generations):
94         print(f"\nGeneration■{generation+1}")

```

```

91
92     num_Hawks = np.random.randint(1, int(self.pop_size
93                                     /5)+1)
94
95     print(f"Number of Fire Hawks: {num_Hawks}")
96
97     # Select fire hawks and preys
98     Fire_Hawks = Pop[np.argsort(fitness)[-num_Hawks:]]
99     Preys = Pop[np.argsort(fitness)[:num_Hawks]]
100
101     print("Fire Hawks:")
102     print(Fire_Hawks)
103     print("Preys:")
104     print(Preys)
105
106     # Print the equation for X
107     print("\nEquation for X:")
108     print("X=[")
109     for i in range(len(Pop)):
110         print(f" {Pop[i]}")
111     print("]")
112
113     territories = self.territories(Fire_Hawks, Preys)
114     print(f"\nNumber of territories: {len(territories)}")
115
116     new_Pop = []
117     for i in territories:
118         FHl = Fire_Hawks[i]
119         PR = territories[i]
120         SPL = PR.mean(axis=0) if len(PR) > 0 else np.
121             zeros(FHl.shape)
122
123         print(f"\nTerritory {i+1}:")
124         print(f" Fire Hawk {FHl[i+1]}: {FHl}")
125         print(f" Number of Preys: {len(PR)}")
126         print(f" Safe Place (SP_{i+1}): {SPL}")

```

```

124
125     # Print distances
126     print("\nDistances:")
127     for k, prey in enumerate(PR):
128         D = np.linalg.norm(FHl - prey)
129         print(f"D_{k+1}_{i+1}={D}")
130
131     # Fire Hawk movement
132     a, b = np.random.uniform(0, 1, size=2)
133     FHNear = Fire_Hawks[np.random.randint(num_Hawks)
134                        ]
135     FHl_new = FHl + (a * self.best_solution - b *
136                    FHNear)
137     FHl_new = np.round(np.clip(FHl_new, 0, 3)).
138                    astype(int)
139     if self.constraints(FHl_new):
140         new_Pop.append(FHl_new)
141         print(f"\nUpdated Fire Hawk (FH_{i+1}^new): {
142               FHl_new}")
143         print(f"FH_{i+1}^new= FH_{i+1}+({a:.2f}*
144              GB-{b:.2f}*FH_near)")
145         print(f"Where GB={self.best_solution} and
146              FH_near={FHNear}")
147
148     # Prey movements (showing only the first prey
149     for brevity)
150     if len(PR) > 0:
151         PRq = PR[0]
152         a, b = np.random.uniform(0, 1, size=2)
153         PRq_new1 = PRq + (a * FHl - b * SP1)
154         PRq_new1 = np.round(np.clip(PRq_new1, 0, 3))
155         .astype(int)
156         if self.constraints(PRq_new1):
157             new_Pop.append(PRq_new1)
158             print(f"\nUpdated Prey (PR_1^new, inside
159                   territory): {PRq_new1}")

```

```

151         print(f"PR_1^new={PR_1}+({a:.2f}*FH_
           {i+1}-{b:.2f}*SP_{i+1})")
152         print(f"Where PR_1={PRq}, FH_{i+1}={
           FHl}, and SP_{i+1}={SPl}")

153
154         a, b = np.random.uniform(0, 1, size=2)
155         FHAlter = Fire_Hawks[np.random.randint(
           num_Hawks)]
156         SP = Pop.mean(axis=0)
157         PRq_new2 = PRq + (a * FHAlter - b * SP)
158         PRq_new2 = np.round(np.clip(PRq_new2, 0, 3))
           .astype(int)
159         if self.constraints(PRq_new2):
160             new_Pop.append(PRq_new2)
161             print(f"\nUpdated Prey (PR_1^new,
           outside territory): {PRq_new2}")
162             print(f"PR_1^new={PR_1}+({a:.2f}*
           FH_Alter-{b:.2f}*SP)")
163             print(f"Where PR_1={PRq}, FH_Alter={
           FHAlter}, and SP={SP}")

164
165         # Ensure population size remains constant
166         while len(new_Pop) < self.pop_size:
167             new_Pop.append(self.initialize_solution())
168         new_Pop = new_Pop[:self.pop_size]
169
170         Pop = np.array(new_Pop)
171         fitness = np.array([self.objective_function(sol) for
           sol in Pop])
172
173         best_idx = np.argmax(fitness)
174         if fitness[best_idx] > self.best_fitness:
175             self.best_solution = Pop[best_idx]
176             self.best_fitness = fitness[best_idx]
177             print(f"New best solution found: {self.
           best_solution}")

```

```

178         print(f"New■best■fitness:■{self.best_fitness}")
179
180         self.fitness_history.append(self.best_fitness)
181
182         print("\nOptimization■completed!")
183         return self.best_solution, self.best_fitness
184
185     def plot_results(self):
186         plt.figure(figsize=(10, 6))
187         plt.plot(range(1, len(self.fitness_history) + 1), self.
188                 fitness_history)
189         plt.title('Best■Fitness■over■Generations')
190         plt.xlabel('Generation')
191         plt.ylabel('Fitness')
192         plt.grid(True)
193         plt.show()
194
195     # Usage
196     fho = SimpleFHO(pop_size=10, max_generations=100)
197     best_solution, best_fitness = fho.optimize()
198     print(f"Best■Solution:■{best_solution}")
199     print(f"Best■Fitness:■{best_fitness}")
200     fho.plot_results()

```

LAMPIRAN E

LAMPIRAN SKRIP PENYELESAIAN ALGORITMA FIRE HAWK OPTIMIZER

```
1 # -*- coding: utf-8 -*-
2 """fork_of_skripsi_v2
3
4 Automatically generated by Colab.
5
6 Original file is located at
7 https://colab.research.google.com/drive/19qzJXWbgeGK040cSKp-
  AqNA52LqEJVZ2
8 """
9
10 import pandas as pd
11 import numpy as np
12 import random
13 from geopy.distance import geodesic
14 import folium
15 import pickle
16 import matplotlib.pyplot as plt
17 import math
18
19 # Load data
20 stations_df = pd.read_csv('/kaggle/input/skripsi-generating-data
  /stations_data.csv')
21 scenarios_df = pd.read_csv('/kaggle/input/skripsi-generating-
  data/scenarios_data.csv')
22 distance_matrix = pd.read_csv('/kaggle/input/skripsi-generating-
  data/distance_matrix.csv', index_col=0)
23
```

```

24 with open('/kaggle/input/skripsi-generating-data/other_data.pkl'
25           , 'rb') as f:
26
27     num_stations = other_data['num_stations']
28     num_scenarios = other_data['num_scenarios']
29     max_trips = max(scenarios_df.groupby('scenario').size())
30     Tmax = other_data['Tmax']
31     budget_fraction = other_data['budget_fraction']
32
33     scenarios_df['origin'] = scenarios_df['origin'].apply(eval)
34     scenarios_df['destination'] = scenarios_df['destination'].apply(
35         eval)
36
37     scenario_probabilities = {
38         0: 0.30, # Weekdays
39         1: 0.34, # Saturday
40         2: 0.36 # Sunday
41     }
42
43     def calculate_distance(coord1, coord2):
44         return geodesic(coord1, coord2).kilometers
45
46     class BikeShareFHO:
47         def __init__(self, stations_df, scenarios_df,
48                     distance_matrix, num_stations, num_scenarios, max_trips,
49                     Tmax, budget_fraction, pop_size, max_generations,
50                     other_data):
51             self.stations_df = stations_df
52             self.scenarios_df = scenarios_df
53             self.distance_matrix = distance_matrix
54             self.num_stations = num_stations
55             self.num_scenarios = num_scenarios
56             self.max_trips = max_trips
57             self.Tmax = Tmax
58             self.budget_fraction = budget_fraction

```

```

55     self.pop_size = pop_size
56     self.max_generations = max_generations
57     self.scenario_probabilities = scenario_probabilities
58     self.bike_cost = 4000000
59
60     self.charging_time = other_data['charging_time']
61     self.max_trip_duration = other_data['max_trip_duration']
62     self.max_battery_distance = other_data['
        max_battery_distance']
63
64     self.charging_rate = 1 / self.charging_time
65
66     self.charging_status = np.zeros((num_scenarios, 75, Tmax
        ))
67
68     self.n_dims = num_stations + 75 + num_scenarios *
        max_trips
69     self.Pop = np.random.randint(0, 2, (pop_size, self.
        n_dims))
70     for solution in self.Pop:
71         if sum(solution[:self.num_stations]) == 0:
72             solution[np.random.randint(0, self.num_stations)
                ] = 1
73     self.best_costs = []
74     self.costs_iter = []
75     self.best_solution = None
76     self.path = []
77
78     def calculate_distance(self, coord1, coord2):
79         return geodesic(coord1, coord2).kilometers
80
81     def objective_function(self, solution):
82         y = solution[:self.num_stations]
83         b = solution[self.num_stations:self.num_stations+75]
84         x = solution[self.num_stations+75:].reshape(self.
            num_scenarios, self.max_trips)

```

```

85
86     # Calculate expected profit
87     expected_profit = 0
88     for sc in range(self.num_scenarios):
89         scenario_prob = scenario_probabilities[sc] # Use
90             the defined probabilities
91         scenario_trips = self.scenarios_df[self.scenarios_df
92             ['scenario'] == sc]
93         num_trips_in_scenario = len(scenario_trips)
94         scenario_profit = sum(scenario_trips['profit'].iloc
95             [:min(num_trips_in_scenario, self.max_trips)] *
96             x[sc, :min(
97                 num_trips_in_scenario,
98                 self.max_trips)])
99         expected_profit += scenario_prob * scenario_profit
100
101     # Calculate costs
102     station_costs = np.dot(self.stations_df['opening_cost'],
103         y)
104     bike_costs = self.bike_cost * np.sum(b)
105
106     # Calculate net profit
107     net_profit = expected_profit - station_costs -
108         bike_costs
109
110     # Check constraints
111     if not self.constraints(solution):
112         return 0 # Return 0 if constraints are violated
113
114     return max(0, net_profit) # Ensure non-negative return
115         value
116
117 def constraints(self, solution):
118     y = solution[:self.num_stations]
119     b = solution[self.num_stations:self.num_stations+75]

```

```

112     x = solution[self.num_stations+75:].reshape(self.
113           num_scenarios, self.max_trips)
114
115     # Budget constraint
116     total_cost = np.dot(self.stations_df['opening_cost'], y)
117     + self.bike_cost * np.sum(b)
118     budget = self.budget_fraction * (np.sum(self.stations_df
119     ['opening_cost']) + self.bike_cost * 75)
120     if total_cost > budget:
121         return False
122
123     total_bikes = np.sum(b)
124
125     for sc in range(self.num_scenarios):
126         scenario_trips = self.scenarios_df[self.scenarios_df
127         ['scenario'] == sc]
128         num_trips_in_scenario = len(scenario_trips)
129         battery_levels = np.ones(75)
130
131         for t in range(self.Tmax):
132             # Check trips starting at this time
133             starting_trips = scenario_trips[scenario_trips['
134             start_time'] == t]
135             for idx, trip in starting_trips.iterrows():
136                 if idx < self.max_trips and x[sc, idx] == 1:
137                     # If this trip is accepted and within
138                     max_trips
139                     available_bikes = np.where((b == 1) & (
140                     battery_levels >= trip['
141                     battery_consumption']))[0]
142                     if len(available_bikes) == 0:
143                         return False # No bike available
144                         with sufficient charge
145                     chosen_bike = available_bikes[0]
146                     battery_levels[chosen_bike] -= trip['
147                     battery_consumption']

```

```

137
138         # Charge bikes
139         battery_levels = np.minimum(1, battery_levels +
140                                     self.charging_rate)
141
142         # Capacity constraint
143         for st in range(self.num_stations):
144             station_usage = np.sum(x[sc, scenario_trips['
145                                     origin'] == st]) + np.sum(x[sc,
146                                     scenario_trips['destination'] == st])
147             if station_usage > self.stations_df.loc[st, '
148                 capacity'] * y[st]:
149                 return False
150
151         # Vehicle allocation constraint
152         if np.sum(x[sc]) > total_bikes:
153             return False
154
155         # Trip allocation constraint
156         if np.any(x[sc] > 1):
157             return False
158
159         # Flow conservation constraint
160         for st in range(self.num_stations):
161             incoming_flow = np.sum(x[sc, scenario_trips['
162                 destination'] == st])
163             outgoing_flow = np.sum(x[sc, scenario_trips['
164                 origin'] == st])
165             if incoming_flow != outgoing_flow:
166                 return False
167
168         # Time-dependent constraint
169         for t in range(self.Tmax):
170             bikes_in_use = np.sum(x[sc, (scenario_trips['
171                 start_time'] <= t) & (scenario_trips['
172                 end_time'] > t)])

```

```

165         if bikes_in_use > total_bikes:
166             return False
167
168     # Rebalancing constraint (simplified)
169     max_imbalance = 0.2 * total_bikes
170     for st in range(self.num_stations):
171         net_flow = np.sum(x[sc, scenario_trips['
172             destination'] == st]) - np.sum(x[sc,
173             scenario_trips['origin'] == st])
174
175         if abs(net_flow) > max_imbalance:
176             return False
177
178     # New constraint: Build unaccepted stations if no
179     # station within radius
180     radius = 1
181     stations_coords = self.stations_df[['latitude', '
182         longitude']].values
183
184     # Calculate distances using Haversine formula
185     lat1, lon1 = np.radians(stations_coords[:, 0])[:, np.
186         newaxis], np.radians(stations_coords[:, 1])[:, np.
187         newaxis]
188     lat2, lon2 = np.radians(stations_coords[:, 0]), np.
189         radians(stations_coords[:, 1])
190     dlat, dlon = lat2 - lat1, lon2 - lon1
191     a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np
192         .sin(dlon/2)**2
193     distances = 2 * 6371 * np.arcsin(np.sqrt(a)) # 6371 is
194         Earth's radius in km
195
196     for st in range(self.num_stations):
197         if y[st] == 0: # If station is not built
198             if not np.any((distances[st] <= radius) & (y ==
199                 1)):
200                 y[st] = 1 # Build the station

```

```

191         # Recalculate budget constraint after potential new
            stations
192         if np.dot(self.stations_df['opening_cost'], y) + 50000 *
            np.sum(b) > budget:
193             return False
194
195         return True
196
197     def territories(self, Fire_Hawks, Preys):
198         territories = {i: np.array([]) for i in range(len(
            Fire_Hawks))}
199         preys_left = Preys.copy()
200         for i in range(len(Fire_Hawks)):
201             D = np.linalg.norm(Fire_Hawks[i] - preys_left, axis
                =1)
202             sorted_preys_idx = np.argsort(D)
203             alpha = np.random.randint(1, len(preys_left)) if len
                (preys_left) > 1 else 1
204             my_preys = sorted_preys_idx[:alpha]
205             territories[i] = preys_left[my_preys]
206             preys_left = preys_left[sorted_preys_idx[alpha:]]
207             if len(preys_left) == 0:
208                 break
209             if len(preys_left) > 0:
210                 territories[len(Fire_Hawks)-1] = np.concatenate((
                    territories[len(Fire_Hawks)-1], preys_left))
211         return territories
212
213     def maximize_FHO(self):
214         print("Starting■Fire■Hawk■Optimizer")
215         Pop = self.Pop
216         pop_size = self.pop_size
217         max_generations = self.max_generations
218
219         print(f"Evaluating■initial■population■of■size■{pop_size}
            ")

```

```

220     cost = np.array([self.objective_function(Pop[i]) for i
221                     in range(pop_size)])
222
223     print("Ordering■candidates")
224     Pop = Pop[np.argsort(cost)[::-1]]
225     cost = np.sort(cost)[::-1]
226     SP = Pop.mean(axis=0)
227
228     num_hawks_history = []
229     num_prey_history = []
230
231     GB = cost[0]
232     Best_Hawk = Pop[0]
233     self.path.append(Best_Hawk)
234     print(f"Initial■best■solution:■{GB}")
235
236     FEs = pop_size
237
238     # Convergence parameters
239     convergence_threshold = 1
240     patience = 50
241     min_iterations = 50
242     no_improvement_counter = 0
243     best_cost_history = [GB]
244
245     print(f"Starting■main■loop■for■up■to■{max_generations}■
246           generations")
247
248     for generation in range(max_generations):
249         print(f"\nGeneration■{generation+1}")
250
251         num_Hawks = np.random.randint(1, int(pop_size/5)+1)
252         num_Prey = pop_size - num_Hawks
253
254         print(f"Number■of■Fire■Hawks:■{num_Hawks},■Number■of
255               ■Prey:■{num_Prey}")
256         num_hawks_history.append(num_Hawks)

```

```

253     num_prey_history.append(num_Prey)
254
255     Fire_Hawks = Pop[:num_Hawks]
256     Preys = Pop[num_Hawks:]
257
258     print("Getting territories")
259     territories = self.territories(Fire_Hawks, Preys)
260
261     Pop_Tot = []
262     cost = []
263
264     print("Moving Fire Hawks and Preys")
265     for i in territories:
266         PR = territories[i].copy()
267         FHl = Fire_Hawks[i].copy()
268         SP1 = PR.mean(axis=0) if len(territories[i]) > 0
269             else np.zeros(FHl.shape)
270         a, b = np.random.uniform(0, 1, size=2)
271         FHNear = Fire_Hawks[np.random.randint(num_Hawks)
272             ]
273         FHl_new = FHl + (a * GB - b * FHNear)
274         FHl_new = np.clip(FHl_new, 0, 1)
275         Pop_Tot.append(FHl_new)
276
277         for q in range(len(PR)):
278             a, b = np.random.uniform(0, 1, size=2)
279             PRq_new1 = PR[q].copy() + ((a * FHl - b *
280                 SP1))
281             PRq_new1 = np.clip(PRq_new1, 0, 1)
282             Pop_Tot.append(PRq_new1)
283
284             a, b = np.random.uniform(0, 1, size=2)
285             FHAlter = Fire_Hawks[np.random.randint(
286                 num_Hawks)]
287             PRq_new2 = PR[q].copy() + ((a * FHAlter - b
288                 * SP))

```

```

284         PRq_new2 = np.clip(PRq_new2, 0, 1)
285         Pop_Tot.append(PRq_new2)
286
287     print("Converting■to■binary■and■evaluating■cost")
288     Pop_Tot = np.round(np.array(Pop_Tot)).astype(int)
289     for i in range(len(Pop_Tot)):
290         cost.append(self.objective_function(Pop_Tot[i]))
291         FEs += 1
292
293     print("Creating■new■population")
294     order_idx = np.argsort(cost)[::-1]
295     cost = np.sort(cost)[::-1]
296     Pop_Tot = Pop_Tot[order_idx]
297
298     # Ensure population size remains constant
299     Pop = Pop_Tot[:pop_size]
300     cost = cost[:pop_size]
301
302     print(f"Current■best■solution:■{cost[0]}")
303     if cost[0] > GB:
304         Best_Position = Pop[0]
305         GB = cost[0]
306         self.best_costs.append(GB)
307         self.best_solution = Pop[0]
308         self.path.append(Best_Position)
309         print(f"New■best■solution■found:■{GB}")
310         no_improvement_counter = 0
311     else:
312         self.best_costs.append(GB)
313         no_improvement_counter += 1
314
315     self.costs_iter.append(cost[0])
316     best_cost_history.append(GB)
317
318     # Check for convergence
319     if generation >= min_iterations:

```

```

320         relative_improvement = (best_cost_history[-1] -
                                   best_cost_history[-patience-1]) /
                                   best_cost_history[-patience-1]
321         if relative_improvement < convergence_threshold:
322             print(f"Converged after {generation+1} generations")
323             break
324
325         if no_improvement_counter >= patience:
326             print(f"No improvement for {patience} generations. Stopping.")
327             break
328
329         print("\nOptimization completed")
330         print(f"Best solution found: {GB}")
331
332         self.num_hawks_history = num_hawks_history
333         self.num_preys_history = num_preys_history
334
335         return (GB, self.best_solution)
336
337     def plot_results(self):
338         plt.figure(figsize=(12, 8))
339
340         # Plot cost over iterations
341         plt.subplot(2, 1, 1)
342         plt.plot(range(len(self.costs_iter)), self.costs_iter,
343                  label='Iteration Objective Function')
344         plt.plot(range(len(self.best_costs)), self.best_costs,
345                  label='Best Objective Function')
346         plt.title('Objective Function Iterations')
347         plt.xlabel('Iteration')
348         plt.ylabel('Objective Function')
349         plt.legend()

```

Plot number of Fire Hawks and Prey over iterations

```

350     plt.subplot(2, 1, 2)
351     plt.plot(range(len(self.num_hawks_history)), self.
              num_hawks_history, label='Fire■Hawks')
352     plt.plot(range(len(self.num_pre_y_history)), self.
              num_pre_y_history, label='Prey')
353     plt.title('Number■of■Fire■Hawks■and■Prey■over■Iterations
              ')
354     plt.xlabel('Iteration')
355     plt.ylabel('Number')
356     plt.legend()
357
358     plt.tight_layout()
359     plt.show()
360
361     pop_size = 250
362     max_generations = 100
363
364     fho = BikeShareFHO(stations_df, scenarios_df, distance_matrix,
              num_stations, num_scenarios,
365                      max_trips, Tmax, budget_fraction, pop_size,
                      max_generations, other_data)
366     best_cost, best_solution = fho.maximize_FHO()
367
368     # Extract results
369     y = best_solution[:num_stations]
370     b = best_solution[num_stations:num_stations+75]
371     x = best_solution[num_stations+75:].reshape(num_scenarios,
              max_trips)
372
373     print("\nFinal■Results:")
374     print("Optimal■station■locations:", y)
375     print("Number■of■bikes■purchased:", sum(b))
376     print("Expected■profit:", best_cost)
377
378     # Additional analysis
379     opened_stations = [i for i in range(num_stations) if y[i] == 1]

```

```

380 print("\nDetailed■Analysis:")
381 print("Opened■stations:", opened_stations)
382 print("Total■capacity■of■opened■stations:", sum(stations_df.loc[
    opened_stations, 'capacity']))
383 print("Total■charging■slots■of■opened■stations:", sum(
    stations_df.loc[opened_stations, 'charging_slots']))
384
385 for sc in range(num_scenarios):
386     accepted_trips = sum(x[sc])
387     total_trips = len(scenarios_df[scenarios_df['scenario'] ==
        sc])
388     scenario_name = ["Weekdays", "Saturday", "Sunday"][sc]
389     print(f"Scenario■{sc}■({scenario_name},■prob:■{
        scenario_probabilities[sc]:.2f}):■Accepted■trips:■{
        accepted_trips}/{total_trips}■({accepted_trips /
        total_trips:.2%})")
390
391 # Calculate and print the average utilization of bikes
392 total_trips = sum(sum(x[sc]) for sc in range(num_scenarios))
393 avg_utilization = total_trips / (sum(b) * num_scenarios)
394 print(f"\nAverage■bike■utilization:■{avg_utilization:.2%}")
395
396 # Plot the results
397 fho.plot_results()
398
399 from folium.plugins import MarkerCluster
400
401 # Create a map centered on Yogyakarta
402 m = folium.Map(location=[-7.7956, 110.3695], zoom_start=13)
403
404 # Add markers for built stations
405 for idx, row in stations_df.iterrows():
406     if y[idx] == 1:
407         folium.Marker(
408             location=[row['latitude'], row['longitude']],
409             popup=f"Station■{idx}:■Capacity■{row['capacity']}",

```

```

410         icon=folium.Icon(color='red', icon='info-sign')
411     ).add_to(m)
412
413     # Create a MarkerCluster for trip visualization
414     trip_cluster = MarkerCluster().add_to(m)
415
416     # Visualize accepted trips for each scenario
417     for scenario in range(num_scenarios):
418         scenario_trips = scenarios_df[scenarios_df['scenario'] ==
419                                         scenario]
420         for trip_id in range(len(scenario_trips)):
421             if x[scenario][trip_id] == 1:
422                 trip = scenario_trips.iloc[trip_id]
423                 origin = trip['origin']
424                 destination = trip['destination']
425
426                 # Draw a line for the trip
427                 folium.PolyLine(
428                     locations=[origin, destination],
429                     color="blue",
430                     weight=2,
431                     opacity=0.8
432                 ).add_to(trip_cluster)
433
434                 # Add markers for origin and destination
435                 folium.Marker(
436                     location=origin,
437                     popup=f"Origin: ■Trip■{trip_id}, ■Scenario■{
438                         scenario}",
439                     icon=folium.Icon(color='green', icon='play')
440                 ).add_to(trip_cluster)
441
442                 folium.Marker(
443                     location=destination,
444                     popup=f"Destination: ■Trip■{trip_id}, ■Scenario■{
445                         scenario}",

```

```

443         icon=folium.Icon(color='orange', icon='stop')
444     ).add_to(trip_cluster)
445
446 # Save the map
447 m.save("bike_sharing_optimization_results.html")
448
449 print("Map with optimization results has been saved as '
      bike_sharing_optimization_results.html'")
450
451 # Print accepted trips for each scenario
452 for scenario in range(num_scenarios):
453     print(f"\nAccepted trips for Scenario {scenario}:")
454     scenario_trips = scenarios_df[scenarios_df['scenario'] ==
      scenario]
455     for trip_id in range(len(scenario_trips)):
456         if x[scenario][trip_id] == 1:
457             trip = scenario_trips.iloc[trip_id]
458             print(f"Trip {trip_id}: From {trip['origin']} to {
      trip['destination']},
459                 Start time: {trip['start_time']}, End time: {
      trip['end_time']},
460                 Profit: {trip['profit']}")

```